Código Sintético

Fundamentos — Arquitectura, economía y ética de los sistemas multi-agente en producción

Sergio Perez Ruiz

2026

Título: Código sintético, Fundamentos: Arquitectura, economía y ética de los sistemas multi-agente en producción

Serie: Código Sintético (Volumen 1 de 4)

Autor: Sergio Perez Ruiz

Editor: Sergio Perez Ruiz Jr. Juan Guillermo Moore 357, Santa Anita - Lima, Perú

Primera edición digital, 2026

ISBN (edición digital / EPUB): [ISBN PENDIENTE] ISBN (edición digital / PDF): [ISBN PENDIENTE]

Depósito Legal N° [PENDIENTE]

Libro disponible en: codigosintetico.specsolid.com

Índice

Guía de lectura y mapa de prerrequisitos **1**
Dos rutas de lectura 1
Mapa de prerrequisitos por capítulo 1
Por qué no se dividió el volumen en más entregas 2

Nota sobre vigencia tecnológica **3**

Cuaderno de lectura crítica **5**
1. Rúbrica para evaluar este volumen 5
2. Plantilla para redactar una crítica 6
3. Preguntas de discusión por capítulo 7
Capítulo 1. El despertar agéntico 7
Capítulo 2. El AI Engineer 7
Capítulo 3. Platform Engineering en la era agéntica . 8
Capítulo 4. Fundamentos técnicos del agente moderno 8
Capítulo 5. La Empresa Sintética 8
Capítulo 6. DDD como sistema inmunológico 9
Capítulo 7. Creación de skills 9
Capítulo 8. Patrones de orquestación multi-agente . . 9
Capítulo 9. MCP: el protocolo que une el mundo agéntico 10
Capítulo 10. La Arquitectura del Contexto 10
4. Autoevaluación de comprensión 10
5. Cómo citar y discutir este volumen 11

Glosario operativo del Volumen 1 **13**

Índice de consulta rápida: problema → dónde acudir **19**
1. Por síntoma o pregunta 19
2. Índice de ejercicios resueltos del volumen 20

3. Qué hacer si no encuentra la respuesta aquí 21

Capítulo 1: El despertar agéntico **23**
La transición del desarrollador tradicional al orquestador de agentes 23
Qué es, formalmente, un agente 24
Reactivo y proactivo: las dos mitades de la agencia . . 25
El espectro de autonomía 26
Ejercicio resuelto 1.1: ubique cada sistema en el espectro de autonomía 28
Lo esencial del capítulo 1 29

Capítulo 2: El AI Engineer: el perfil más valioso de la próxima década **31**
Un nuevo rol emerge 31
La matriz de diferenciación 31
Las siete competencias núcleo del AI Engineer 32
Demostrar, no acumular: el manifiesto práctico 35
Ejercicio resuelto 2.1: diseñe el programa de conversión de ocho semanas 39
Lo esencial del capítulo 2 40
Preguntas para discusión 40

Capítulo 3: Platform Engineering en la era agéntica **41**
La plataforma como producto interno 41
El AI Gateway: el componente más crítico 42
Gateway multiproveedor 46
Ejercicio resuelto 3.1: añada el corte por proyecto al gateway 48
Lo esencial del capítulo 3 49
Preguntas para discusión 49

Capítulo 4: Fundamentos técnicos del agente moderno **51**
De la cadena de texto al sistema cognitivo 51
El patrón ReAct: la base de todo agente 52
Más allá de ReAct: la familia de patrones de razonamiento 54
La ventana de contexto no es memoria 55
Razonamiento extendido: la nueva frontera del agente deliberativo 56
Resiliencia en agentes de producción: el arnés como sistema de control 59

Ejercicio resuelto 4.1: diseñe el arnés de producción 61
El Código Limpio Sintético: los principios que perduran . . 62
SOLID como interrogatorio: las preguntas que refactorizan 64
Ejercicio resuelto 4.2: dos herramientas, un presupuesto y un error bien diseñado 66
Lo esencial del capítulo 4 68

Capítulo 5: La Empresa Sintética **69**
Aplicando la especialización al código 69
La instrucción del revisor: una lección de evaluación . 73
El prompt maestro: DRY aplicado a las instrucciones . . . 74
Caso de estudio: refactorizar un agente monolítico a Empresa Sintética 76
Ejercicio resuelto 5.1: diseñe el pipeline de triaje de tickets . 77
Lo esencial del capítulo 5 78

Capítulo 6: DDD como sistema inmunológico **81**
Por qué la IA sin dominio genera entropía 81
El diagrama de clases: estructura de un agregado en DDD 82
Del documento al guardarraíl ejecutable 84
Ejercicio resuelto 6.1: escriba el bounded context de un inventario 85
Lo esencial del capítulo 6 86
Preguntas para discusión 87

Capítulo 7: Creación de skills, enseñar procedimientos a los agentes **89**
Skills: el conocimiento procedimental del agente 89
Los cinco patrones de skill más valiosos 91
Skills limpios: plantillas y la checklist de revisión 94
Ejercicio resuelto 7.1: un skill de validación pre-deploy . . 95
Lo esencial del capítulo 7 96

Capítulo 8: Patrones de orquestación multi-agente **97**
La arquitectura de la inteligencia distribuida 97
Los cinco patrones canónicos de orquestación 98
Cómo elegir patrón: el árbol de decisión 102
El ciclo de vida del agente como máquina de estados: rigor técnico de producción 103

Arquitectura empresarial de agentes: la plataforma de producción 108
Orquestadores limpios: los cuatro anti-patrones 112
AOP para agentes: propagación de errores como aspecto transversal 113
Ejercicio resuelto 8.1: elija el patrón 116
Lo esencial del capítulo 8 117

Capítulo 9: MCP: el protocolo que une el mundo agéntico 119
El USB-C de la inteligencia artificial 119
La seguridad de MCP es la seguridad de su organización 124
Herramientas MCP limpias 125
Ejercicio resuelto 9.1: añada la cuarta herramienta 126
El protocolo de admisión al catálogo 127
Lo esencial del capítulo 9 129
Preguntas para discusión 129

Capítulo 10: La Arquitectura del Contexto 131
El corpus de instrucciones también es código 131
La regla de dependencia aplicada a los archivos `.md` 132
Agentes guardianes y prompts maestros 134
Caso de estudio: el onboarding que dejó de leer miles de líneas 135
Límites y buenas prácticas 135
Memoria del Agente: el Ciclo de Vida Observable 136
Lo esencial del capítulo 10 137

Estado del Arte: Sistemas Multi-Agente basados en LLM y el Rol del AI Engineer 139
1. Alcance y método de esta revisión 139
2. Líneas de investigación principales 139
3. Marcos teóricos utilizados en la literatura 141
4. Metodologías aplicadas en los estudios primarios 142
5. Hallazgos recurrentes 142
6. Discrepancias entre autores 143
7. Limitaciones metodológicas de la literatura revisada . . 143
8. Brechas de conocimiento 144
9. Relación con la tesis central del libro 145

Referencias **147**

Guía de lectura y mapa de prerrequisitos

Nota editorial añadida en respuesta a la crítica de "curva de densidad conceptual elevada": este volumen agrupa en diez capítulos temas que, por separado, suelen ocupar libros enteros (arquitectura de agentes, ReAct, Platform Engineering, MCP, DDD y patrones multi-agente). Esta guía no reduce esa densidad, sería deshonesto pretender que se puede simplificar sin perder contenido, pero le da al lector dos formas distintas de atravesarla según su punto de partida.

Dos rutas de lectura

Ruta secuencial completa (recomendada si usted no tiene experiencia previa en arquitectura de microservicios o sistemas distribuidos): lea los diez capítulos en orden. Cada uno asume que ya leyó los anteriores. Calcule una sesión de estudio por capítulo, no una lectura corrida, especialmente para los Capítulos 4 y 8, que son los más densos del volumen.

Ruta exprés (si usted ya domina microservicios, DDD o sistemas distribuidos): puede leer 1 → 4 → 5 → 9 en una primera pasada para obtener el núcleo argumental del libro, y volver después a 2, 3, 6, 7 y 10 como profundización.

Mapa de prerrequisitos por capítulo

Capítulo	Prerrequisito real	Si le resulta denso, relea primero
1. El despertar agéntico	Ninguno	—
2. El AI Engineer	Cap. 1	—
3. Platform Engineering	Nociones de infraestructura/DevOps	Cap. 1
4. Fundamentos técnicos del agente moderno	Cap. 1; ayuda haber programado con APIs de LLM	Cap. 1, es el capítulo más denso del volumen
5. La Empresa Sintética	Cap. 4 (roles y arnés)	Cap. 4
6. DDD como sistema inmunológico	Nociones básicas de diseño orientado a objetos	Cap. 4, 5
7. Creación de skills	Cap. 4 (herramientas y arnés)	Cap. 4
8. Patrones de orquestación multi-agente	Cap. 5 (roles), Cap. 6 (dominio)	Cap. 5, 6, segundo capítulo más denso
9. MCP	Cap. 4 (herramientas)	Cap. 4
10. La Arquitectura del Contexto	Cap. 4, 9	Cap. 4, 9

Por qué no se dividió el volumen en más entregas

Se consideró partir el Capítulo 4 en dos capítulos independientes. Se decidió no hacerlo porque el capítulo forma una unidad argumental: del modelo cognitivo se deriva directamente el criterio de diseño del arnés, y partirlo habría obligado a repetir contexto. La solución adoptada fue esta guía de lectura y la nota de orientación que abre el propio Capítulo 4.

Nota sobre vigencia tecnológica

Añadida en respuesta a la crítica de "riesgo de obsolescencia tecnológica a corto plazo".

Este libro nombra herramientas, SDKs y modelos concretos porque un libro que solo hablara de principios abstractos sería más cómodo de mantener vigente, pero mucho menos útil para alguien que necesita construir algo esta semana. La decisión editorial fue asumir el costo de la desactualización parcial a cambio de utilidad inmediata, y compensarlo de dos formas:

1. **Separación explícita entre patrón y herramienta.** Cada vez que el texto describe un patrón arquitectónico (el arnés, la Empresa Sintética, el AI Gateway, MCP como protocolo), ese patrón es independiente del proveedor o SDK usado para ilustrarlo.
2. **Lo que sí puede quedar desactualizado, y cómo verificarlo.** Nombres de modelos, precios por token, versiones de SDK y tablas comparativas de herramientas son las secciones con vida útil más corta. Verifique la documentación oficial del proveedor antes de tomar una decisión de arquitectura basada en una cifra concreta de este libro.

Cuaderno de lectura crítica

Para qué sirve este cuaderno: un libro técnico se entiende mejor cuando el lector discute con él. Este cuaderno ofrece tres instrumentos: una rúbrica para evaluar el volumen con criterios explícitos, una plantilla para redactar una crítica fundamentada, y preguntas de discusión por capítulo que convierten cada idea en una decisión que usted puede tomar o rechazar con razones.

1. Rúbrica para evaluar este volumen

Use esta tabla para construir una crítica propia. No se trata de puntuar por gusto, sino de poder justificar el juicio con evidencia del propio texto.

Criterio	La pregunta que usted debe poder responder	Señal de debilidad
Claridad del propósito	¿Puedo enunciar en una frase qué problema resuelve el capítulo y a quién le sirve?	El capítulo mezcla audiencias o promete más de lo que entrega
Solidez de la evidencia	¿Cada afirmación cuantitativa tiene fuente verificable o es una estimación del autor?	Cifras sin fuente o con fuente no cotejable
Trazabilidad	¿Puedo llegar del concepto a su implementación (código, tabla, protocolo) sin saltos?	Se pasa del principio a la conclusión sin el paso intermedio

Criterio	La pregunta que usted debe poder responder	Señal de debilidad
Aplicabilidad	¿Puedo tomar una decisión concreta mañana con lo que el capítulo dice?	Todo queda en «depende» sin criterio de decisión
Vigencia	¿Sé qué parte del capítulo caduca con el próximo release y cómo verificarlo?	Se presentan precios, versiones o modelos como si fueran estables
Reconocimiento de límites	¿El capítulo dice cuándo **no** aplica su recomendación?	Se presenta una solución como universal
Utilidad como manual	¿Puedo volver al capítulo buscando una cosa concreta y encontrarla rápido?	Falta la tabla de decisión, el índice local o el ejercicio resuelto

Cómo se usa en la práctica. Elija tres capítulos, aplique la rúbrica y escriba un juicio de una página. Si su veredicto es «me gustó» o «no me gustó» sin poder señalar la fila de la tabla que lo sostiene, la crítica todavía no está lista.

2. Plantilla para redactar una crítica

1. **Resumen en una frase.** «Este capítulo sostiene que ___ y lo justifica con ___».
2. **Tesis del autor.** Enúnciela con sus palabras, sin ironía. Si no puede, todavía no la entendió.
3. **Evidencia a favor.** La pieza del propio capítulo que mejor sostiene la tesis (tabla, caso, ejercicio resuelto, cita indexada).
4. **Evidencia en contra o hueco.** Lo que el capítulo no cubre, lo que generaliza de más, lo que depende de un contexto distinto al suyo.
5. **Veredicto con destinatario.** No «es bueno» o «es malo», sino «le sirve a ___ para ___ , y falla para ___ porque ___».

Regla de oro. Critique la afirmación, no el tema. «Los agentes

autónomos son peligrosos» no es una crítica; «el capítulo afirma que el nivel 4 de autonomía se justifica solo si el nivel 3 opera bien, pero no ofrece criterio para medir *opera bien*» sí lo es.

3. Preguntas de discusión por capítulo

Las preguntas están ordenadas para llevarlo de la comprensión a la crítica y de ahí a la decisión. No todas tienen respuesta en el texto: las que no la tienen son, precisamente, el espacio donde su criterio importa.

Capítulo 1. El despertar agéntico

1. El capítulo clasifica los sistemas en un espectro de autonomía (niveles 0 a 4). ¿Su organización usa la palabra «agente» con esa misma precisión, o llama agente a lo que aquí es un *workflow*? ¿Qué decisión concreta (presupuesto, pruebas, permisos, quién responde) cambia al reclasificar?
2. El nivel de autonomía se elige por la estructura de la tarea, no por la ambición del equipo. Busque un caso de su entorno que hoy opere un nivel por encima de lo que la tarea exige. ¿Qué coste paga por ese exceso y quién lo asume?
3. Crítica al capítulo: ¿basta la taxonomía propuesta para decidir el nivel, o falta una dimensión (costo del error, reversibilidad, exigencia regulatoria) que usted consideraría indispensable?

Capítulo 2. El AI Engineer

1. De las siete competencias núcleo, ¿cuál no se puede comprar y solo se adquiere con práctica deliberada? ¿Qué evidencia verificable exigiría usted en una contratación para comprobarla en una hora?
2. El capítulo sostiene «demostrar, no acumular». ¿Qué tres piezas presentaría usted hoy? Si no puede, ¿qué le falta y en cuánto tiempo lo consigue?
3. El capítulo afirma que la competencia 7 (orquestación) sin las otras seis «es la receta del incendio distribuido». ¿Está de

acuerdo con ese orden de dependencias? ¿Qué contraejemplo conoce?

Capítulo 3. Platform Engineering en la era agéntica

1. ¿Cuántos equipos de su organización consumen hoy modelos con claves de API propias, fuera de cualquier inventario central? ¿Cuál es el riesgo y quién lo está asumiendo sin saberlo?
2. El AI Gateway centraliza presupuesto, cuotas y auditoría. ¿Qué se pierde al centralizar (autonomía, velocidad, experimentación) y con qué criterio decide el capítulo que la pérdida vale la pena?
3. Crítica al capítulo: ¿es realista exigir un gateway único en una organización con silos fuertes? ¿Qué mínimo viable de gobernanza propone usted si la centralización completa no es viable?

Capítulo 4. Fundamentos técnicos del agente moderno

1. El capítulo distingue ventana de contexto de memoria. En su equipo, ¿hay decisiones de diseño que confunden ambas? ¿Qué fallo concreto produce esa confusión en producción?
2. El arnés es el sistema de control alrededor del modelo. Separe su sistema en dos columnas: lo que es determinista y lo que queda al criterio del modelo. ¿Puede justificar cada elemento de la segunda columna?
3. El capítulo presenta ReAct y su familia de variantes. ¿Qué patrón usa hoy y por qué ese y no otro? ¿Qué evidencia le haría cambiarlo?
4. Crítica al capítulo: es el más denso del volumen y agrupa temas que otros libros tratan por separado. ¿La unidad argumental que el autor defiende como razón para no dividirlo se sostiene, o el lector paga un coste de densidad evitable?

Capítulo 5. La Empresa Sintética

1. ¿Qué ventaja obtiene al especializar sus agentes en roles y qué coste de coordinación paga por ello? ¿Cómo lo mide hoy?

2. El prompt maestro aplica DRY a las instrucciones. ¿Qué riesgo de acoplamiento introduce un prompt maestro único y cómo lo mitigaría?
3. Crítica al capítulo: ¿la metáfora de la «empresa» ilumina o confunde? ¿Qué se gana y qué se pierde al tratar un enjambre como organigrama?

Capítulo 6. DDD como sistema inmunológico

1. ¿Existe en su organización un documento de dominio que un agente pudiera consumir hoy? Si no, ¿quién tiene ese conocimiento en la cabeza y qué pasa el día que se va?
2. Identifique un invariante de su dominio que un agente podría violar sin que ningún verificador determinista lo detecte. ¿Qué capa debería atraparlo?
3. Crítica al capítulo: ¿el vocabulario de DDD (agregado, contexto delimitado, lenguaje ubicuo) explica bien el problema a un equipo sin cultura de diseño orientado a objetos, o se convierte en barrera de entrada?

Capítulo 7. Creación de skills

1. ¿Qué procedimiento de su equipo vive solo en la cabeza de una persona? Conviértalo en skill: ¿cuál es el disparador, qué salida produce y cómo sabría si funcionó?
2. El capítulo insiste en la divulgación progresiva: la descripción está siempre disponible y el contenido se carga solo cuando hace falta. ¿Qué falla cuando la descripción es ambigua?
3. Crítica al capítulo: la checklist de revisión de skills, ¿es verificable por alguien que no escribió el skill, o exige el mismo contexto que el autor?

Capítulo 8. Patrones de orquestación multi-agente

1. De los cinco patrones canónicos, ¿cuál fallaría peor en su dominio y por qué? ¿Qué lo haría fallar: latencia, costo, consistencia o la naturaleza del error?

2. El capítulo exige una política de degradación distinta por patrón. ¿Su sistema distingue «degradar» de «fallar»? ¿Qué evidencia tiene de que la degradación queda documentada y no silenciosa?
3. Revisión crítica: de los cuatro anti-patrones de orquestación que el capítulo describe, ¿reconoce alguno en su arquitectura actual? ¿Cuál y con qué consecuencia observable?

Capítulo 9. MCP: el protocolo que une el mundo agéntico

1. De los diez sistemas internos más consultados por sus equipos (tickets, métricas, documentación, CRM…), ¿cuántos son hoy accesibles para un agente de forma gobernada?
2. Puntúe sus cinco servidores MCP más usados con el protocolo de admisión del capítulo (mantenimiento, seguridad del esquema, versionado, trazabilidad, degradación). ¿Alguno quedaría por debajo del umbral y qué haría al respecto?
3. Crítica al capítulo: ¿el umbral «menos de 4 no se admite, sin excepción de urgencia» es realista en una organización con presión de negocio, o es un ideal que nadie cumple y por tanto no gobierna nada?

Capítulo 10. La Arquitectura del Contexto

1. El corpus de instrucciones también es código: ¿quién lo revisa, quién lo versiona y quién lo prueba en su equipo?
2. La regla de dependencia aplicada a los archivos `.md`: ¿su corpus de prompts depende hacia arriba (hacia lo estable) o hacia abajo (hacia lo cambiante)? Dé un ejemplo concreto.
3. Crítica al capítulo: ¿la analogía «el corpus de instrucciones es código» se sostiene hasta el final, o hay prácticas del código (tests, tipado, cobertura) sin equivalente razonable en un corpus en lenguaje natural?

4. Autoevaluación de comprensión

Marque lo que puede hacer **sin volver al texto**; lo que no marque es su lista de relectura.

- ☐ Explicar la diferencia entre *workflow* y agente con un ejemplo propio.
- ☐ Enunciar las siete competencias del AI Engineer sin leerlas.
- ☐ Justificar por qué el gateway es el componente más crítico de la plataforma.
- ☐ Dibujar el ciclo ReAct y señalar dónde entra el arnés.
- ☐ Distinguir contexto de memoria en un caso real.
- ☐ Nombrar los cinco patrones de orquestación y el fallo típico de cada uno.
- ☐ Aplicar el protocolo de admisión de MCP a un servidor concreto.
- ☐ Decir qué parte de su corpus de instrucciones es dependiente y cuál estable.

5. Cómo citar y discutir este volumen

Al citar, distinga siempre tres cosas: el **principio** (invariante y probablemente duradero), la **implementación** (atada a una herramienta o versión) y el **dato** (cifra o precio, con fecha de caducidad). Una crítica honesta de este volumen separa las tres: se puede estar de acuerdo con el principio, en desacuerdo con la implementación y dudar del dato, sin contradicción. Esa distinción es, en sí misma, la competencia que el libro quiere enseñar.

Glosario operativo del Volumen 1

Definiciones breves, ancladas al capítulo donde el término se desarrolla. Cuando un término tiene acepciones distintas según el volumen de la serie, se indica. El glosario no sustituye al capítulo: sirve para desambiguar una palabra a mitad de la lectura y para fijar el vocabulario común del equipo.

Término	Definición operativa	Capítulo
Agente	Sistema que decide el flujo de sus propios pasos en función del estado de la tarea, no solo el contenido de su salida. Se distingue del *workflow* por quién decide el control.	1, 4
Workflow (**cadena fija**)	Secuencia de pasos predefinida; el modelo rellena contenido, no decide el camino. Se presupuesta, prueba y gobierna distinto que un agente.	1

Término	Definición operativa	Capítulo
Espectro de autonomía	Escala de niveles 0 a 4 que clasifica un sistema según cuánto decide por sí mismo. Se elige por la estructura de la tarea, no por la ambición del equipo.	1
Arnés (***harness***)	Conjunto de controles deterministas alrededor del modelo: presupuesto de iteraciones, herramientas permitidas, validación de salida, corte de bucle. Es lo que hace que un agente de laboratorio sobreviva en producción.	4
ReAct	Patrón de razonamiento-acción-observación que alterna pensamiento y uso de herramientas hasta cumplir el objetivo. Base de la familia de patrones de razonamiento.	4
Ventana de contexto	Espacio finito de tokens disponible en una llamada. No es memoria: se vacía al terminar la interacción si nada la persiste.	4, 10
Memoria del agente	Estado que sobrevive a la llamada y se recupera por algún mecanismo explícito (almacén, resumen, índice). Tiene ciclo de vida observable.	10

Término	Definición operativa	Capítulo
Herramienta (***tool***)	Capacidad expuesta al agente con nombre, descripción y esquema de entrada. La descripción sugiere su uso; el servidor lo impone.	4, 9
Skill	Unidad de conocimiento procedimental con divulgación progresiva: descripción siempre disponible, contenido cargado cuando la tarea lo requiere.	7
Divulgación progresiva	Estrategia de contexto por la que solo se carga el detalle necesario en el momento necesario, para no degradar rendimiento ni costo.	7
Empresa Sintética	Especialización de agentes en roles con responsabilidades acotadas, análoga a una organización, para reducir la entropía de un agente monolítico.	5
Prompt maestro	Instrucción única reutilizada por varios agentes; aplica DRY a los prompts. Introduce acoplamiento que debe gestionarse.	5

Término	Definición operativa	Capítulo
Contexto delimitado (***bounded context***)	Frontera de dominio dentro de la cual un modelo y un lenguaje tienen significado único. Actúa como sistema inmunológico frente a la entropía del modelo.	6
Invariante de dominio	Condición que debe cumplirse siempre en un agregado. Un agente puede violarla sin que un verificador la detecte si el invariante no está explicitado.	6
Patrón de orquestación	Forma canónica de coordinar varios agentes: pipeline, fan-out, *council*, jerárquico y *self-reflection loop*. Cada uno exige su propia política de degradación.	8
Degradación	Operar con calidad o cobertura reducidas pero explícitas, en lugar de fallar. Debe ser visible al consumidor; degradar en silencio es mentir con infraestructura.	3, 8
MCP (**Model Context Protocol**)	Protocolo que reduce la integración de M×N a M+N entre aplicaciones y sistemas, sobre JSON-RPC. Expone *tools*, *resources* y *prompts*.	9

Término	Definición operativa	Capítulo
AI Gateway	Componente de plataforma que centraliza el acceso a modelos: claves, cuotas, presupuesto, caché, auditoría y enrutamiento.	3
Corpus de instrucciones	Conjunto versionado de archivos (`.md`) que gobierna el comportamiento de los agentes. Se trata como código: se revisa, versiona y prueba.	10
Regla de dependencia (en contexto)	Las instrucciones deben depender hacia lo estable, no hacia lo cambiante, igual que en arquitectura limpia.	10
Verificador determinista	Comprobación no probabilística (test, esquema, lint) que valida la salida de un agente. Segunda capa de defensa junto al contexto en el prompt.	6
Eval	Prueba reproducible de calidad de un comportamiento probabilístico; sustituye al test unitario donde el resultado no es binario.	2, 4

Índice de consulta rápida: problema → dónde acudir

Sección de apoyo. Cuando el lector no busca leer de corrido sino resolver algo concreto, esta tabla lo lleva a la parte del volumen que responde. Las referencias «Ej. X.Y» remiten a los ejercicios resueltos.

1. Por síntoma o pregunta

Si usted necesita…	Acuda a	Apoyo directo
Decidir si su sistema es un agente o un *workflow*	Cap. 1, «Qué es, formalmente, un agente»	Ej. 1.1 (espectro de autonomía)
Saber qué perfil contratar y cómo evaluarlo	Cap. 2, «Las siete competencias núcleo»	Ej. 2.1 (programa de ocho semanas)
Centralizar claves, cuotas y auditoría de modelos	Cap. 3, «El AI Gateway»	Ej. 3.1 (corte por proyecto)
Evitar bucles infinitos y controlar el costo por tarea	Cap. 4, «Resiliencia… el arnés como sistema de control»	Ej. 4.1 (diseñe el arnés)
Diseñar una herramienta que un agente no pueda usar mal	Cap. 4, «El error bien diseñado»; Cap. 9, «Herramientas MCP limpias»	Ej. 4.2, Ej. 9.1

Si usted necesita…	Acuda a	Apoyo directo
Repartir trabajo entre varios agentes sin caos	Cap. 5, «Aplicando la especialización al código»	Ej. 5.1 (triaje de tickets)
Que la IA no viole las reglas del negocio	Cap. 6, «El diagrama de clases: un agregado en DDD»	Ej. 6.1 (bounded context)
Convertir un procedimiento del equipo en algo que el agente ejecute	Cap. 7, «Los cinco patrones de skill»	Ej. 7.1 (skill predeploy)
Elegir un patrón de orquestación y su política de fallo	Cap. 8, «Los cinco patrones canónicos»	Ej. 8.1 (elija el patrón)
Conectar el agente con sistemas internos de forma gobernada	Cap. 9, «El protocolo de admisión al catálogo»	Ej. 9.1 (cuarta herramienta)
Ordenar y versionar el corpus de instrucciones	Cap. 10 (arquitectura del contexto)	—
Separar lo determinista de lo probabilístico	Cap. 4 (arnés) y Cap. 6 (verificadores)	Ej. 4.1, Ej. 6.1
Entender qué parte del libro caducará y cómo verificarla	Nota sobre vigencia tecnológica	—

2. Índice de ejercicios resueltos del volumen

Ejercicio	Qué entrena	Capítulo
Ej. 1.1	Clasificar sistemas en el espectro de autonomía	1
Ej. 2.1	Diseñar un programa de conversión de ocho semanas	2
Ej. 3.1	Añadir corte por proyecto al AI Gateway	3

Ejercicio	Qué entrena	Capítulo
Ej. 4.1	Diseñar el arnés de producción	4
Ej. 4.2	Diseñar dos herramientas, un presupuesto y un error	4
Ej. 5.1	Diseñar el pipeline de triaje de tickets	5
Ej. 6.1	Escribir el *bounded context* de un inventario	6
Ej. 7.1	Crear un skill de validación pre-deploy	7
Ej. 8.1	Elegir el patrón de orquestación adecuado	8
Ej. 9.1	Añadir una cuarta herramienta MCP	9

3. Qué hacer si no encuentra la respuesta aquí

Este volumen cubre fundamentos y arquitectura. Si su problema es de **escala y operación** (costo por iteración, testing, DR, métricas), consulte el Vol. 4. Si es de **seguridad, industria o geopolítica**, el Vol. 2. Si es de **organización, ética, equipos y gobierno**, el Vol. 3. El «Mapa de prerrequisitos» de la Guía de lectura indica el orden en que conviene cruzarlos.

Capítulo 1: El despertar agéntico

La transición del desarrollador tradicional al orquestador de agentes

Durante cuatro décadas, la productividad de un desarrollador de software se midió en líneas de código por hora, tickets cerrados por sprint y *pull requests* integrados por semana. Era una métrica industrial aplicada a un trabajo intelectual, y funcionaba a duras penas: la velocidad de producción de código tenía un tope físico, el de la escritura humana.

Esa limitación desapareció.

La secuencia es reconstruible con precisión a partir de la literatura. En 2017, la arquitectura Transformer estableció el sustrato técnico de todo lo que vino después Vaswani et al., 2017. En 2020, los modelos de lenguaje de gran escala demostraron que podían realizar tareas nuevas a partir de unos pocos ejemplos en el contexto, sin reentrenamiento Brown et al., 2020. En 2021, un modelo especializado resolvió problemas de programación descritos en lenguaje natural con una tasa de éxito que sorprendió a sus propios autores, quienes acompañaron el resultado con una discusión explícita de sus riesgos Chen et al., 2021. En 2022, el ajuste con retroalimentación humana convirtió esos modelos en asistentes capaces de seguir instrucciones Ouyang et al., 2022, y se documentó que ciertas capacidades (aritmética en varios pasos, razonamiento encadenado) emergen de forma abrupta con la escala Wei, Tay, et al., 2022.

En 2023 llegó la síntesis decisiva: los modelos aprendieron a *ac-*

tuar. Aprendieron a alternar razonamiento con uso de herramientas externas (Yao, Zhao, et al., 2023; Schick et al., 2023) y a poblar entornos simulados con comportamientos sociales creíbles Park et al., 2023. Entre 2023 y 2024, los marcos multi-agente demostraron que varios modelos coordinados superan a uno solo en tareas de ingeniería de software (Qian et al., 2024; Hong et al., 2024; Wu et al., 2024). Y hacia 2025–2026, los primeros sistemas multi-agente con supervisión humana intermitente alcanzaron entornos de producción empresarial; los proveedores empezaron a publicar guías de arquitectura específicas para construirlos Anthropic, 2024.

El desarrollador cuya única ventaja competitiva es escribir sintaxis correcta enfrenta hoy la situación del operador de telégrafo ante la conmutación automática: su destreza no desapareció, pero dejó de ser el cuello de botella que justificaba su precio. La historia de la ingeniería es, en buena parte, la historia de profesiones que ascendieron en la cadena de abstracción o quedaron fuera de ella.

► **PISTA TÉCNICA**

Qué es, formalmente, un agente

La teoría de agentes autónomos es muy anterior a los LLMs. La *Distributed Artificial Intelligence* (DAI) establece que un sistema es inteligente en proporción a la calidad de las interacciones entre sus componentes, no a la capacidad de cada componente aislado (Ferber, 1999). La definición clásica de Wooldridge (2009) describe al agente como una entidad computacional situada en un entorno, capaz de acción autónoma para alcanzar sus objetivos de diseño. Un agente, en el sentido técnico formal:

- **Percibe** su entorno a través de sensores. En los agentes LLM: el contexto de la conversación, los mensajes del usuario, los resultados de las herramientas que invoca.
- **Razona** sobre ese entorno usando un modelo interno del mundo. En los agentes LLM: el conocimiento paramétrico del modelo más el contexto inyectado.
- **Actúa** sobre el entorno mediante efectores. En los agentes LLM: llamadas a funciones, herramientas, APIs, sistemas de archivos.

- **Aprende** de los resultados para refinar su comportamiento futuro. En los agentes LLM actuales, este aprendizaje ocurre *dentro de la sesión* (vía contexto y memoria externa), no en los pesos del modelo.

Figura 1: Diagrama 1.1: Ciclo de un agente autónomo

Esta definición tiene una consecuencia práctica inmediata: **un chatbot no es un agente**. Un chatbot percibe y razona, pero no actúa sobre el mundo ni cierra el ciclo observando las consecuencias de sus acciones. La frontera entre ambos no es el modelo subyacente, que puede ser el mismo, sino la arquitectura que lo rodea. Anthropic (2024) propone una distinción operativa que adoptaremos en todo el libro: los **flujos de trabajo** (*workflows*) son sistemas donde el código del desarrollador orquesta los pasos y el LLM ejecuta tareas dentro de un guion predefinido; los **agentes** son sistemas donde el propio modelo decide dinámicamente su trayectoria: qué herramienta usar, cuándo iterar, cuándo detenerse. La misma fuente ofrece el consejo de ingeniería más valioso y menos seguido del campo: empiece por el patrón más simple que resuelva el problema, y escale a agente solo cuando la tarea lo justifique.

Reactivo y proactivo: las dos mitades de la agencia

La teoría clásica descompone la agencia en dos propiedades en tensión Wooldridge, 2009. Un agente es **reactivo** cuando percibe los cambios de su entorno y responde a ellos a tiempo: el bug que aparece, el test que se rompe, el mensaje del usuario que corrige el rumbo. Y es **proactivo** cuando no se limita a responder, sino que toma la iniciativa dirigida por un objetivo: descompone la meta, planifica y persigue el plan aunque nadie lo empuje en cada paso. La distinción no es nueva ni exclusiva de los LLMs: refleja la frontera entre los *agentes de reflejo simple* y los *agentes basados en objetivos* de la teoría clásica Russell & Norvig, 2020.

La tensión es real y define el diseño. Un agente **puramente reactivo**, de estímulo y respuesta, sin objetivo propio, nunca emprende

nada por sí mismo: es rápido y robusto, pero no resuelve una tarea de varios pasos. Un agente **puramente proactivo**, que ejecuta su plan sin volver a mirar el mundo, actúa sobre un entorno que ya cambió y persigue, ciego, un objetivo que dejó de tener sentido. La buena ingeniería agéntica vive en el equilibrio, y ese equilibrio ya tiene nombre: es lo que el patrón ReAct (capítulo 4) hace explícito al **intercalar** observación (la mitad reactiva) y razonamiento dirigido a la meta (la mitad proactiva), ciclo a ciclo. Un agente que solo reacciona es un disparador glorificado; uno que solo planifica es un guion que ignora la realidad. El agente útil alterna las dos mitades bajo un presupuesto. Conviene no confundir esta *reactividad del agente* con la *reactividad de los sistemas* (backpressure, flujos, Mono/Flux), que gobierna la infraestructura sobre la que el agente corre: son ideas distintas con el mismo nombre, y la segunda se trata en el capítulo 7 (Vol. 4).

La frontera viva de la industria empuja justamente hacia la mitad proactiva. Lo que en 2024–2025 los equipos de Silicon Valley empezaron a llamar **agentes ambientales** (*ambient agents*) no es una categoría nueva de agente. Son sistemas que corren en segundo plano, se disparan con eventos en lugar de con un *prompt*, y solo reclaman al humano cuando detectan algo que merece su atención Chase, 2025: el agente proactivo llevado a su consecuencia operativa, montado sobre la arquitectura orientada a eventos del capítulo 7 (Vol. 4). La promesa es seductora («guarda tu atención para cuando de verdad importe»), y por eso exige el escrutinio más frío: cada grado de proactividad que el sistema gana es un grado de control que el humano cede, y con él, más superficie de riesgo Mitchell et al., 2025. El capítulo 2 (Vol. 3) retoma esta frontera no como una promesa de marketing, sino como un problema de ingeniería con su propio criterio de diseño, y como el terreno donde los agentes dejan de asistir y empiezan a anticipar.

El espectro de autonomía

Conviene pensar la agencia como un espectro y no como una categoría binaria:

Nivel	Sistema	Quién decide el flujo	Ejemplo
0	Llamada única	El desarrollador	Clasificar un correo
1	Cadena (*workflow*)	El desarrollador	Pipeline extracción → validación → resumen
2	Enrutador	El modelo elige entre rutas fijas	Triaje de tickets a especialistas
3	Agente con herramientas	El modelo, dentro de un presupuesto	Depurar un test que falla
4	Sistema multiagente	Un orquestador + agentes especializados	Desarrollar una funcionalidad completa

Cada nivel hacia abajo en la tabla aumenta la capacidad y, simultáneamente, la varianza del comportamiento, el costo por tarea y la superficie de fallo. La tesis económica de este libro (que desarrollaremos en la Parte III) es que el nivel correcto es el **mínimo** que resuelve el problema, no el máximo que la tecnología permite.

■ **PISTA EJECUTIVA**

Para un director de ingeniería, la transición agéntica tiene una traducción directa al estado de resultados: el costo marginal de *escribir* una unidad de software ha colapsado, pero se ha desplazado hacia la *validación y el gobierno* de lo escrito. La evidencia cuantitativa disponible es consistente: la asistencia de IA acelera de forma medible la tarea de programar (Peng et al., 2023; Ziegler et al., 2024), pero los benchmarks de tareas reales muestran el límite con la misma claridad: cuando se evaluó a los mejores modelos de 2023–2024 sobre incidencias reales de GitHub, resolvían sin ayuda solo una fracción pequeña de ellas Jimenez et al., 2024. La brecha entre «escribe código plausible» y «resuelve problemas de producción de extremo a extremo» es el espacio donde su organización gana o pierde dinero.

El error estratégico más común del período 2025–2026 es optimizar la primera dimensión (contratar menos desarrolladores porque «la IA escribe el código») mientras se ignora la segunda (quién valida que el código es correcto, seguro y alineado con el negocio). Las

organizaciones que ganan en la era agéntica no son las que eliminan ingenieros: son las que **reentrenan a sus ingenieros para gobernar agentes**.

Tres preguntas para su próximo comité de dirección:

1. ¿Sabemos cuánto gastamos hoy, al mes, en tokens de modelos de lenguaje, y quién lo gasta? (Si la respuesta es no, el capítulo 1 (Vol. 4) es para usted.)
2. ¿Qué porcentaje del código generado por IA llega a producción sin retrabajo humano? (Si nadie lo mide, la cifra de productividad que le reportan es una ilusión.)
3. ¿Tenemos un plan de formación para convertir ingenieros senior en orquestadores de agentes, o asumimos que ocurrirá solo? (El capítulo 2 propone uno.)

Ejercicio resuelto 1.1: ubique cada sistema en el espectro de autonomía

Enunciado. Clasifique estos cuatro sistemas en el espectro de autonomía (niveles 0 a 4) y justifique: (a) un servicio que resume cada ticket entrante en una línea; (b) un asistente que, ante un bug reportado, busca en el código, reproduce el fallo y propone un parche; (c) un flujo que extrae datos de facturas, los valida contra reglas y los carga al ERP; (d) un sistema donde un coordinador reparte una migración entre agentes de análisis, conversión y prueba.

Solución comentada. (a) **Nivel 0**: una llamada por ticket, sin decisiones de flujo; cualquier nivel superior sería costo sin retorno. (b) **Nivel 3**: el modelo decide qué archivos leer, cuándo ejecutar tests y cuándo detenerse; exige presupuesto de iteraciones y herramientas de solo lectura más una de propuesta de parche, nunca de escritura directa. (c) **Nivel 1**: los pasos son fijos y conocidos; la tentación común es llamarlo «agente» en la presentación interna, pero es un *workflow*, y esa precisión importa: se presupuesta, se prueba y se gobierna distinto. (d) **Nivel 4**: descomposición dinámica multidominio; solo se justifica si (b) ya opera bien, porque hereda todos sus requisitos y añade coordinación. La lección transversal: el nivel se elige por la **estructura de la tarea**, no por la ambición del equipo.

Lo esencial del capítulo 1

- La secuencia 2017–2026 es reconstruible con literatura: Transformer → few-shot → código → instrucciones → herramientas → multi-agente (Vaswani et al., 2017; Brown et al., 2020; Chen et al., 2021; Ouyang et al., 2022; Yao, Zhao, et al., 2023).
- Un agente percibe, razona, actúa y aprende Wooldridge, 2009; un chatbot no actúa: la diferencia es la arquitectura, no el modelo.
- La agencia es reactiva (responde a tiempo a los cambios) y proactiva (toma la iniciativa hacia un objetivo); ReAct equilibra ambas intercalando observación y razonamiento (Wooldridge, 2009; Russell & Norvig, 2020).
- Workflows (el código decide) vs. agentes (el modelo decide): empiece por el patrón más simple que resuelva el problema Anthropic, 2024a.
- El nivel de autonomía correcto es el mínimo que resuelve la tarea; cada nivel añade varianza, costo y superficie de fallo.
- El costo se desplazó de escribir a validar: las organizaciones ganadoras reentrenan ingenieros para gobernar agentes.

Capítulo 2: El AI Engineer: el perfil más valioso de la próxima década

Un nuevo rol emerge

En 2023, Shawn Wang, conocido en la comunidad como «swyx», publicó un ensayo que circuló por miles de equipos de ingeniería: *The Rise of the AI Engineer* Wang, 2023. La tesis era simple y precisa: estaba emergiendo una nueva categoría de profesional técnico que no era un *Machine Learning Engineer* (quien entrena modelos) ni un *Data Scientist* (quien analiza datos), sino alguien que **construye aplicaciones sobre modelos ya entrenados**. La intuición de fondo era económica: entrenar un modelo de frontera cuesta cientos de millones de dólares y está al alcance de un puñado de laboratorios Bommasani et al., 2021; construir sobre esos modelos cuesta una clave de API y está al alcance de cualquier equipo. Donde hay una asimetría así, emerge una profesión.

Ese profesional (el **AI Engineer**) es el arquitecto de la era agéntica.

La matriz de diferenciación

Para entender qué hace único al AI Engineer, hay que distinguirlo con precisión de los roles adyacentes:

La distinción no es burocrática; es de **dirección del flujo de trabajo**. El ML Engineer trabaja *hacia* el modelo: datos, entrena-

Figura 2: Tabla 2.1: Diferenciación de roles en la era de la IA

miento, despliegue de pesos. El AI Engineer trabaja *desde* el modelo: prompts, contexto, herramientas, evaluación, producto. Ambos comparten frontera con el software engineering clásico, pero sus artefactos, sus métricas y sus modos de fallo son distintos.

► **PISTA TÉCNICA: El stack del AI Engineer**

El AI Engineer opera con un stack tecnológico específico que combina ingeniería de software clásica con primitivas propias de la era agéntica:

Figura 3: Diagrama 2.1: Stack tecnológico del AI Engineer

Las siete competencias núcleo del AI Engineer

1. Ingeniería de prompts como disciplina formal. No es «hablarle bien a la IA»: es el diseño sistemático de instrucciones que producen salidas predecibles y verificables. Sus técnicas tienen literatura revisada por pares: la cadena de pensamiento (*chain-of-thought*) mejora de forma medible el razonamiento en varios pasos Wei, Wang, et al., 2022; el aprendizaje con pocos ejemplos en contexto es una capacidad emergente documentada desde GPT-3 Brown et al., 2020; y los marcos como DSPy demuestran que los prompts pueden tratarse como programas optimizables, no como texto artesanal Khattab et al., 2024. El corolario profesional: un prompt de producción se versiona, se testea y se revisa como cualquier otro artefacto de código.

2. Arquitectura RAG (Retrieval-Augmented Generation). El patrón fundamental para conectar LLMs con conocimiento privado sin reentrenamiento, formalizado por Lewis et al. (2020): recuperar los fragmentos relevantes de una base de conocimiento e inyectarlos en el contexto antes de generar. El AI Engineer debe dominar estrategias de segmentación (*chunking*), representación vectorial (*embeddings*), indexación y reordenamiento (*re-ranking*); y, sobre todo, sus límites: la recuperación irrelevante no es neutra, contami-

na la generación. El RAG es además la principal defensa práctica contra la alucinación, el fenómeno (exhaustivamente catalogado por Ji et al., 2023) por el cual un modelo genera contenido fluido pero infundado.

3. Diseño de herramientas (*tool use*). Definir qué herramientas puede usar un agente, cómo se documentan y cómo se manejan los errores en la cadena de invocaciones. La regla de oro: **el nombre y la descripción de una herramienta son su prompt**. El modelo decide cuándo invocarla leyendo esa descripción, igual que un desarrollador decide qué función llamar leyendo su documentación (Schick et al., 2023; Anthropic, 2024).

4. Gestión de contexto y memoria. Las ventanas de contexto crecieron de 4.000 tokens a más de un millón en cuatro años, pero el espacio no equivale a atención: los modelos recuperan peor la información situada en medio de contextos largos, un fenómeno medido con precisión por Liu et al. (2024) y conocido como *lost in the middle*. El AI Engineer diseña sistemas de memoria que persisten, recuperan y comprimen la información relevante entre sesiones, en lugar de volcar todo el corpus en cada llamada, que es, además, la primera causa de Token-Burn (capítulo 1 (Vol. 4)).

5. Evaluación de sistemas LLM (*evals*). Construir arneses de evaluación para medir si un agente mejora o empeora con cada cambio. Es el equivalente funcional del TDD para sistemas no deterministas, y el capítulo 3 (Vol. 4) lo desarrolla en profundidad. La técnica central, usar un LLM como juez de las salidas de otro, alcanza acuerdos con evaluadores humanos superiores al 80 %, comparables al acuerdo entre humanos Zheng et al., 2023, siempre que se controlen sus sesgos conocidos.

6. Optimización de costos. Seleccionar el modelo correcto para cada tarea, usar caché de prompts, procesar en lote lo que no es urgente y diseñar pipelines que minimicen tokens sin sacrificar calidad. Es una competencia de ingeniería, no de compras: las decisiones que más mueven la factura (enrutamiento por complejidad, estructura del prefijo cacheable) viven en el código (capítulo 1 (Vol. 4)).

7. Orquestación multi-agente. El nivel más avanzado: diseñar sistemas donde múltiples agentes colaboran, se especializan y se go-

biernan mutuamente (capítulos 5 y 8). Requiere las seis competencias anteriores; un sistema multi-agente construido sin evals ni control de costos no es arquitectura, es un incendio distribuido.

```
# Ejemplo 2.1: La firma de una herramienta bien diseñada
# La descripción ES el prompt que el modelo usa para decidir cuándo
llamarla

tools = [
    {
        "name": "buscar_documentacion_tecnica",
        "description": """Busca en la base de conocimiento técnica de
        la empresa.
        USAR cuando: el usuario pregunta sobre arquitectura interna,
APIs propias,
        decisiones de diseño pasadas, o estándares del equipo.
        NO USAR para: código de librerías externas, conceptos
generales de programación.""",
        "input_schema": {
            "type": "object",
            "properties": {
                "consulta": {
                    "type": "string",
                    "description": "La pregunta técnica a buscar. Ser
                    específico."
                },
                "dominio": {
                    "type": "string",
                    "enum": ["arquitectura", "apis", "seguridad",
                    "base-de-datos", "frontend"],
                    "description": "Área técnica de la consulta para
                    filtrar resultados"
                }
            },
            "required": ["consulta"]
        }
    }
]
```

Obsérvese el patrón «USAR cuando / NO USAR para»: las descripciones prescriptivas, que dicen *cuándo* invocar la herramienta, no solo qué hace, mejoran de forma medible la tasa de invocación correcta en los modelos recientes, que tienden a ser conservadores al decidir si una herramienta es necesaria Anthropic, 2024.

■ **PISTA EJECUTIVA: El valor económico del AI Engineer**

El AI Engineer es, hacia 2026, uno de los roles técnicos con mayor escasez relativa en el mercado. Las razones estructurales:

- Los *bootcamps* producen desarrolladores junior a escala, pero no producen AI Engineers: el rol exige experiencia previa en sistemas de producción.
- Los ML Engineers tienen la mitad del stack (modelos) pero no la otra mitad (ingeniería de software de producto).
- Los desarrolladores senior tienen la mitad del stack (ingeniería) pero desconocen las primitivas LLM.

Esa escasez tiene un correlato salarial directo y verificable: las plataformas públicas de compensación técnica (por ejemplo, levels.fyi) sitúan de forma consistente los roles de ingeniería de IA en los percentiles superiores de la compensación del sector, por encima de los roles equivalentes de software generalista en los mismos niveles de seniority. No citaremos cifras absolutas (caducan más rápido que cualquier libro), pero la *prima relativa* es el dato estratégico, y se ha sostenido durante todo el período 2023–2026.

Para organizaciones en Latinoamérica y otras regiones con brecha salarial respecto a los polos tecnológicos, esa brecha crea una ventana competitiva: un equipo pequeño de AI Engineers bien formados en Lima, Buenos Aires o Medellín puede competir en *output* con equipos varias veces mayores que no han hecho la transición, a una fracción del costo. La ventana es temporal, como toda ventaja de arbitraje, y se cierra cuando la competencia completa su propia transición.

La pregunta estratégica para su organización es «¿estamos formando AI Engineers internamente o los estamos tercerizando?», no «¿compramos herramientas de IA?»

Demostrar, no acumular: el manifiesto práctico

Para trabajar con multiagentes no se necesita acumular cursos.

Se necesita demostrar que se sabe hacer cosas.

Construir arquitecturas.

Desplegar aplicaciones.

Trabajar con tecnologías de punta (Docker, Kubernetes, Terraform) aplicadas a un problema concreto el mismo día que se aprenden, no acumuladas como certificaciones en un currículum.

Y, como ingeniero: resolver problemas reales.

Esa es la prueba que un comité de contratación (o un cliente) puede verificar en una hora de conversación técnica; un certificado no la reemplaza. La sección anterior definió las siete competencias; esta define cómo un evaluador externo las reconoce en la práctica, y cómo un equipo que ya las tiene puede aprovechar la brecha temporal descrita antes de que se cierre.

El estándar de tres piezas. Un AI Engineer que pueda mostrar (1) un repositorio con un sistema multiagente propio, con arnés y evals (no un tutorial copiado), (2) esa misma arquitectura desplegada y accesible, no solo funcionando en su máquina, y (3) al menos un incidente real que depuró y documentó, ha demostrado más que la mayoría de los currículums con cinco certificaciones. Las tres piezas son verificables en la misma hora de entrevista; las certificaciones no lo son.

Discriminar entre tecnología de punta y ruido. No toda herramienta nueva merece el tiempo de aprenderla; la disciplina que distingue al AI Engineer del entusiasta es saber cuándo una tecnología resuelve un problema estructural y cuándo solo añade una capa de moda. Tres criterios prácticos: ¿la mantiene un actor con incentivo de largo plazo (un proveedor cloud, una fundación, no un solo mantenedor de fin de semana)? ¿Existe ya una alternativa establecida y la nueva opción resuelve algo que la anterior no resolvía? ¿Sobrevive la herramienta a un fallo de red o una caída de servicio sin dejar el sistema en un estado inconsistente? Aplicado a los servidores MCP del capítulo 9, esto separa lo que vale la pena adoptar hoy de lo que vale la pena solo vigilar:

Herramienta	Qué resuelve realmente	Por qué discriminarla como madura
MCP de GitHub	Revisión de PRs, triage de issues, disparo de CI/CD por lenguaje natural	El servidor MCP más desplegado del ecosistema; mantenido junto al proveedor de origen

Herramienta	Qué resuelve realmente	Por qué discriminarla como madura
MCP de Terraform (HashiCorp, oficial)	Consulta de módulos y providers del Registry; opcionalmente gestión de workspaces de HCP Terraform	Su configuración segura por defecto (solo lectura del Registry público, sin token) es una lección de diseño en sí misma
MCP de Kubernetes / Lens	Gestión de clústeres EKS/AKS, diagnóstico de pods, sin reconfigurar kubeconfig	Integración nativa con los dos proveedores cloud dominantes
Docker MCP Toolkit / Gateway	Empaqueta más de 300 servidores MCP verificados como contenedores, con aislamiento de credenciales y un único punto de enrutamiento hacia los clientes de IA	Es la implementación concreta, lista para usar, del patrón de *gateway multiproveedor* que el capítulo 3 describió en abstracto
Context7	Documentación y ejemplos de código actualizados en el momento de la consulta, no en el corte de entrenamiento del modelo	Reduce directamente el tipo de alucinación más costoso: la API que cambió después del corte de conocimiento
Playwright MCP	Automatización de navegador para pruebas end-to-end y scraping controlado	Estándar de facto para agentes que necesitan verificar una interfaz, no solo generar código para ella
Archify	Genera diagramas de arquitectura como *skill* para Claude Code, Cursor o Codex CLI, validando la representación (esquema, layout, rutas) antes de dibujar, con cada nodo enlazado a la línea de código que lo prueba	Ataca el problema real de los diagramas de arquitectura generados por IA: un componente que no existe en el repo simplemente no se dibuja

Herramienta	Qué resuelve realmente	Por qué discriminarla como madura
Napkin AI	Convierte texto o explicación en diagramas y esquemas editables para comunicar una idea, no para documentar una arquitectura real	Complementa a Archify en el extremo opuesto: cuando lo que se necesita es claridad conceptual para un lector no técnico, no trazabilidad al código

La especificación de MCP alcanzó versión estable en julio de 2026, con el transporte *Streamable HTTP* promovido a la misma categoría que *stdio* (lo que hace viables los servidores compartidos en equipo y remotos, no solo locales). Eso es una señal de madurez del protocolo, no de una herramienta en particular; la discriminación sigue siendo, servidor por servidor, la misma de la tabla anterior.

Figura 4: Diagrama 2.2: Ruta de transición

La ruta de conversión más eficiente que conocemos no pasa por un máster en *machine learning*. Pasa por un programa intensivo de ocho a doce semanas sobre las siete competencias anteriores, aplicado a un proyecto real de la propia organización, con presupuesto de tokens propio y métricas de éxito definidas el primer día.

Escenario ilustrativo: conversión acelerada. Considérese un equipo de cuatro desarrolladores senior con experiencia sólida en Python, sin formación previa en LLMs. (Lo que sigue es un escenario compuesto, construido a partir de patrones recurrentes en programas de adopción reales; las cifras son ilustrativas del orden de magnitud, no datos auditados.) Tras un programa de ocho semanas en las competencias núcleo, el equipo construye en otras seis semanas un sistema de revisión asistida de contratos que antes consumía decenas de horas semanales de un equipo legal. El sistema procesa lotes de contratos con revisión humana final, a un costo por documento de centavos frente a las decenas de dólares del

proceso puramente manual. El retorno del programa de formación se recupera en el primer trimestre de operación.

Dos advertencias convierten ese escenario en ingeniería en lugar de marketing. Primera: la «precisión del 94 %» que suele citarse en casos así no significa nada sin definir el conjunto de evaluación, el costo de cada tipo de error y quién revisa el 6 % restante: las preguntas del capítulo 3 (Vol. 4). Segunda: el sistema descrito es un *workflow* con puntos de revisión humana, no un agente autónomo; esa elección de nivel de autonomía (capítulo 1) es la razón de que funcione.

Ejercicio resuelto 2.1: diseñe el programa de conversión de ocho semanas

Enunciado. Diseñe el programa de conversión para tres desarrolladores senior de su equipo, usando las siete competencias como temario. Defina: secuencia, proyecto vehículo y criterio de éxito medible por semana.

Solución comentada. La secuencia sigue las dependencias entre competencias, no su orden de aparición: semanas 1–2, **prompts como artefactos** y **diseño de herramientas** (las bases de todo); semanas 3–4, **RAG** y **gestión de contexto** (el conocimiento propio); semanas 5–6, **evals** (antes de optimizar nada, medir); semanas 7–8, **costos** y una **orquestación mínima** (pipeline de dos roles, no más). El proyecto vehículo debe cumplir tres condiciones: datos propios, fallo barato y un usuario interno real con nombre, el asistente de onboarding del capítulo 5 (Vol. 4) las cumple casi siempre. El criterio de éxito semanal es un entregable verificable, no asistencia: la semana 5, por ejemplo, termina con una suite de 20 evals corriendo en CI, o no termina. Los dos errores que arruinan estos programas: empezar por la orquestación multi-agente (la competencia 7 sin las otras seis es la receta del incendio distribuido) y formar sin presupuesto de tokens propio: quien no administra su gasto no aprende la competencia 6, la aprende su jefe a fin de mes.

Lo esencial del capítulo 2

- El AI Engineer construye *desde* el modelo (prompts, contexto, herramientas, evals, producto); el ML Engineer construye *hacia* el modelo Wang, 2023.
- Siete competencias núcleo, con literatura: prompts (Wei, Wang, et al., 2022; Khattab et al., 2024), RAG Lewis et al., 2020, herramientas Schick et al., 2023, contexto Liu et al., 2024, evals Zheng et al., 2023, costos y orquestación.
- La descripción de una herramienta es su prompt; el patrón «USAR cuando / NO USAR para» mejora la invocación correcta.
- La prima salarial del rol es el dato estratégico; las cifras absolutas caducan.
- La conversión interna (8–12 semanas, proyecto real, presupuesto propio, métricas desde el día uno) supera al máster genérico.
- Se demuestra la competencia con tres piezas verificables en una hora (repositorio propio, despliegue accesible, un incidente real documentado), no con certificados. Discriminar herramientas de punta (MCP incluido) exige mantenedor con incentivo de largo plazo, un problema real no cubierto antes, y tolerancia a fallo sin dejar el sistema inconsistente.

Preguntas para discusión

1. ¿Cuántas de las siete competencias núcleo puede cubrir hoy su equipo sin contratar? ¿Cuál es la más urgente de desarrollar?
2. Si su organización tuviera que elegir entre contratar un AI Engineer senior externo o formar a dos ingenieros internos, ¿qué criterio usaría para decidir?
3. ¿Qué proyecto interno de bajo riesgo y alta visibilidad serviría como vehículo de formación?

Capítulo 3: Platform Engineering en la era agéntica

La plataforma como producto interno

La *ingeniería de plataforma* (Platform Engineering) es la disciplina que construye los caminos pavimentados por donde transitan los equipos de desarrollo de una organización. No es infraestructura: es la abstracción que convierte operaciones complejas (desplegar una aplicación, conectar una base de datos, configurar observabilidad) en acciones de un clic, o de una instrucción en lenguaje natural.

El marco de referencia es *Team Topologies* Skelton & Pais, 2019: la estructura de los equipos debe diseñarse para minimizar la carga cognitiva de los equipos alineados al flujo de valor (*stream-aligned teams*). El equipo de plataforma existe para absorber esa carga cognitiva, y su producto, la plataforma interna, se gestiona como cualquier producto: con usuarios, hoja de ruta y métricas de adopción.

En la era agéntica, el equipo de plataforma adquiere una responsabilidad adicional: **exponer las capacidades de IA como primitivas de primera clase de la plataforma interna**. Si cada equipo negocia su propia clave de API, elige su propio modelo, implementa su propio control de costos y resuelve su propia seguridad, la organización tiene tantos puntos de fallo (técnicos, económicos y regulatorios) como equipos.

► **PISTA TÉCNICA: Arquitectura de una AI Platform**

Justificación de diseño: diagrama de secuencia para modelado de interacción

El AI Gateway es ante todo un flujo: una solicitud entra, pasa por capas (validación, enrutamiento, ejecución, registro), y sale con un costo. Lo que importa es el *orden temporal* de esos pasos: cuánto cuesta cada uno, cuánto tiempo toma, dónde se revisa antes de proseguir.

Un diagrama de componentes mostraría dónde vive el gateway en la arquitectura; un diagrama de clases mostraría sus partes internas. Aquí necesitamos el *flujo en el tiempo*, la secuencia de decisiones y gastos que definen si la solicitud prospera o se rechaza.

En este libro reaparece el diagrama de secuencia en el **Capítulo 7 (Vol. 4)** (el flujo CDC → Agente → Consumidores), donde nuevamente el tiempo y el desacoplamiento temporal son el mensaje central.

Figura 5: Diagrama 3.1: Internal AI Developer Platform (AI-IDP), flujo de una solicitud a través del Gateway

El AI Gateway: el componente más crítico

El AI Gateway es el punto de entrada único a todos los modelos de IA de la organización. Sus responsabilidades mínimas: enrutamiento por complejidad de tarea, contabilidad de costos por equipo, presupuestos con cortes automáticos, registro de auditoría y lógica de reserva (*fallback*) entre proveedores.

```
# Ejemplo 3.1: AI Gateway mínimo viable
# Centraliza routing, cost tracking y presupuestos por equipo.
#
# NOTA SOBRE PRECIOS: los valores corresponden a la documentación
pública
# de Anthropic al cierre de edición (2026). Los precios y los
identificadores
# de modelo cambian: consúltelos en https://platform.claude.com/docs
antes
# de usar este código en producción. El PATRÓN es lo estable.
```

```
import time
from dataclasses import dataclass, field
from typing import Optional

import anthropic

@dataclass
class ModelConfig:
    model_id: str
    cost_per_million_input: float   # USD por millón de tokens de
entrada
    cost_per_million_output: float  # USD por millón de tokens de
salida
    max_context_tokens: int
    latency_tier: str               # "fast" | "balanced" | "quality"

MODEL_REGISTRY = {
    "quality": ModelConfig(
        model_id="claude-opus-4-8",
        cost_per_million_input=5.00,
        cost_per_million_output=25.00,
        max_context_tokens=1_000_000,
        latency_tier="quality",
    ),
    "balanced": ModelConfig(
        model_id="claude-sonnet-4-6",
        cost_per_million_input=3.00,
        cost_per_million_output=15.00,
        max_context_tokens=1_000_000,
        latency_tier="balanced",
    ),
    "fast": ModelConfig(
        model_id="claude-haiku-4-5",
        cost_per_million_input=1.00,
        cost_per_million_output=5.00,
        max_context_tokens=200_000,
        latency_tier="fast",
    ),
}

class BudgetExceededError(Exception):
    pass

class AIGateway:
    """
```

```
    Punto único de acceso a modelos de IA en la organización.
    Responsable de: routing, cost tracking, presupuesto, auditoría.
    """

    def __init__(self, team_id: str, budget_usd_monthly: float):
        self.client = anthropic.Anthropic()  # usa ANTHROPIC_API_KEY
        del entorno
        self.team_id = team_id
        self.budget_usd_monthly = budget_usd_monthly
        self.cost_this_month = 0.0
        self.request_log: list[dict] = []

    def _select_model(self, task_complexity: str,
                      force_model: Optional[str] = None) -> ModelConfig:
        """Selección automática de modelo según complejidad
        declarada."""
        if force_model:
            return MODEL_REGISTRY[force_model]
        # Routing por complejidad: tareas simples no deben pagar
        precio premium
        routing = {
            "simple": "fast",       # clasificación, extracción,
            resumen corto
            "moderate": "balanced", # generación de código, análisis,
            Q&A
            "complex": "quality",   # arquitectura, razonamiento
            multi-paso
        }
       return MODEL_REGISTRY[routing.get(task_complexity, "balanced")]

    def _calculate_cost(self, model: ModelConfig,
                        input_tokens: int, output_tokens: int) -> float:
        return (
            (input_tokens / 1_000_000) * model.cost_per_million_input
            + (output_tokens / 1_000_000) *
            model.cost_per_million_output
        )

    def complete(
        self,
        messages: list,
        system: str = "",
        task_complexity: str = "moderate",
        force_model: Optional[str] = None,
        max_tokens: int = 2048,
    ) -> dict:
```

```
        model_config = self._select_model(task_complexity, force_model)

        # Verificar presupuesto ANTES de ejecutar
        if self.cost_this_month >= self.budget_usd_monthly * 0.95:
            raise BudgetExceededError(
                f"Equipo {self.team_id}: 95% del presupuesto mensual
                consumido. "
                f"Costo actual: ${self.cost_this_month:.2f} / "
                f"${self.budget_usd_monthly:.2f}"
            )

        start_time = time.time()
        response = self.client.messages.create(
            model=model_config.model_id,
            max_tokens=max_tokens,
            system=system,
            messages=messages,
        )
        latency_ms = (time.time() - start_time) * 1000

        cost = self._calculate_cost(
            model_config,
            response.usage.input_tokens,
            response.usage.output_tokens,
        )
        self.cost_this_month += cost

        # Audit log: cada request queda registrado para análisis
        posterior
        self.request_log.append({
            "team": self.team_id,
            "model": model_config.model_id,
            "complexity": task_complexity,
            "input_tokens": response.usage.input_tokens,
            "output_tokens": response.usage.output_tokens,
            "cost_usd": cost,
            "latency_ms": round(latency_ms),
            "timestamp": time.time(),
        })

        text = next(b.text for b in response.content if b.type ==
"text")
        return {
            "content": text,
            "model_used": model_config.model_id,
            "cost_usd": cost,
```

```
            "tokens": {
                "input": response.usage.input_tokens,
                "output": response.usage.output_tokens,
            },
        }


# Uso típico en un equipo:
# gateway = AIGateway(team_id="backend-platform",
budget_usd_monthly=500.0)
# result = gateway.complete(
#     system="Eres un experto en revisión de código.",
#     messages=[{"role": "user", "content": f"Revisa este
código:\n\n{code}"}],
#     task_complexity="moderate",  # el Gateway selecciona el modelo
balanceado
# )
# print(f"Costo de esta operación: ${result['cost_usd']:.4f}")
```

Gateway multiproveedor

Un gateway de producción rara vez habla con un solo proveedor. La buena noticia es que las tres APIs dominantes comparten la misma forma conceptual (mensajes con roles, herramientas con esquemas JSON, conteo de tokens en la respuesta), de modo que la capa de abstracción es delgada:

```
# Ejemplo 3.2: Adaptadores mínimos para Claude, OpenAI y Gemini.
# Cada proveedor tiene su SDK oficial; el contrato interno es común.
# Verifique modelos y precios vigentes en la documentación de cada
proveedor.

class ClaudeAdapter:
    def __init__(self):
        import anthropic
        self.client = anthropic.Anthropic()

    def complete(self, system: str, user: str, model: str, max_tokens:
    int) -> str:
        resp = self.client.messages.create(
            model=model, max_tokens=max_tokens,
            system=system, messages=[{"role": "user", "content":
user}],
        )
        return next(b.text for b in resp.content if b.type == "text")
```

```python
class OpenAIAdapter:
    def __init__(self):
        from openai import OpenAI
        self.client = OpenAI()  # usa OPENAI_API_KEY del entorno

    def complete(self, system: str, user: str, model: str, max_tokens:
    int) -> str:
        resp = self.client.chat.completions.create(
            model=model, max_completion_tokens=max_tokens,
            messages=[{"role": "system", "content": system},
                      {"role": "user", "content": user}],
        )
        return resp.choices[0].message.content

class GeminiAdapter:
    def __init__(self):
        from google import genai
        self.client = genai.Client()  # usa GEMINI_API_KEY del entorno

    def complete(self, system: str, user: str, model: str, max_tokens:
    int) -> str:
        resp = self.client.models.generate_content(
            model=model, contents=user,
            config={"system_instruction": system,
                    "max_output_tokens": max_tokens},
        )
        return resp.text
```

La decisión de cuándo enrutar a cada proveedor es de negocio, no de fe: se toma con evaluaciones propias (capítulo 3 (Vol. 4)) sobre las tareas propias, y se revisa trimestralmente, porque la frontera de calidad-precio entre proveedores se mueve varias veces al año.

■ **PISTA EJECUTIVA: El ROI del Platform Engineering agéntico**

Un equipo de plataforma de cuatro a seis ingenieros que construye una AI-IDP bien diseñada puede multiplicar la productividad de cincuenta a cien ingenieros de producto. El modelo económico es el mismo que el de la infraestructura cloud: en lugar de que cada equipo configure sus propios servidores, la plataforma abstrae esa complejidad. La diferencia en la era agéntica es que el «servidor» es el acceso a la inteligencia, y la «configuración» incluye selección de modelos, gestión de costos, evaluación de calidad y gobernanza de

datos.

Hay además un argumento de riesgo que suele pesar más que el de productividad: sin gateway central no hay inventario de uso, y sin inventario de uso no hay forma de responder a un auditor, interno o regulatorio, qué datos salieron hacia qué proveedor. El marco de gestión de riesgos de IA del NIST (2023) y el Reglamento Europeo de IA Unión Europea, 2024 convergen en la misma exigencia práctica: trazabilidad. La plataforma es el lugar natural donde esa trazabilidad se implementa una vez y se hereda gratis.

Métricas clave para medir el éxito de una AI Platform:

Métrica	Línea base (sin plataforma)	Objetivo (6 meses post-plataforma)
Tiempo de integración de IA en una funcionalidad nueva	2–3 semanas	2–3 días
Costo por request a LLM	Sin visibilidad	Visible y $<$ \$0,01 en tareas simples
% de requests enrutados al modelo de costo correcto	0 % (manual)	$>$ 80 % (automático)
Incidentes por alucinaciones en producción	Sin métricas	Medidos, con presupuesto de error explícito
Cobertura de evaluación automática de agentes	0 %	$>$ 70 % de los casos núcleo

Ejercicio resuelto 3.1: añada el corte por proyecto al gateway

Enunciado. El `AIGateway` del ejemplo 3.1 presupuesta por equipo. Finanzas pide presupuesto **por proyecto** dentro de cada equipo, con una política distinta: al agotarse, no fallar, degradar al modelo económico y avisar. Diseñe el cambio mínimo.

Solución comentada. Tres cambios, ninguno estructural. Primero, `complete()` recibe `project_id` y el registro de presupuestos pa-

sa de un escalar a un diccionario `{project_id: Budget}`; el log de auditoría ya registra todo lo necesario para reconstruir el gasto por proyecto retroactivamente, dividendo directo de haberlo registrado desde el día uno. Segundo, la política de agotamiento se vuelve explícita y enumerada (`HARD_STOP`, la actual, o `DEGRADE`, la nueva), porque una política implícita es un incidente esperando ocurrir. Tercero, en `DEGRADE`, `_select_model` fuerza el tier `fast` y el resultado lleva `"degraded": True`: el consumidor **debe saber** que recibió calidad reducida. Degradar en silencio es mentir con infraestructura. El matiz que separa al senior: la degradación también se registra como métrica; si un proyecto pasa el 30 % del mes degradado, el problema no es el presupuesto, es la estimación (capítulo 2 (Vol. 4)) o las cinco causas de Token-Burn (capítulo 1 (Vol. 4)).

Lo esencial del capítulo 3

- La plataforma interna es un producto Skelton & Pais, 2019; en la era agéntica expone la IA como primitiva de primera clase.
- El AI Gateway es el componente crítico: enrutamiento por complejidad, costo por equipo, presupuestos con corte, auditoría, fallback.
- Sin gateway no hay inventario; sin inventario no hay respuesta para el auditor (NIST, 2023; Unión Europea, 2024).
- Las tres APIs dominantes comparten forma conceptual: la capa multiproveedor es delgada y el enrutamiento se decide con evals propios, trimestralmente.
- Un equipo de plataforma de 4–6 multiplica a 50–100 ingenieros de producto: la economía del cloud, aplicada a la inteligencia.

Preguntas para discusión

1. ¿Cuántos equipos de su organización usan hoy modelos de IA con claves de API propias, fuera de cualquier inventario central?
2. Si mañana un proveedor duplicara sus precios o degradara un modelo, ¿cuánto tardaría su organización en redirigir el tráfico a otro?

3. ¿Quién es, hoy, el «dueño de producto» de la plataforma interna de IA? Si la respuesta es «nadie», ¿qué incidente tendrá que ocurrir para que exista?

Capítulo 4: Fundamentos técnicos del agente moderno

> **Antes de empezar, este es el capítulo más denso del volumen.** Reúne el modelo cognitivo del agente, los patrones de razonamiento (ReAct y variantes), el manejo de contexto, el diseño del arnés de producción y la traducción de Clean Code y SOLID al código sintético. Véase la Guía de lectura en las páginas preliminares.

De la cadena de texto al sistema cognitivo

El salto conceptual más importante para un ingeniero que viene del mundo tradicional es entender que un agente moderno no es un «chatbot mejorado». Es un **sistema cognitivo con estructura interna**: percepción, estado, razonamiento, acción y memoria, organizados alrededor de un modelo de lenguaje que hace las veces de motor de inferencia.

La arquitectura de referencia sigue siendo la de la literatura clásica de agentes Russell & Norvig, 2020, y su traducción a la era LLM tiene cinco componentes:

Figura 6: Diagrama 4.1: Arquitectura interna de un agente moderno

1. **El modelo** (motor de razonamiento): un LLM accesible por API, intercambiable y versionado.

2. **El contexto** (percepción + memoria de trabajo): instrucciones de sistema, historial, resultados de herramientas, conocimiento recuperado.
3. **Las herramientas** (efectores): funciones tipadas que el modelo puede invocar.
4. **La memoria** (persistencia): almacenamiento externo (archivos, vectores, bases de datos) que sobrevive a la sesión.
5. **El arnés** (*harness*, el cuerpo): el código que ejecuta el bucle, aplica permisos, registra auditoría e impone presupuestos. El arnés es la parte que el proveedor no le da y que define la seguridad real del sistema.

El patrón ReAct: la base de todo agente

El patrón **ReAct** (*Reasoning* + *Acting*) es el algoritmo fundamental que subyace a la mayoría de los agentes modernos Yao, Zhao, et al., 2023. Su contribución, validada experimentalmente sobre tareas de razonamiento y de decisión, fue demostrar que **intercalar** trazas de razonamiento con acciones sobre un entorno supera tanto al razonamiento puro (que alucina sin contacto con la realidad) como a la acción pura (que actúa sin plan):

```
Ciclo ReAct:

1. THOUGHT:  "Necesito saber qué archivos Python existen en el
proyecto"
2. ACTION:   glob(pattern="**/*.py")
3. OBSERVE:  ["src/main.py", "src/agents/orchestrator.py",
"tests/test_agent.py"]
4. THOUGHT:  "Tengo 3 archivos. El más relevante es orchestrator.py"
5. ACTION:   read_file("src/agents/orchestrator.py")
6. OBSERVE:  [contenido del archivo]
7. THOUGHT:  "Veo que usa el patrón X. La respuesta al usuario es..."
8. ANSWER:   [respuesta final al usuario]
```

Cada ciclo consume tokens; cada ciclo tiene un costo. Entender ReAct es entender por qué ocurre el Token-Burn, y cómo controlarlo: el número de ciclos por tarea es la variable económica fundamental de un agente, y se gobierna con criterios de parada, presupuestos y diseño de herramientas que minimicen iteraciones fallidas.

```python
# Ejemplo 4.1: Bucle agéntico mínimo y completo con la API de Claude.
# Este es el esqueleto de TODO agente: un bucle while que alterna
# inferencia y ejecución de herramientas hasta que el modelo decide
parar.

import json
import anthropic

client = anthropic.Anthropic()

TOOLS = [{
    "name": "get_weather",
    "description": "Obtiene el clima actual de una ciudad. USAR cuando
    el "
                   "usuario pregunte por clima o temperatura actual.",
    "input_schema": {
        "type": "object",
        "properties": {"city": {"type": "string", "description":
        "Ciudad"}},
        "required": ["city"],
    },
}]

def execute_tool(name: str, tool_input: dict) -> str:
    if name == "get_weather":
        # En producción: llamada real a un servicio meteorológico
        return f"22°C y despejado en {tool_input['city']}"
    return f"Herramienta desconocida: {name}"

def run_agent(user_input: str, max_iterations: int = 10) -> str:
    """Bucle ReAct con criterio de parada explícito (presupuesto de
    ciclos)."""
    messages = [{"role": "user", "content": user_input}]

    for _ in range(max_iterations):          # SIEMPRE acotar el bucle
        response = client.messages.create(
            model="claude-opus-4-8",
            max_tokens=4096,
            tools=TOOLS,
            messages=messages,
        )

        if response.stop_reason != "tool_use":
            return next(b.text for b in response.content if b.type ==
            "text")
```

```
        # El modelo pidió herramientas: ejecutarlas y devolver
        resultados
        messages.append({"role": "assistant", "content":
response.content})
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                result = execute_tool(block.name, block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result,
                })
        messages.append({"role": "user", "content": tool_results})

    return "Presupuesto de iteraciones agotado sin respuesta final."
```

Tres detalles de este código son lecciones de producción. Primero, el bucle está **acotado** (`max_iterations`): un agente sin criterio de parada es un incidente de facturación esperando a ocurrir. Segundo, el contenido íntegro de la respuesta del asistente, incluidos los bloques de invocación, se reenvía en el historial: el protocolo de la API lo exige y omitirlo es el error de principiante más común. Tercero, cada resultado de herramienta lleva el identificador de la invocación que lo originó: la trazabilidad empieza en el protocolo.

Más allá de ReAct: la familia de patrones de razonamiento

ReAct no está solo. La investigación 2022–2024 produjo una familia de patrones que el AI Engineer debe conocer porque cada uno compra calidad con tokens de forma distinta:

- **Chain-of-Thought (CoT).** Inducir al modelo a razonar paso a paso antes de responder mejora sustancialmente el desempeño en problemas de varios pasos Wei, Wang, et al., 2022. Es el patrón más barato: un solo pase de inferencia, más tokens de salida. Los modelos recientes lo incorporan de forma nativa como «razonamiento extendido» o «pensamiento adaptativo», donde el propio modelo decide cuánto pensar.

- **Tree of Thoughts (ToT).** Generalizar el CoT a un árbol: explorar varias líneas de razonamiento, evaluarlas y retroceder Yao, Yu, et al., 2023. Multiplica el costo; se reserva para problemas de búsqueda y planificación donde el primer camino rara vez es el bueno.
- **Self-Refine.** El modelo critica su propia salida y la mejora en iteraciones sucesivas, sin entrenamiento adicional, con mejoras medias del orden del 20 % en preferencia humana sobre tareas diversas Madaan et al., 2023.
- **Reflexion.** Tras un intento fallido verificable (un test que no pasa, una respuesta incorrecta), el agente genera una reflexión verbal sobre el fallo, la guarda en memoria episódica y reintenta con esa lección en el contexto Shinn et al., 2023. Es el fundamento del *self-reflection loop* del capítulo 8.

La regla de selección es económica: **CoT por defecto; Self-Refine cuando hay criterio de calidad automático; Reflexion cuando hay señal de éxito verificable; ToT solo cuando el espacio de soluciones lo exige**. Cada peldaño multiplica el costo por tarea; el capítulo 1 (Vol. 4) da las herramientas para decidir con números.

La ventana de contexto no es memoria

Un error conceptual caro: tratar la ventana de contexto como si fuera memoria fiable. La evidencia dice otra cosa: el rendimiento de los modelos al usar información de contextos largos degrada significativamente cuando la información relevante está en el medio del contexto, con una curva en U: se recuerda mejor lo del principio y lo del final Liu et al., 2024. Las consecuencias de diseño:

1. **Posición importa.** Las instrucciones críticas van al principio (sistema) o al final (último mensaje), nunca enterradas en medio de un volcado documental.
2. **Menos es más.** Inyectar el fragmento relevante (RAG bien afinado) supera a inyectar el documento completo, en calidad además de en costo.
3. **La memoria de largo plazo se externaliza.** Arquitecturas como MemGPT formalizan la analogía con los sistemas operativos:

una jerarquía de memoria donde el contexto es la RAM, escasa y cara, y el almacenamiento externo es el disco, con el propio modelo gestionando qué se carga y qué se archiva Packer et al., 2024.

Razonamiento extendido: la nueva frontera del agente deliberativo

El advenimiento de los modelos con capacidad de razonamiento extendido (categoría formalizada entre 2024 y 2025) ha reconfigurado sustancialmente los fundamentos arquitectónicos presentados en las secciones precedentes. La evidencia empírica acumulada en benchmarks de complejidad elevada indica que la asignación dinámica de cómputo en tiempo de inferencia (*test-time compute scaling*) constituye una dimensión de optimización cualitativamente distinta del escalado de parámetros: en determinadas clases de problemas, incrementar los recursos computacionales durante la inferencia resulta más eficiente que aumentar el tamaño del modelo Snell et al., 2024.

De esta observación se derivan dos consecuencias arquitectónicas de primer orden. En primer lugar, la distinción entre razonamiento rápido y razonamiento deliberativo (conceptualización originalmente avanzada por Kahneman (2011) en el contexto de la cognición humana) adquiere traducción operativa directa en los modelos contemporáneos: sistemas como o1 de OpenAI (2024) y el mecanismo de *extended thinking* de la familia Claude Anthropic, 2024c generan una cadena de pensamiento interna que puede abarcar miles de tokens de reflexión antes de emitir la respuesta visible al usuario. En segundo lugar, este modelo introduce un nuevo parámetro de optimización (el presupuesto de razonamiento (*thinking budget*)) que el AI Engineer debe administrar con la misma precisión con que gestiona el presupuesto de iteraciones ReAct.

La implicación práctica central puede enunciarse con precisión: **no toda tarea justifica razonamiento extendido**. El costo computacional puede superar en un orden de magnitud al de una llamada convencional; activarlo en una clasificación de correos sería equivalente a emplear cálculo diferencial para ordenar cinco elementos. La

heurística de selección reproduce la lógica ya establecida para los patrones de razonamiento:

Situación	Razonamiento estándar (CoT)	Razonamiento extendido
Clasificación, extracción estructurada	✓ Suficiente	✗ Sobredimensionado
Generación de código de complejidad moderada	✓ Suficiente	✗ Salvo requisitos críticos
Razonamiento matemático multi-paso	✓ Con limitaciones	✓ Cuando los errores son costosos
Planificación arquitectónica de alto impacto	~ Límite del método	✓ Justificado
Diagnóstico de incidentes de producción complejos	~	✓ El razonamiento profundo preserva el SLA

```
# Ejemplo 4.2: Razonamiento extendido con presupuesto explícito.
# El thinking_budget es el equivalente de max_iterations para el
razonamiento interno.
# Reservar exclusivamente para tareas donde el costo del error supera
el del razonamiento.
#
# NOTA: requiere modelos con soporte de razonamiento extendido.
# Consulte la documentación de Anthropic para disponibilidad
actualizada.

import anthropic

client = anthropic.Anthropic()

def analizar_arquitectura_compleja(
    descripcion: str,
    thinking_budget: int = 10_000,
) -> dict:
    """
    Análisis arquitectónico con razonamiento extendido.
    Costo potencial: 10–20× el de una llamada convencional al mismo
modelo.
    """
    respuesta = client.messages.create(
        model="claude-opus-4-8",
```

```
        max_tokens=16_000,
        thinking={"type": "enabled", "budget_tokens": thinking_budget},
        messages=[{
            "role": "user",
            "content": (
                "Analiza esta arquitectura y determina: "
               "(1) los tres riesgos de mayor probabilidad × impacto, "
                "(2) la decisión de diseño más urgente de revisar, "
                "(3) el camino de migración de menor riesgo si se
                detecta un problema crítico.\n\n"
                f"Arquitectura: {descripcion}"
            ),
        }],
    )

    tokens_razonamiento, texto = 0, ""
    for bloque in respuesta.content:
        if bloque.type == "thinking":
            tokens_razonamiento = len(bloque.thinking.split())
        elif bloque.type == "text":
            texto = bloque.text

    return {
        "analisis": texto,
        "tokens_razonamiento_estimados": tokens_razonamiento,
        "costo_estimado_usd": (
            (respuesta.usage.input_tokens / 1_000_000) * 5.00
            + (respuesta.usage.output_tokens / 1_000_000) * 25.00
        ),
    }
```

En el plano de la evidencia empírica, los benchmarks AIME 2024 (competición de matemáticas de nivel olímpico) documentaron que modelos convencionales resolvían menos del 15 % de los problemas, mientras que los dotados de razonamiento extendido alcanzaron tasas superiores al 70 % OpenAI, 2024. En el dominio del software, SWE-bench Verified (versión curada del benchmark original Jimenez et al., 2024) evidenció que agentes con razonamiento extendido superaron en un 40 % relativo a los basados en cadena de pensamiento convencional en corrección de defectos reales. Estos resultados establecen la **clase de problemas** donde el razonamiento extendido genera retorno económico positivo; no toda tarea los encarna.

Resiliencia en agentes de producción: el arnés como sistema de control

Los modos de fallo de un agente en producción difieren cualitativamente de los de un sistema determinista. A diferencia de los servicios web convencionales (cuyo estado de error es discreto (el endpoint responde o no responde)) un agente puede fallar de forma *gradual y silenciosa*: producir salidas que aparentan ser válidas pero omiten el paso crítico, o correctas el 95 % del tiempo y erróneas en los casos límite que ninguna prueba manual alcanza. Tres patrones de resiliencia son aplicables de forma inmediata:

Disyuntor agéntico (*Agentic Circuit Breaker*). Derivado del patrón documentado por Fowler (2002), adapta su lógica a los agentes: monitoriza la tasa de ciclos fallidos en una ventana temporal; si supera el umbral, abre el circuito y enruta hacia un flujo de degradación controlada en lugar de permitir que el agente continúe consumiendo tokens en estado de fallo encubierto.

Reintentos con retroceso exponencial y *jitter*. El bucle ReAct ya implementa reintentos implícitos; la diferencia con un reintento de producción reside en la gestión del estado entre intentos: el agente persiste su razonamiento parcial antes de reintentar para no repetir el trabajo completado. El patrón Reflexion Shinn et al., 2023 formaliza esta gestión del estado inter-ciclo.

Presupuesto de confianza por herramienta. Cada herramienta mantiene un historial de tasa de éxito en la sesión; si falla más de N veces consecutivas, el arnés la deshabilita y emite alerta. El registro de auditoría (que el arnés ya mantiene por otras razones) provee toda la información necesaria sin instrumentación adicional.

```
# Ejemplo 4.3: Disyuntor agéntico minimal.
# El presupuesto de confianza por herramienta sigue el mismo patrón de
datos.

from collections import deque
from dataclasses import dataclass, field

@dataclass
class DisyuntorAgéntico:
    """
    Abre el circuito cuando la tasa de ciclos fallidos supera el umbral
```

```
    en la ventana temporal configurada. Sin estado global: una
instancia
    por sesión de agente.
    """
    tamaño_ventana: int = 10
    umbral_fallo: float = 0.5
    _resultados: deque = field(
        default_factory=lambda: deque(maxlen=10))
    _abierto: bool = False

    def registrar(self, exito: bool):
        self._resultados.append(exito)
        if len(self._resultados) == self.tamaño_ventana:
            tasa = 1 - sum(self._resultados) / len(self._resultados)
            self._abierto = tasa >= self.umbral_fallo

    @property
    def esta_abierto(self) -> bool:
        return self._abierto

    def flujo_degradado(self, peticion: str) -> str:
        """Llamar cuando el circuito está abierto: nunca continuar en
        silencio."""
        return (
            f"Agente en modo degradado. Petición encolada para revisión
            "
            f"humana: {peticion[:100]}…"
        )
```

■ **PISTA EJECUTIVA: Las cuatro dimensiones del SLA agéntico**

Un SLA de disponibilidad (expresado como porcentaje de *uptime*) resulta necesario pero insuficiente para los sistemas agénticos: un agente puede estar disponible (responder en menos de 500 ms) y producir código incorrecto el 20 % de las veces. El SLA completo comprende al menos cuatro dimensiones que deben definirse antes de cualquier adopción:

1. **Disponibilidad técnica**: porcentaje de solicitudes sin error técnico. Métrica convencional; se instrumenta igual que cualquier servicio HTTP.
2. **Tasa de aceptación a la primera**: porcentaje de salidas aprobadas por el revisor humano sin modificación. Esta es la métrica

de calidad; sin ella, las cifras de productividad reportadas son ilusiones (Peng et al., 2023; Ziegler et al., 2024).

3. **Costo por tarea completada**: no por invocación, sino por tarea de negocio cerrada; se relaciona directamente con el CIU (Costo de Iteración por Unidad) del capítulo 1 (Vol. 4).
4. **Latencia del ciclo completo**: desde el inicio hasta la salida verificada, incluyendo todos los ciclos ReAct y la ejecución de herramientas. Esta métrica difiere cualitativamente de la latencia de API y es la que el usuario de negocio experimenta.

Un comité ejecutivo que adopta agentes sin definir previamente estas cuatro dimensiones está adoptando tecnología sin criterio de éxito. Lo que no se define, no se mide; lo que no se mide, no mejora.

Ejercicio resuelto 4.1: diseñe el arnés de producción

Enunciado. Un agente de revisión de contratos lleva tres semanas en producción con un promedio de 2,3 ciclos por contrato. El equipo detecta que ciertos contratos producen 15+ ciclos antes de agotar el presupuesto sin respuesta útil. Diseñe las tres modificaciones al arnés que diagnosticarían y resolverían el problema.

Solución comentada. Las tres modificaciones son diagnóstico, protección y mejora (en ese orden, porque optimizar sin comprender es adivinar):

Diagnóstico: añadir telemetría por tipo de contrato (longitud, idioma, tipo de cláusula) al registro de auditoría. El patrón de 15+ ciclos casi siempre está correlacionado con una categoría específica: contratos en formato no estándar, idiomas mixtos o tablas que el extractor procesa mal. Sin el dato, la búsqueda es aleatoria.

Protección: implementar el disyuntor con umbral basado en percentil, no en máximo. Si el P95 de ciclos es 4, un presupuesto de 10 es razonable; un presupuesto de 15 es un incidente esperando facturarse. El disyuntor debe escalar, no abortar en silencio: el contrato que excede el presupuesto va a la cola de revisión humana con su traza completa.

Mejora: identificada la categoría problemática, la solución raramente es «más ciclos»; es una herramienta especializada (preprocesador de tablas, normalizador de formato) que reduce los ciclos a 2–3. La lección central: los problemas de rendimiento de los agentes casi siempre son problemas de diseño de herramientas, no de presupuesto de tokens.

El Código Limpio Sintético: los principios que perduran

Las tecnologías de esta industria caducan; los principios, no. *Clean Code* enseñó a una generación que el código se escribe para personas y que la legibilidad es una decisión económica, no estética Martin, 2009; *Refactoring* enseñó que mejorar el diseño sin cambiar el comportamiento es una disciplina con catálogo propio Fowler, 2018. Ninguno de los dos libros menciona modelos de lenguaje y, sin embargo, ambos describen con precisión cómo se construyen buenos sistemas agénticos, porque los artefactos centrales de la era agéntica (prompts, herramientas, skills, orquestadores) son **texto que será leído por un intérprete falible**. Eso es lo que siempre fue el código.

Proponemos el **Código Limpio Sintético**: la traducción de los principios clásicos a los artefactos agénticos. Lo presentamos aquí, en el capítulo de fundamentos, porque el resto de la Parte II lo aplicará pieza por pieza:

Principio clásico	En código (Martin, 2009; Fowler, 2018)	En sistemas agénticos
Nombres significativos	Una variable se llama por lo que contiene	El nombre de una herramienta o skill ES su prompt: `buscar_documentacion_tecnica` se invoca bien; `helper2` no se invoca nunca o se invoca mal

Principio clásico	En código (Martin, 2009; Fowler, 2018)	En sistemas agénticos
Responsabilidad única (SRP)	Una función hace una cosa	Una herramienta ejecuta una acción; un agente ejerce un rol; un skill encapsula un procedimiento. El agente «que hace de todo» es la god class de esta década
Comentarios honestos	El comentario explica el porqué, no el qué	La descripción de la herramienta dice **cuándo usarla y cuándo no**, no repite lo obvio del esquema
Errores explícitos	Excepciones tipadas, nunca códigos mágicos	Resultados de herramienta con `is_error` y mensajes accionables: «Ciudad no encontrada; usa el formato Ciudad, País» le permite al modelo corregirse; «Error 500» lo deja adivinando
DRY	Una sola fuente de verdad	Prompts maestros y plantillas versionadas (capítulo 5); precios y modelos en un único módulo (apéndices A y B)
Fronteras (boundaries)	Interfaces que aíslan lo externo	Bounded contexts (capítulo 6) y separación instrucciones/datos (capítulo 4 (Vol. 4))
Tests primero	El test define el comportamiento	Los evals definen la calidad antes de optimizarla (capítulo 3 (Vol. 4))

Principio clásico	En código (Martin, 2009; Fowler, 2018)	En sistemas agénticos
La regla del Boy Scout	Deja el código más limpio de lo que lo encontraste	Deja el prompt más limpio de lo que lo encontraste: cada incidente de producción termina en una mejora versionada del prompt, el skill o el contexto

Dos consecuencias de este cuadro merecen subrayarse. La primera: **el refactoring de prompts existe** y se practica con la misma disciplina que el de código: pequeños pasos, comportamiento verificado por la suite de evals antes y después Fowler, 2018. La segunda: la revisión de prompts en *pull request* deja de ser opcional; un prompt de producción que cambia sin revisión es un *commit* directo a la rama principal.

Este es el hilo que cose la Parte II: el capítulo 5 aplica estos principios a los roles y a los **prompts maestros**; el 6, a las fronteras de dominio; el 7, a los skills y sus **plantillas**; el 8, a los orquestadores; el 9, a las herramientas MCP. Si el lector retoma el libro meses después en cualquiera de esos capítulos, esta tabla es el mapa para reubicarse.

SOLID como interrogatorio: las preguntas que refactorizan

La tabla anterior traduce los principios; falta la herramienta que los pone en movimiento. Un principio enunciado es una afirmación que el lector aprueba con la cabeza y olvida a la semana; un principio convertido en **pregunta** es un procedimiento que se ejecuta. La diferencia es la misma que entre «el código debe tener responsabilidad única» y «¿qué segunda razón de cambio esconde esta clase?». La primera adorna una diapositiva; la segunda encuentra el defecto.

El autor sintetizó los cinco principios SOLID Martin, 2017 como un cuestionario de refactorización (material publicado primero ante la comunidad; véase «Obra pública del autor»), y que aquí adquiere

una segunda vida con un giro propio de la era agéntica: **estas mismas preguntas son el prompt de un agente revisor**. Lo que durante años fue una lista mental del ingeniero experto se vuelve el cuerpo de un skill de revisión de diseño (capítulo 7), ejecutable sobre cada *pull request*.



Figura 7: Diagrama 4.2: SOLID como cuestionario de refactorización, las preguntas que convierten cada principio en un procedimiento ejecutable (figura publicada por el autor).

El interrogatorio, principio por principio:

- **Responsabilidad única (SRP).** ¿Cómo puedo dividir esta clase o función en varias con una única responsabilidad? ¿Qué responsabilidades adicionales tiene esta clase que podrían extraerse? En el dominio agéntico la pregunta se traslada sin cambios: ¿qué hace este agente además de su rol declarado? El revisor que también clasifica y el clasificador que también redacta son la *god class* del prompt.
- **Abierto/cerrado (OCP).** ¿Cómo modificar esta clase para que esté abierta a la extensión pero cerrada a la modificación? ¿Qué patrón de diseño facilita las extensiones futuras? Para un sistema de agentes: ¿puedo añadir un rol nuevo sin reescribir el orquestador? Si la respuesta exige tocar el bucle de control, el orquestador acopla lo que debería delegar (capítulo 8).
- **Sustitución de Liskov (LSP).** ¿Esta subclase puede sustituir a su superclase sin alterar el comportamiento? ¿Las subclases cumplen el contrato de la base? Su análogo agéntico es el contrato de herramienta: ¿un adaptador de proveedor distinto (capítulo 3) respeta el mismo esquema de entrada y salida, o introduce sorpresas que el modelo no puede prever?
- **Segregación de interfaces (ISP).** ¿Esta interfaz tiene demasiados métodos? ¿Cómo dividirla en interfaces más específicas? ¿Qué métodos sobran para ciertos clientes? Para el agente: una herramienta con quince parámetros opcionales es una interfaz gorda: el modelo la usa peor que tres herramientas precisas.

- **Inversión de dependencias (DIP).** ¿Cómo invertir las dependencias para depender de abstracciones y no de implementaciones concretas? Es, literalmente, la tesis del capítulo 6 (Vol. 4): «el modelo es un detalle». El dominio depende de un *puerto* de inferencia, no de un SDK.

Cerrando la rueda, las **preguntas de mejora general** que el cuestionario añade a los cinco principios: ¿qué patrones de diseño elevarían la calidad? ¿Cómo mejorar la legibilidad sin tocar la funcionalidad? ¿Qué partes son más difíciles de probar y cómo abordarlas? Esta última es la bisagra con el capítulo 3 (Vol. 4): *lo difícil de probar suele ser lo mal diseñado*, y en sistemas agénticos lo que no se puede evaluar no se puede gobernar.

El valor de formular SOLID como interrogatorio es doble. Para el humano, convierte un principio abstracto en una acción concreta durante la revisión. Para el agente, **las preguntas son directamente el prompt**: un skill `revision-de-diseno` que recorre la rueda sobre el diff produce hallazgos accionables en lugar de la felicitación genérica («el código se ve bien») que es el modo de fallo por defecto de un revisor automático sin criterio. La disciplina del refactoring de Fowler (2018), pasos pequeños y comportamiento verificado por la suite de evals antes y después, gobierna la aplicación: el agente *propone* la refactorización; el humano la aprueba; los evals certifican que el comportamiento no cambió.

Ejercicio resuelto 4.2: dos herramientas, un presupuesto y un error bien diseñado

Enunciado. Extienda el bucle agéntico mínimo (ejemplo 4.1) con: (a) una segunda herramienta `convertir_temperatura`, (b) manejo de errores accionables con `is_error`, y (c) verifique que el presupuesto de iteraciones se respeta.

Solución comentada.

```
# Solución 4.1 — las líneas nuevas están señaladas.

TOOLS = [
    {
```

```
        "name": "get_weather",
        "description": "Obtiene el clima actual de una ciudad. USAR
        cuando el "
                       "usuario pregunte por clima o temperatura
                       actual.",
        "input_schema": {
            "type": "object",
            "properties": {"city": {"type": "string", "description":
            "Ciudad"}},
            "required": ["city"],
        },
    },
    {   # (a) Segunda herramienta: nombre = verbo + objeto, sin
ambigüedad
        "name": "convertir_temperatura",
        "description": "Convierte una temperatura entre °C y °F. USAR
        cuando "
                       "el usuario pida la conversión explícitamente o
                       pida "
                       "el clima en una unidad distinta a la
                       disponible.",
        "input_schema": {
            "type": "object",
            "properties": {
                "valor": {"type": "number"},
                "a_unidad": {"type": "string", "enum": ["C", "F"]},
            },
            "required": ["valor", "a_unidad"],
        },
    },
]

def execute_tool(name: str, tool_input: dict) -> tuple[str, bool]:
    """Devuelve (contenido, is_error). El error es un mensaje
    ACCIONABLE."""
    if name == "get_weather":
        city = tool_input["city"].strip()
        if not city:
            # (b) Error que enseña al modelo cómo corregirse:
            return ("Ciudad vacía. Reintenta indicando la ciudad, "
                    "por ejemplo: 'Lima'.", True)
        return (f"22°C y despejado en {city}", False)
    if name == "convertir_temperatura":
        v, u = tool_input["valor"], tool_input["a_unidad"]
        return (f"{v * 9 / 5 + 32:.1f}°F" if u == "F"
                else f"{(v - 32) * 5 / 9:.1f}°C", False)
```

```
    return (f"Herramienta desconocida: {name}.", True)
```

En el bucle, el resultado se reenvía con la marca de error: `{"type": "tool_result", "tool_use_id": block.id, "content": contenido, "is_error": es_error}`. Para (c), la verificación es un test determinista (nivel 1 de la pirámide del capítulo 3 (Vol. 4)): con una herramienta trucada que siempre devuelva error, el bucle debe terminar exactamente en `max_iterations` llamadas y devolver el mensaje de presupuesto agotado.

Por qué importa. Los tres cambios son el Código Limpio Sintético en miniatura: nombres que se autoinvocan, errores que enseñan y presupuestos que se prueban. El error accionable es el detalle que más mejora la tasa de recuperación de un agente en producción, y el que casi nadie escribe.

Lo esencial del capítulo 4

- Un agente moderno = modelo + contexto + herramientas + memoria + **arnés**; el arnés es suyo y define la seguridad real.
- ReAct intercala razonamiento y acción Yao, Zhao, et al., 2023; cada ciclo cuesta; el bucle SIEMPRE se acota.
- CoT por defecto; Self-Refine con criterio automático; Reflexion con señal verificable; ToT solo si el espacio de búsqueda lo exige.
- La ventana de contexto no es memoria: posición importa, menos es más, la memoria larga se externaliza (Liu et al., 2024; Packer et al., 2024).
- Los principios del Código Limpio Sintético (nombres, SRP, errores accionables, DRY, fronteras, tests, Boy Scout) gobiernan el resto de la Parte II.

Capítulo 5: La Empresa Sintética

Aplicando la especialización al código

Frederick Winslow Taylor revolucionó la manufactura a principios del siglo XX con un principio devastadoramente simple: la especialización sistemática del trabajo aumenta la productividad más que cualquier otro factor. Un trabajador que hace una sola tarea con excelencia supera en rendimiento a diez que hacen diez tareas con mediocridad.

La misma lógica aplica a los sistemas de agentes, y aquí la intuición taylorista tiene respaldo experimental moderno. Un agente de propósito general, que planifica, escribe, revisa y despliega simultáneamente, comete más errores, consume más tokens y produce salidas de menor calidad que un sistema donde cada agente tiene un rol definido y restringido. Los marcos que organizaron agentes como una empresa de software lo demostraron de forma independiente: ChatDev estructura el desarrollo como una cadena de fases (diseño, codificación, prueba) atendidas por agentes especializados que dialogan entre sí Qian et al., 2024; MetaGPT codifica procedimientos operativos estándar (los SOP de una consultora) en el flujo de comunicación entre agentes con roles de gerente de producto, arquitecto e ingeniero, y con ello reduce drásticamente las alucinaciones en cascada Hong et al., 2024; y AutoGen generalizó el patrón de conversación entre agentes como primitiva de programación Wu et al., 2024.

Este es el principio de la **Empresa Sintética**: la organización de agentes en roles especializados, con interfaces de comunicación ex-

plícitas entre ellos, refleja la estructura de una empresa de software bien organizada.

La **Ley de Conway** da el marco formal: «cualquier organización que diseñe un sistema producirá un diseño cuya estructura es una copia de la estructura de comunicación de la organización» Conway, 1968. Aplicada a agentes, la lectura es literal: si sus agentes no tienen protocolos de comunicación claros (quién habla con quién, con qué formato, con qué autoridad), el sistema resultante será tan caótico como una empresa sin organigrama. La novedad histórica es que, por primera vez, el arquitecto diseña *las dos cosas a la vez*: la organización y el sistema son el mismo artefacto.

> **Justificación de diseño: diagrama de clases para modelado estructural**
>
> La Empresa Sintética es ante todo una *estructura*: qué roles existen, cuál es la jerarquía, qué contratos cada rol cumple. Un diagrama de clases (con sus herencias y relaciones) revela exactamente eso: quién reporta a quién, quién es especialista de qué, dónde están los límites de responsabilidad.
>
> Un diagrama de secuencia mostraría cómo interactúan en tiempo de ejecución; un componente mostraría dónde viven en el servidor. Aquí necesitamos ver *la estructura estática del organigrama*, la topología que gobierna todo lo demás.
>
> El diagrama de clases reaparece en este libro en el **Capítulo 6** (DDD y agregados), en el **Capítulo 6 (Vol. 4)** (Clean Architecture y puertos), donde nuevamente usamos clases para mostrar relaciones estructurales que no cambian en el tiempo.

Figura 8: Diagrama 5.1: La Empresa Sintética: estructura organizacional de agentes (diagrama de clases)

► **PISTA TÉCNICA: Implementando la Empresa Sintética**

```
# Ejemplo 5.1: Orquestador básico de la Empresa Sintética.
# Roles especializados con modelos de costo distinto por rol:
# el planner usa el modelo más capaz; los analistas, modelos más
económicos.

import anthropic

client = anthropic.Anthropic()

AGENT_ROLES = {
    "planner": {
       "model": "claude-opus-4-8",      # razonamiento de arquitectura
        "system": """Eres el CTO de un equipo de ingeniería.
        Recibes un objetivo de negocio y lo descompones en tareas
técnicas concretas.
        Output: JSON con lista de tareas, cada una con: descripción,
agente responsable,
        criterios de aceptación, y estimado de complejidad
(low/medium/high).""",
    },
    "coder": {
        "model": "claude-sonnet-4-6",     # generación de código,
        balanceado
        "system": """Eres un senior engineer. Implementas código
        limpio, testeable y
        siguiendo los principios SOLID. Solo produces código, no
explicaciones largas.
        Si necesitas aclaración sobre la tarea, pregunta en una sola
línea antes de codificar.""",
    },
    "reviewer": {
        "model": "claude-sonnet-4-6",
        "system": """Eres un tech lead experto en code review. Evalúas
        código en 4 dimensiones:
        1. Correctitud (¿hace lo que debe hacer?), 2. Seguridad (OWASP
Top 10),
        3. Mantenibilidad (SOLID, DDD), 4. Performance (complejidad
algorítmica).
        Reporta TODOS los hallazgos, incluidos los de severidad baja,
con tu nivel
        de confianza en cada uno: un filtro posterior decidirá qué se
corrige.
        Output: JSON con score 0-10 por dimensión y lista de cambios
requeridos.""",
    },
    "security": {
```

```
        "model": "claude-haiku-4-5",      # análisis de patrones,
        económico
        "system": """Eres un experto en seguridad de aplicaciones.
        Analiza código buscando
        las vulnerabilidades más comunes: SQL injection, XSS, SSRF,
deserialización insegura,
        credenciales en el código, path traversal. Output: JSON con
vulnerabilidades,
        severidad (critical/high/medium/low), y remediación
sugerida.""",
    },
}

def run_agent(role: str, task: str, context: str = "") -> dict:
    """Ejecuta un agente especializado con su configuración
    predefinida."""
    config = AGENT_ROLES[role]
    content = f"{context}\n\nTarea: {task}" if context else task

    response = client.messages.create(
        model=config["model"],
        max_tokens=4096,
        system=config["system"],
        messages=[{"role": "user", "content": content}],
    )
    text = next(b.text for b in response.content if b.type == "text")
    return {
        "role": role,
        "output": text,
        "tokens_used": response.usage.input_tokens +
        response.usage.output_tokens,
        "model": config["model"],
    }

def synthetic_company_pipeline(objetivo: str) -> dict:
    """
    Pipeline de la Empresa Sintética:
    Planner -> Coder -> (Reviewer + Security) -> métricas.
    """
    total_tokens = 0

    plan = run_agent("planner", objetivo)
    total_tokens += plan["tokens_used"]

    code = run_agent("coder", plan["output"],
                     context=f"Plan técnico:\n{plan['output']}")
```

```
    total_tokens += code["tokens_used"]

    # Review y Security son independientes: en producción, ejecútelos
    en
    # paralelo (asyncio / ThreadPoolExecutor). Aquí secuencial por
    claridad.
    review = run_agent("reviewer", code["output"], context="Código a
revisar:")
    total_tokens += review["tokens_used"]

    security_audit = run_agent("security", code["output"],
                               context="Código a auditar:")
    total_tokens += security_audit["tokens_used"]

    return {
        "plan": plan["output"],
        "codigo": code["output"],
        "review": review["output"],
        "security": security_audit["output"],
        "metricas": {
            "total_tokens": total_tokens,
            # El costo real se calcula por modelo y por dirección
            # (input/output); véase el AIGateway del capítulo 3.
        },
    }
```

Obsérvese la decisión económica embebida en la tabla de roles: el planificador usa el modelo más capaz porque sus errores se propagan a todo el pipeline; el auditor de seguridad usa el más económico porque su tarea es de reconocimiento de patrones sobre un texto acotado. Asignar modelo por rol, y no «el mejor modelo para todo», es la primera optimización de costos de cualquier sistema multi-agente, y por sí sola suele reducir la factura a la mitad.

La instrucción del revisor: una lección de evaluación

El prompt del rol `reviewer` incluye una línea que merece explicación: «Reporta TODOS los hallazgos [...] un filtro posterior decidirá». Los modelos recientes siguen las instrucciones de severidad con tanta literalidad que un prompt del tipo «reporta solo problemas graves» deprime la cobertura: el modelo encuentra el defecto y decide no reportarlo por no parecerle suficientemente grave. La práctica robusta

es separar **detección** (exhaustiva, con confianza declarada) de **filtrado** (un paso posterior, humano o automático). Es el mismo principio de los sistemas de detección clásicos: nunca mezclar el sensor con el umbral.

El prompt maestro: DRY aplicado a las instrucciones

Si observa los cuatro prompts de sistema del ejemplo 5.1, verá repetición estructural: todos definen un rol, un alcance, unas reglas y un formato de salida. Esa estructura repetida es una *plantilla* pidiendo a gritos ser extraída, el principio DRY del Código Limpio Sintético (capítulo 4). Llamamos **prompt maestro** a esa plantilla versionada de la que derivan todos los prompts de rol de la organización:

```
# Ejemplo 5.2: Prompt maestro — una sola estructura, muchos roles.
# La plantilla se versiona en el repositorio (prompts/master_v3.md) y
# cada rol es un conjunto de parámetros, no un texto artesanal.

MASTER_PROMPT = """Eres {rol}, parte de un equipo de agentes
especializados.

ALCANCE — tu única responsabilidad:
{alcance}

REGLAS — en orden de prioridad; ante conflicto, gana la primera:
{reglas}

FUERA DE ALCANCE — si la tarea cae aquí, decláralo y detente:
{fuera_de_alcance}

FORMATO DE SALIDA — exactamente esto, sin texto adicional:
{formato_salida}
"""

ROL_REVIEWER = {
    "rol": "un tech lead experto en code review",
    "alcance": "evaluar código en correctitud, seguridad (OWASP Top
    10), "
               "mantenibilidad (SOLID, DDD) y performance",
    "reglas": "1. Reporta TODOS los hallazgos con confianza
    declarada.\n"
```

```
            "2. Cita la línea exacta de cada hallazgo.\n"
            "3. No propongas refactorings fuera del diff revisado.",
    "fuera_de_alcance": "decidir si el PR se aprueba (lo decide un
    humano); "
                        "modificar el código",
    "formato_salida": 'JSON: {"hallazgos": [{"linea": int,
    "dimension": str, '
                      '"severidad": str, "confianza": float, "detalle":
                       str}]}',
}

def system_prompt(rol_config: dict) -> str:
    return MASTER_PROMPT.format(**rol_config)
```

La anatomía (rol, alcance, reglas priorizadas, fuera de alcance, formato) no es arbitraria: cada bloque neutraliza un modo de fallo conocido. El **alcance** combate la deriva de responsabilidad (la god class agéntica); las **reglas priorizadas** resuelven los conflictos de instrucciones que los modelos, ante ambigüedad, resuelven de forma impredecible; el **fuera de alcance** explícito es la diferencia entre un agente que escala lo que no le corresponde y uno que improvisa; y el **formato de salida** convierte la integración con el siguiente eslabón en un contrato verificable (capítulo 3 (Vol. 4)). Los beneficios operativos son los de siempre con DRY: una mejora en la plantilla se propaga a todos los roles, un eval de regresión cubre la estructura común, y el diff de un *pull request* sobre `prompts/` muestra exactamente qué cambió en el comportamiento del sistema.

> **Nota editorial: alcance del caso de estudio.** El caso que sigue es un escenario compuesto, construido para ilustrar el proceso de refactorización rol por rol, no una auditoría de una empresa real ni un estudio longitudinal. La sección "Estado del Arte" de este volumen documenta que no existe todavía perspectiva longitudinal publicada sobre sistemas agénticos en producción a varios años.

Caso de estudio: refactorizar un agente monolítico a Empresa Sintética

(*Caso compuesto ilustrativo: condensa un patrón de refactorización recurrente; las cifras indican orden de magnitud.*)

Situación inicial. Un equipo construyó «Dev-Bot»: un único agente con un prompt de sistema de 4.000 palabras que planifica, codifica, revisa su propio código y escribe la documentación. En la demo funcionaba; tres meses después, el equipo observa: tasa de código aceptado a la primera en torno al 40 %, costo por funcionalidad errático (el CIU del capítulo 1 (Vol. 4) oscila 10×), y cada ajuste del prompt para arreglar un comportamiento rompe otro: el síntoma inequívoco de una god class Martin, 2009, solo que escrita en prosa.

El refactoring, en cuatro pasos disciplinados (comportamiento verificado con evals antes y después de cada paso, como manda Fowler, 2018):

1. **Caracterización.** Antes de tocar nada, el equipo construye una suite de 40 evals con casos reales de los últimos tres meses (capítulo 3 (Vol. 4)). Es el equivalente de los *characterization tests*: sin ellos, «refactorizar» un prompt es adivinar.
2. **Extracción del primer rol.** El revisor se separa a un agente propio con el prompt maestro anterior. Hallazgo típico: el monolito «se aprobaba a sí mismo», el sesgo de autocomplacencia que el patrón evaluador-generador del capítulo 8 existe para romper. La tasa de defectos detectados sube de inmediato.
3. **Extracción del planificador y contratos JSON.** El plan deja de ser un párrafo libre y pasa a ser una lista de tareas con criterios de aceptación (formato de salida del prompt maestro). El codificador recibe tareas acotadas; el contexto por llamada baja, y con él, el costo y el efecto *lost in the middle* Liu et al., 2024.
4. **Asignación de modelo por rol.** Planificador en el modelo premium; codificador en el balanceado; barrido de seguridad en el económico (ejemplo 5.1). El CIU se estabiliza y cae en torno al 40 % respecto al monolito.

Resultado. Mismo proveedor, mismos modelos, ninguna técnica

exótica: solo SRP, DRY, contratos y tests aplicados a prosa. La lección del caso es la tesis del capítulo: la calidad de un sistema multi-agente es, ante todo, una propiedad de su *diseño organizacional*, no del modelo de turno.

Ejercicio resuelto 5.1: diseñe el pipeline de triaje de tickets

Enunciado. Una empresa recibe 500 tickets de soporte diarios. Diseñe una Empresa Sintética mínima que clasifique cada ticket (facturación/técnico/ventas), extraiga los datos estructurados y redacte un borrador de respuesta que un humano aprueba. Asigne modelo por rol y justifique.

Solución comentada. Tres roles bastan; la tentación de añadir un cuarto «coordinador» viola el árbol de decisión del capítulo 8 (los pasos son conocidos: es un *pipeline*, no necesita orquestador razonante):

Rol	Tarea	Modelo	Justificación
Clasificador	Categoría + urgencia + idioma	Económico (`claude-haiku-4-5`)	Clasificación cerrada de texto corto: la gama económica rinde igual aquí; a 500/día, la diferencia de precio financia el resto del pipeline
Extractor	JSON con cliente, producto, problema	Económico, con salida estructurada	Extracción contra esquema: verificable en el nivel 1 de la pirámide de testing

Rol	Tarea	Modelo	Justificación
Redactor	Borrador de respuesta con tono de marca	Balanceado (`claude-sonnet-4-6`)	Único rol donde la calidad de prosa importa; recibe SOLO el JSON del extractor, no el ticket crudo: menos tokens y menos inyección indirecta (capítulo 4 (Vol. 4))

Los detalles que separan la solución profesional de la ingenua: (1) el redactor consume el JSON del extractor, contrato explícito, contexto mínimo; (2) el humano aprueba el borrador, el sistema «falla barato» (capítulo 5 (Vol. 4)); (3) los tres prompts derivan del prompt maestro; y (4) el lote nocturno de tickets no urgentes va por la API batch a mitad de precio (capítulo 1 (Vol. 4)). Costo estimado del pipeline completo: calcúlelo con el apéndice B: es un ejercicio dentro del ejercicio, y la respuesta sorprende por lo baja.

Lo esencial del capítulo 5

- Especialización vence a generalismo, con evidencia: ChatDev, MetaGPT y AutoGen (Qian et al., 2024; Hong et al., 2024; Wu et al., 2024).
- La Ley de Conway (1968) ahora se diseña: organización y sistema son el mismo artefacto.
- Modelo por rol = primera optimización de costos; suele recortar la factura a la mitad.
- Detección separada de filtrado en los revisores: el sensor nunca decide el umbral.
- El prompt maestro (rol, alcance, reglas priorizadas, fuera de alcance, formato) es DRY aplicado a instrucciones: una plantilla versionada, muchos roles parametrizados.

■ **PISTA EJECUTIVA**

La Empresa Sintética no elimina la organización humana: la *refleja y la amplifica*. Cada equipo humano pasa de ejecutar tareas a gobernar una célula de agentes que las ejecuta. Las preguntas de diseño organizacional que esto abre (¿quién es responsable cuando el agente falla?, ¿cómo se mide a un ingeniero que orquesta?) se desarrollan en los capítulos 19 y 20. El dato para la decisión de hoy: el patrón de roles especializados no es una preferencia estética; es el patrón con mejor evidencia publicada de reducción de errores en cascada Hong et al., 2024 y mejora de factualidad Du et al., 2023, y su costo adicional es cuantificable y gobernable.

Capítulo 6: DDD como sistema inmunológico

Por qué la IA sin dominio genera entropía

Un modelo de lenguaje, por naturaleza estadística, tiende a producir la solución más *promedio* para un problema dado: la continuación más probable según su corpus de entrenamiento. En el desarrollo de software, la solución «más promedio» rara vez refleja las reglas de negocio específicas de su dominio. Y cuando el modelo no dispone de la información necesaria, no se detiene: genera contenido plausible y sin fundamento, el fenómeno de alucinación, sistematizado en la literatura como uno de los retos centrales de la generación con LLMs Ji et al., 2023. La crítica académica temprana lo formuló sin anestesia: un modelo de lenguaje es, en ausencia de anclaje, un repetidor estocástico de formas lingüísticas sin compromiso con la verdad Bender et al., 2021. Se puede discutir el alcance de esa crítica; no se puede ignorar su consecuencia de ingeniería: **el anclaje (grounding) no es opcional en producción**.

El **Domain-Driven Design** (Evans, 2003) es la metodología que establece un *lenguaje ubicuo* (un vocabulario compartido entre el negocio y el código) y unas fronteras explícitas (los *bounded contexts*) dentro de las cuales ese vocabulario tiene un significado preciso. Escrito veinte años antes de los LLMs, el DDD resulta ser exactamente el artefacto que un agente necesita: una ontología del negocio, legible por máquina, que separa lo válido de lo inválido.

Cuando un agente conoce el contexto acotado del dominio, puede distinguir entre:

- una *transferencia bancaria* (que implica reglas de validación de saldo, verificación KYC, límites regulatorios), y
- un simple movimiento de números en una base de datos.

Sin ese contexto, el agente produce lo segundo: rápido, barato y potencialmente ilegal. Por eso llamamos al DDD el **sistema inmunológico** de la Empresa Sintética: no acelera al agente; le impide generar tejido dañino a velocidad industrial.

El diagrama de clases: estructura de un agregado en DDD

Justificación de diseño: diagrama de clases para modelado estructural

En DDD, lo que interesa es la *estructura estática del dominio*: qué entidades existen, cuáles son raíces de agregados, qué invariantes gobiernan sus relaciones. Un diagrama de clases revela exactamente eso: jerarquía, responsabilidades, límites de consistencia.

Un diagrama de estados mostraría *cuándo* cambia un agregado; un componente mostraría *dónde* vive en el sistema. Aquí necesitamos ver *qué* es un agregado y *quién* es su raíz, el punto único de entrada para todos los cambios del dominio.

En este libro reaparece el diagrama de clases en el **Capítulo 5** (roles de agentes), en el **Capítulo 6 (Vol. 4)** (Clean Architecture y puertos), donde nuevamente usamos clases para mostrar estructuras que no cambian en el tiempo, solo en la composición.

Figura 9: Diagrama 6.1: Estructura de un Agregado en DDD: Entidad Raíz, sub-entidades y eventos de dominio

► PISTA TÉCNICA: Alimentando el bounded context al agente

```
# Ejemplo 6.1: Inyectando el contexto de dominio como "sistema
inmunológico".
# El domain context actúa como guardarraíl semántico: invariantes,
lenguaje
# ubicuo y anti-patrones, en texto plano versionado junto al código.

DOMAIN_CONTEXT_FINTECH = """
BOUNDED CONTEXT: Transferencias Bancarias — Reglas de Negocio
OBLIGATORIAS

INVARIANTES (nunca violar):
1. Una transferencia solo puede crearse si saldo_disponible >= monto +
comisión
2. El monto máximo por transferencia es PEN 50,000 (verificar con
ConfigService)
3. Transferencias > PEN 10,000 requieren verificación KYC del receptor
4. El estado de una transferencia es inmutable una vez en estado
PROCESANDO

LENGUAJE UBICUO:
- "Cuenta" = entidad con IBAN, saldo_disponible, saldo_en_transito,
estado
- "Transferencia" = agregado raíz con id_unico, origen, destino,
monto, comision, estado
- "Comisión" = calculada por CommissionService según tipo de cuenta y
monto
- "Disponible" = saldo total - saldo_en_transito (NO usar saldo total
directamente)

CONTEXTOS EXTERNOS (no implementar aquí):
- Autenticación -> AuthContext
- Notificaciones -> NotificationContext
- Reportes regulatorios -> ComplianceContext

ANTI-PATTERNS PROHIBIDOS:
- No acceder a la base de datos directamente (usar repositorios)
- No exponer entidades de dominio en la API (usar DTOs)
- No manejar notificaciones dentro de este contexto
"""

import anthropic

client = anthropic.Anthropic()

def code_with_domain_guard(tarea_tecnica: str) -> str:
```

```
    """Genera código con el bounded context inyectado como
    guardarraíl."""
    response = client.messages.create(
        model="claude-opus-4-8",
        max_tokens=4096,
        system=f"""Eres un senior engineer especializado en DDD y
arquitectura hexagonal.

CONTEXTO DE DOMINIO OBLIGATORIO — lee y respeta estas reglas antes de
escribir código:
{DOMAIN_CONTEXT_FINTECH}

INSTRUCCIÓN: Genera código que respete ESTRICTAMENTE el bounded
context definido.
Si la tarea viola algún invariante, DETENTE y explica qué invariante
se viola y por qué.""",
        messages=[{"role": "user", "content": tarea_tecnica}],
    )
    return next(b.text for b in response.content if b.type == "text")
```

Tres prácticas convierten este patrón en infraestructura:

1. **El contexto de dominio vive en el repositorio**, junto al código que gobierna (`domain/CONTEXT.md`), y se revisa en los mismos *pull requests*. Un contexto desactualizado es peor que ninguno: da confianza falsa.
2. **El contexto es un prefijo estable y cacheable.** Las APIs modernas permiten cachear el prefijo del prompt; un contexto de dominio extenso que se reutiliza en cada llamada cuesta una fracción del precio normal a partir de la segunda invocación (véase el capítulo 1 (Vol. 4)). La arquitectura correcta (contexto estable primero, tarea variable al final) es a la vez la más barata y la más fiable, una coincidencia afortunada que conviene explotar.
3. **Cada invariante tiene un test.** El contexto le dice al agente qué no debe violar; el test verifica que no lo violó. El capítulo 3 (Vol. 4) cierra ese bucle.

Del documento al guardarraíl ejecutable

El paso de madurez siguiente es derivar verificadores automáticos del propio contexto: si el invariante 1 dice que toda transferencia exi-

ge saldo suficiente, un verificador estático puede buscar las rutas de código que crean transferencias y comprobar que todas pasan por la validación. El agente revisor del capítulo 5 (`reviewer`) recibe el contexto de dominio en su prompt y el verificador corre en el CI: dos capas inmunológicas, una probabilística y una determinista. La defensa en profundidad no es desconfianza en el modelo; es ingeniería.

■ **PISTA EJECUTIVA**

La pregunta ejecutiva sobre el DDD agéntico es de inversión: documentar los bounded contexts de un sistema legado cuesta semanas de trabajo de las personas más ocupadas de la organización. La respuesta corta: ese documento es ahora un *activo de producción*, no documentación. Se inyecta en cada llamada de cada agente que toca el dominio, multiplica la tasa de código válido a la primera y reduce el retrabajo que hoy consume a esas mismas personas. Las organizaciones que ya practicaban DDD parten con ventaja estructural en la era agéntica; las que no, tienen en la adopción de agentes el incentivo económico que faltaba para pagar esa deuda de modelado.

Ejercicio resuelto 6.1: escriba el bounded context de un inventario

Enunciado. Redacte el contexto de dominio (formato del ejemplo 6.1) para el inventario de un e-commerce con esta regla de negocio: «no se puede vender lo que no está físicamente disponible, y lo reservado por un carrito activo no está disponible».

Solución comentada.

```
BOUNDED CONTEXT: Inventario — Reglas de Negocio OBLIGATORIAS

INVARIANTES (nunca violar):
1. disponible = stock_fisico - reservado; NUNCA vender si disponible <
cantidad
2. Una reserva expira a los 30 minutos y devuelve su cantidad a
disponible
3. stock_fisico solo cambia por recepción, venta confirmada o ajuste
auditado
4. Ningún ajuste manual sin id_auditoria
```

```
LENGUAJE UBICUO:
- "Disponible" = stock_fisico - reservado (NUNCA usar stock_fisico a
secas)
- "Reserva" = cantidad bloqueada por un carrito activo, con expiración
- "Ajuste" = corrección auditada de stock_fisico (merma, conteo, daño)

CONTEXTOS EXTERNOS (no implementar aquí):
- Precios y promociones -> PricingContext
- Pagos -> PaymentContext

ANTI-PATTERNS PROHIBIDOS:
- No leer stock_fisico para decidir una venta (usar disponible)
- No crear reservas sin expiración
```

Los aciertos a verificar contra su propia solución: (1) la regla del enunciado se descompuso en **invariantes atómicos y verificables**: cada uno puede convertirse en un test (capítulo 3 (Vol. 4)); (2) el término trampa (`stock_fisico` vs. `disponible`) quedó definido en el lenguaje ubicuo **y además prohibido** en los anti-patterns, la redundancia deliberada es defensa en profundidad contra el modelo «promedio» que usaría el campo obvio; (3) la expiración de reservas, que el enunciado no mencionaba pero la regla implica, está explícita: escribir el contexto es la actividad que *descubre* estas reglas tácitas, y ese descubrimiento es la mitad del valor del ejercicio.

Lo esencial del capítulo 6

- El modelo genera la solución «promedio»; su dominio nunca es promedio. Sin anclaje, alucina con fluidez (Ji et al., 2023; Bender et al., 2021).
- El DDD (Evans, 2003) es la ontología legible por máquina que el agente necesita: invariantes, lenguaje ubicuo, fronteras.
- El contexto vive en el repositorio, es prefijo cacheable (≈10 % del costo) y cada invariante tiene su test: tres prácticas, un sistema inmunológico.
- Dos capas de defensa: el contexto en el prompt (probabilística) y los verificadores en CI (determinista).

Preguntas para discusión

1. ¿Existe en su organización un documento de dominio que un agente pudiera consumir hoy? Si no, ¿quién es la persona que tiene ese conocimiento en la cabeza, y qué pasa si renuncia?
2. ¿Qué invariante de su negocio, violado por código generado a velocidad industrial, causaría el incidente más caro? ¿Está escrito en algún lugar que una máquina pueda leer?

Capítulo 7: Creación de skills, enseñar procedimientos a los agentes

Skills: el conocimiento procedimental del agente

Si los agentes son los trabajadores de la Empresa Sintética, los **skills** son sus especializaciones: procedimientos empaquetados que el agente puede invocar de forma consistente y reproducible. La idea responde a un problema real de ingeniería de contexto: las instrucciones especializadas no pueden vivir todas en el prompt de sistema (lo inflarían hasta degradar el rendimiento y el costo; Liu et al., 2024), pero tampoco pueden improvisarse en cada tarea. Un skill es la solución intermedia: una unidad de conocimiento procedimental con **divulgación progresiva**, donde su descripción breve está siempre disponible y su contenido completo se carga solo cuando la tarea lo requiere.

En Claude Code y en la plataforma de agentes de Anthropic, un skill es un archivo Markdown estructurado que define:

1. **Cuándo activarse** (el disparador, basado en la intención del usuario)
2. **Qué hacer** (el procedimiento, paso a paso)
3. **Qué herramientas usar** (el arsenal disponible)
4. **Qué salida producir** (el formato del resultado)

El concepto trasciende a un proveedor: OpenAI y Google ofrecen mecanismos análogos de instrucciones empaquetadas y herramien-

tas predefinidas. Lo que el AI Engineer debe dominar es el principio, no la sintaxis: **el conocimiento procedimental del equipo se versiona en el repositorio y se carga bajo demanda**.

La anatomía de un skill bien diseñado:

```
---
name: domain-review
description: |
  Revisa que el código generado respeta el bounded context del dominio.
  ACTIVAR cuando: el usuario pide revisar código de dominio, menciona
DDD,
  bounded context, o cuando se acaba de generar lógica de negocio.
  NO ACTIVAR para: código de infraestructura, UI, tests unitarios
simples.
tools:
  - Read
  - Grep
  - Bash
---

# Skill: Domain Review

## Objetivo
Verificar que el código respeta los bounded contexts definidos y no
viola
invariantes de dominio.

## Protocolo de Ejecución

### Paso 1: Identificar el Bounded Context
- Buscar el archivo de contexto: `grep -r "BOUNDED CONTEXT" docs/
domain/`
- Si no existe: crear uno básico basado en las entidades encontradas

### Paso 2: Análisis de Violaciones
Buscar estos anti-patterns específicos:
- Acceso directo a DB desde capa de dominio: `grep -r "db\." domain/`
- Entidades expuestas en controllers: verificar que usan DTOs
- Lógica de negocio en repositorios: verificar que solo hacen CRUD

### Paso 3: Verificar Invariantes
Para cada invariante listado en el contexto de dominio:
- Encontrar dónde se implementa en el código
- Verificar que existe un test que lo cubre
- Marcar como "sin cobertura" si no hay test
```

```
### Paso 4: Reporte
Producir reporte en este formato:
- RESUMEN: [1 línea: pass/fail + número de violaciones]
- VIOLACIONES CRÍTICAS: [invariantes rotos]
- ADVERTENCIAS: [anti-patterns encontrados]
- ACCIONES REQUERIDAS: [lista numerada de cambios]
```

Nótese que el skill anterior es la **operacionalización del capítulo 6**: el contexto de dominio define las reglas; el skill define el procedimiento para auditarlas. Esa composición, documentos de conocimiento más procedimientos que los consumen, es la forma madura de la ingeniería de contexto.

Los cinco patrones de skill más valiosos

```
Patrón 1: SKILL DE ANÁLISIS
──────────────────────────────

Propósito:  Inspección profunda de artefactos existentes
Disparador: "analiza", "revisa", "audita", "¿qué hace X?"
Estructura: Read -> Grep -> Análisis -> Reporte estructurado
Ejemplos:   security-review, domain-review, performance-audit

Patrón 2: SKILL DE GENERACIÓN
──────────────────────────────

Propósito:  Crear artefactos nuevos siguiendo plantillas
Disparador: "crea", "genera", "scaffoldea", "implementa X"
Estructura: Lee plantillas -> Adapta al contexto -> Escribe -> Valida
Ejemplos:   generate-api-endpoint, scaffold-domain-entity,
create-test-suite

Patrón 3: SKILL DE TRANSFORMACIÓN
──────────────────────────────

Propósito:  Convertir un artefacto del formato A al formato B
Disparador: "migra", "convierte", "refactoriza a X"
Estructura: Lee -> Analiza estructura -> Transforma -> Verifica
equivalencia
Ejemplos:   refactor-to-ddd, migrate-to-typescript, convert-to-async

Patrón 4: SKILL DE ORQUESTACIÓN
──────────────────────────────

Propósito:  Coordinar múltiples agentes o herramientas en secuencia
Disparador: "despliega", "pipeline completo", "proceso de inicio a
fin"
Estructura: Plan -> Lanza sub-agentes -> Recoge resultados -> Sintetiza
```

```
Ejemplos:   full-deploy-pipeline, code-review-council,
onboard-new-service

Patrón 5: SKILL DE VALIDACIÓN
──────────────────────────────

Propósito:  Verificar que invariantes y contratos se cumplen
Disparador: "verifica", "valida", "¿está correcto?", antes de
merge/deploy
Estructura: Define criterios -> Ejecuta checks -> Produce go/no-go
Ejemplos:   verify-api-contracts, validate-domain-invariants,
pre-deploy-check
```

► PISTA TÉCNICA: Creando su primer skill

```
# Estructura de directorios para skills en un proyecto Claude Code
mkdir -p .claude/skills

# Skill de ejemplo: auditor de costos de tokens
cat > .claude/skills/cost-audit.md << 'EOF'
---
name: cost-audit
description: |
  Audita el uso de tokens en el código y estima costos mensuales.
  ACTIVAR cuando: el usuario pregunta sobre costos, menciona
token-burn,
  quiere optimizar el gasto en IA, o antes de lanzar a producción.
tools:
  - Grep
  - Read
  - Bash
---

# Cost Audit Skill

## Objetivo
Identificar llamadas a APIs de LLM en el código y estimar su costo
mensual
basado en el uso esperado.

## Proceso

1. Encontrar todas las llamadas a APIs de modelos:
   `grep -rn "messages.create\|chat.completions\|generate_content"
--include="*.py" .`

2. Para cada llamada encontrada:
```

```
   - Identificar el modelo usado
   - Estimar tokens de entrada (system prompt + contexto típico)
   - Estimar tokens de salida (max_tokens configurado)
   - Calcular costo por llamada según la tabla de precios local

3. Estimar volumen mensual basado en:
   - Endpoints HTTP: revisar logs o asumir un volumen conservador
documentado
   - Jobs batch: leer configuración del scheduler
   - Llamadas síncronas: contar por usuario activo

4. Producir tabla: Función | Modelo | Tokens/call | Costo/call |
Costo/mes

## Tabla de precios
NO uses precios memorizados: lee el archivo `config/precios_llm.json`
del repositorio (actualizado trimestralmente por el equipo de
plataforma)
y, si no existe, indica al usuario que debe crearlo desde la
documentación
oficial del proveedor.
EOF
```

El detalle final del skill, prohibir precios memorizados y exigir un archivo de configuración versionado, ilustra un principio general: **los skills no deben contener datos perecederos**. Contienen procedimientos (estables) que leen datos (volátiles) de fuentes versionadas. Un skill con precios incrustados de 2026 seguirá susurrando esos precios en 2028.

■ **PISTA EJECUTIVA**

Los skills son la respuesta a una pregunta que toda dirección de ingeniería se hace: «¿cómo evitamos que el conocimiento de nuestros mejores ingenieros viva solo en sus cabezas?». La respuesta clásica, la documentación, fracasa porque la documentación no se ejecuta y, por tanto, se desactualiza sin que nadie lo note. Un skill es documentación que **se ejecuta**: cuando el procedimiento queda obsoleto, el skill falla de forma visible y alguien lo arregla. La biblioteca de skills de una organización es, literalmente, su know-how operativo convertido en activo digital: inventariable, versionable y transferible al siguiente empleado o al siguiente agente.

Skills limpios: plantillas y la checklist de revisión

El Código Limpio Sintético (capítulo 4) aplicado a los skills produce dos artefactos concretos. El primero es la **plantilla de skill**, el equivalente del prompt maestro del capítulo 5 para conocimiento procedimental: todo skill de la organización nace de la misma estructura (frontmatter con disparadores positivos y negativos, objetivo en una frase, protocolo numerado, formato de reporte), de modo que leer un skill ajeno cuesta lo mismo que leer el propio. Los cinco patrones de este capítulo son, precisamente, cinco plantillas: el equipo no redacta skills desde cero, instancia el patrón que corresponde.

El segundo es la **checklist de revisión de skills**, porque un skill entra al repositorio por *pull request* como cualquier código:

1. **Nombre y disparador.** ¿El nombre dice qué hace (`cost-audit`, no `utils`)? ¿La descripción dice cuándo activarse **y cuándo no**? El disparador negativo evita activaciones falsas, el error más común.
2. **Responsabilidad única.** ¿Encapsula UN procedimiento? Un skill que analiza «y de paso corrige» son dos skills mal mezclados.
3. **Sin datos perecederos.** ¿Precios, versiones, URLs cambiantes? Van a configuración versionada, no al skill.
4. **Herramientas mínimas.** ¿La lista `tools` es la mínima necesaria? Un skill de análisis no necesita permisos de escritura (mínimo privilegio, capítulo 4 (Vol. 4)).
5. **Salida verificable.** ¿El formato de reporte permite un eval automático (capítulo 3 (Vol. 4))?
6. **Probado.** ¿Existe al menos un caso de prueba con entrada y salida esperada en la suite?

Seis preguntas, dos minutos por revisión: la diferencia entre una biblioteca de skills y un cajón de prompts sueltos.

Ejercicio resuelto 7.1: un skill de validación pre-deploy

Enunciado. Instancie el patrón 5 (validación) en un skill `pre-deploy-check` que verifique, antes de cada despliegue: que los tests pasan, que no hay credenciales en el código y que el CHANGELOG fue actualizado. Aplique la checklist anterior.

Solución comentada.

```
---
name: pre-deploy-check
description: |
  Verifica condiciones de despliegue: tests, credenciales, changelog.
  ACTIVAR cuando: el usuario pida desplegar, mencione release o
pre-deploy.
  NO ACTIVAR para: despliegues a entornos locales o de experimentación.
tools:
  - Bash
  - Grep
  - Read
---

# Pre-Deploy Check

## Objetivo
Producir un go/no-go verificable antes de desplegar a producción.

## Protocolo
1. Tests: ejecutar la suite (`npm test` o `pytest -q` según el
proyecto);
   capturar el código de salida.
2. Credenciales: `grep -rEn
"(api[_-]?key|secret|password)\s*[:=]\s*['\"]"`
   sobre el código fuente, excluyendo tests y ejemplos documentados.
3. Changelog: verificar que CHANGELOG.md tiene una entrada cuya versión
   coincide con la del manifiesto del proyecto.

## Reporte (formato fijo, evaluable)
- VEREDICTO: GO | NO-GO
- CHECKS: [tests: pass/fail] [credenciales: limpio/hallazgos]
[changelog: ok/falta]
- BLOQUEANTES: [lista, vacía si GO]
```

Contra la checklist: (1) nombre y disparadores (incluido el negativo) explícitos; (2) una sola responsabilidad: *valida*, no corrige ni

despliega (corregir sería otro skill; desplegar, una acción crítica con guardia humana, capítulo 1 (Vol. 3)); (3) sin datos perecederos, los comandos de test se detectan del proyecto, no se fijan versiones; (4) tres herramientas de solo lectura y ejecución acotada, sin permisos de escritura; (5) el formato `VEREDICTO/CHECKS/BLOQUEANTES` es trivialmente evaluable por una aserción determinista; (6) el caso de prueba mínimo: un repositorio de juguete con una credencial plantada debe producir `NO-GO` con el hallazgo listado.

Lo esencial del capítulo 7

- Un skill = conocimiento procedimental versionado con divulgación progresiva: descripción siempre visible, contenido bajo demanda.
- Cinco patrones-plantilla: análisis, generación, transformación, orquestación, validación. Se instancian, no se improvisan.
- Los skills no contienen datos perecederos: procedimientos estables que leen configuración versionada.
- Checklist de revisión de seis puntos en cada PR de skill, es código, y se revisa como código.
- Un skill es documentación que se ejecuta: cuando queda obsoleto, falla visiblemente y alguien lo arregla.

Capítulo 8: Patrones de orquestación multi-agente

La arquitectura de la inteligencia distribuida

El sistema multi-agente es la culminación de todo lo anterior: múltiples agentes especializados, coordinados por un orquestador, producen resultados que ninguno podría producir individualmente. Wooldridge (2009) identifica las propiedades emergentes de los sistemas multi-agente: comportamientos que surgen de la interacción y que no estaban programados explícitamente en ningún agente individual. En ingeniería de software, las propiedades emergentes que buscamos son calidad, resiliencia y cobertura de perspectivas; las que debemos evitar son los bucles de acuerdo mutuo, la amplificación de errores y el costo descontrolado.

Antes de los patrones, la advertencia que la evidencia obliga a dar: **más agentes no es automáticamente mejor**. La evaluación rigurosa de agentes muestra que gran parte de las ganancias reportadas en benchmarks provienen de comparaciones sin control de costo, y que sistemas simples con presupuesto equivalente igualan a menudo a arquitecturas complejas Kapoor et al., 2024. La misma cautela aparece en la guía de ingeniería de Anthropic (2024): el éxito en producción no consiste en construir el sistema más sofisticado, sino el *adecuado*. Cada patrón siguiente se presenta, por eso, con su perfil de costo y su criterio de uso, y el capítulo 3 (Vol. 4) da las herramientas para verificar, con evals propios, que la complejidad adicional paga su precio.

Los cinco patrones canónicos de orquestación

Justificación de diseño: Diagrama de componentes para la arquitectura del sistema

Una topología multi-agente es ante todo *cómo interactúan los módulos especializados*: quién habla con quién, cómo se conectan, dónde está el flujo. Un diagrama de componentes (con sus puertos y dependencias) muestra exactamente eso: la topología arquitectónica del sistema.

Un diagrama de clases mostraría la estructura interna de un agente; un diagrama de estados mostraría cómo uno cambia en el tiempo. Aquí necesitamos ver *la conexión entre múltiples agentes*, el patrón de integración que gobernará toda su coordinación.

En este libro reaparece el diagrama de componentes en el **Capítulo 9** (el ecosistema MCP), donde nuevamente el patrón de módulos especializados es lo que interesa.

Figura 10: Diagrama 8.1.1: Patrón 1: Pipeline (diagrama de componentes)

Pipeline (cadena). Uso ideal: flujos predecibles con dependencias lineales. Ventaja: simple, auditable, fácil de depurar; cada eslabón puede validarse antes de pasar al siguiente. Desventaja: lento (secuencial) y un fallo detiene todo. Ejemplo: especificación → implementación → revisión → documentación. Es el patrón con mejor relación calidad/complejidad y debe ser la opción por defecto.

Figura 11: Diagrama 8.1.2: Patrón 2: Fan-out paralelo

Fan-out paralelo (seccionado). Uso ideal: tareas independientes que pueden ejecutarse simultáneamente. Ventaja: reduce el tiempo total de forma proporcional al paralelismo disponible. Desventaja: exige una fase de fusión (*merge*) que es, ella misma, una tarea de razonamiento; los resultados parciales pueden ser inconsistentes entre

sí. Ejemplo: generar frontend, backend y tests en paralelo a partir de un contrato de API común, el contrato previo es lo que hace al patrón viable.

Figura 12: Diagrama 8.1.3: Patrón 3: Debate y consenso (council)

Debate y consenso (council). Uso ideal: decisiones de alto impacto y evaluación de artefactos críticos. Varios agentes con perspectivas distintas opinan; un sintetizador decide. Su fundamento empírico es directo: el debate entre múltiples instancias mejora la factualidad y el razonamiento respecto al agente único Du et al., 2023. Desventaja: multiplica el costo por el número de consejeros; reservarlo para decisiones cuyo costo de error supere con holgura el costo del debate.

Figura 13: Diagrama 8.1.4: Patrón 4: Jerárquico (Empresa Sintética)

Jerárquico (orquestador-trabajadores). Uso ideal: proyectos complejos con múltiples dominios. Un orquestador descompone, delega a especialistas y sintetiza. Ventaja: escala naturalmente y refleja la organización real Conway, 1968. Desventaja: la coordinación añade latencia y tokens; el orquestador es un punto único de fallo de razonamiento. Es el patrón de ChatDev y MetaGPT (Qian et al., 2024; Hong et al., 2024).

Figura 14: Diagrama 8.1.5: Patrón 5: Self-reflection loop

Self-reflection loop (evaluador-optimizador). Uso ideal: cuando la calidad es crítica y **verificable automáticamente**: tests que pasan, esquemas que validan, criterios formales. Un generador produce; un evaluador critica; el generador corrige. Fundamento: Self-Refine Madaan et al., 2023 y Reflexion Shinn et al., 2023. Desventaja: sin criterio de parada degenera en bucle infinito; sin señal de calidad objetiva degenera en autocomplacencia: el modelo aprueba su

propia salida. Regla práctica: máximo de iteraciones explícito y evaluador distinto del generador.

► PISTA TÉCNICA: Implementación del patrón council

```
# Ejemplo 8.1: Council pattern para revisión arquitectónica.
# Tres consejeros especializados opinan en paralelo; un sintetizador
decide.
# Fundamento: el debate multi-agente mejora la factualidad [Du et al.,
2023](#ref-du2023).

import anthropic

client = anthropic.Anthropic()

COUNSELORS = {
    "scalability": """Eres un arquitecto especializado en
    escalabilidad y
    sistemas distribuidos. Evalúas desde: carga, latencia,
disponibilidad
    y operabilidad a escala. Sé conciso: máximo 300 palabras.
    Argumenta con datos y mecanismos, no con opiniones.""",
    "developer_experience": """Eres un arquitecto especializado en
    developer
    experience y simplicidad operacional. Evalúas desde: velocidad de
onboarding,
    facilidad de depuración, costo de mantenimiento y deuda técnica.
    Defiendes la tecnología aburrida y el principio YAGNI. Máximo 300
palabras.""",
    "economics": """Eres un arquitecto especializado en economía de
    sistemas.
    Evalúas TCO (costo total de propiedad), costos operacionales,
impacto en
    velocidad de entrega y ROI de decisiones técnicas. Piensas en
términos
    de Boehm (economía del software) y métricas DORA. Máximo 300
palabras.""",
}

def consult(role_system: str, problema: str) -> dict:
    resp = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        system=role_system,
        messages=[{"role": "user",
                   "content": f"Evalúa desde tu especialidad:
                   {problema}"}],
```

```
    )
    return {
        "text": next(b.text for b in resp.content if b.type == "text"),
        "tokens": resp.usage.input_tokens + resp.usage.output_tokens,
    }

def architecture_council(problema_arquitectonico: str) -> dict:
    """
    Patrón Council: 3 perspectivas especializadas debaten una decisión
    de arquitectura; el modelo más capaz sintetiza el veredicto final.
    En producción, las tres consultas se lanzan en paralelo.
    """
    views = {name: consult(system, problema_arquitectonico)
             for name, system in COUNSELORS.items()}

    synthesis = client.messages.create(
        model="claude-opus-4-8",   # el modelo más capaz para la
síntesis
        max_tokens=2048,
        system="""Eres el arquitecto jefe que debe tomar una DECISIÓN
FINAL
        basada en los argumentos de tres consejeros especializados. Tu
output:

        1. DECISIÓN: [una línea clara y sin ambigüedad]
        2. RAZÓN PRINCIPAL: [el argumento que inclina la balanza]
        3. CONDICIONES: [3 condiciones que harían cambiar esta
decisión]
        4. PRIMER PASO: [la acción concreta a tomar esta semana]
        5. RIESGO PRINCIPAL: [lo que puede salir mal y cómo mitigarlo]

        No resumas los argumentos. Ve directo a la decisión.""",
        messages=[{
            "role": "user",
            "content": (
                f"Problema: {problema_arquitectonico}\n\n"
                f"ESCALABILIDAD:\n{views['scalability']['text']}\n\n"
                f"DEVELOPER EXPERIENCE:\n{views[
                'developer_experience']['text']}\n\n"
                f"ECONOMÍA:\n{views['economics']['text']}\n\nToma tu
                decisión."
            ),
        }],
    )

    total_tokens = (sum(v["tokens"] for v in views.values())
```

```
                    + synthesis.usage.input_tokens
                    + synthesis.usage.output_tokens)

    return {
        "decision": next(b.text for b in synthesis.content if b.type
        == "text"),
        "perspectives": {k: v["text"] for k, v in views.items()},
        "meta": {
            "total_tokens": total_tokens,
            "agentes_consultados": len(COUNSELORS) + 1,
            "patron": "Council (debate + síntesis)",
        },
    }


# Ejemplo de uso:
# resultado = architecture_council(
#     "Tenemos un monolito Django de 5 años con 50K usuarios activos. "
#     "El equipo propone migrarlo a microservicios para 'escalar
mejor'. "
#     "¿Es la decisión correcta ahora?"
# )
# print(resultado["decision"])
# Output esperado: una recomendación clara, no un "depende".
```

Cómo elegir patrón: el árbol de decisión

1. ¿La tarea cabe en una sola llamada bien diseñada? → **No orqueste.** (La opción que los entusiastas olvidan.)
2. ¿Los pasos son conocidos de antemano? → **Pipeline.**
3. ¿Las subtareas son independientes entre sí? → **Fan-out** con contrato previo.
4. ¿La salida es verificable automáticamente? → **Self-reflection loop** con presupuesto.
5. ¿La decisión es cara de revertir y admite perspectivas? → **Council.**
6. ¿El problema exige descomposición dinámica de varios dominios? → **Jerárquico**. Y prepare la observabilidad del capítulo 1 (Vol. 4), porque la necesitará.

■ **PISTA EJECUTIVA**

La lección directiva de este capítulo cabe en una frase: **la arquitectura multi-agente es una decisión de inversión, no de moda**. Ca-

da patrón compra una propiedad (velocidad, robustez, cobertura de perspectivas) a un precio medible en tokens y en complejidad operativa. Exija a su equipo lo que exigiría ante cualquier inversión: la comparación contra la alternativa simple, con costo total y métricas de calidad en la misma tabla Kapoor et al., 2024. Cuando el equipo proponga un council de cinco agentes, la pregunta correcta es «¿qué obtuvimos respecto a un solo agente bien instruido, y cuánto nos costó?», no «¿funciona?» (casi todo funciona en la demo).

El ciclo de vida del agente como máquina de estados: rigor técnico de producción

Antes de describir cómo los orquestadores coordi coordinan múltiples agentes, es esencial entender cómo un agente individual transiciona entre estados. En producción, ese ciclo es un autómata finito determinista, no un diagrama conceptual: cada transición es observable, auditable y fallible de forma predecible. Esta es la diferencia entre un prototipo que funciona en la demo y un sistema agéntico que funciona en producción a escala.

En primer lugar, el ciclo del agente tiene exactamente cinco estados bien definidos: **Inactivo** (aguardando tarea), **Planificación** (descomposición y selección de herramientas), **Ejecución** (invocación de herramientas y APIs), **Evaluación** (validación de resultados y decisión de reintento), y **Completado** o **Fallido** (terminal). Cada transición es gatillada por un evento observable: recepción de tarea, plan seleccionado, observación registrada, presupuesto agotado.

Figura 15: Diagrama 8.2: Ciclo de vida del agente como máquina de estados, del Inactivo al Completado, con reintentos y paradas de seguridad

A continuación, la estructura del ciclo revela cuatro decisiones críticas que cualquier implementación debe resolver:

La representación del estado es política de negocio. El diagrama distingue entre transiciones que pertenecen al dominio («reintento requerido», «éxito») y transiciones que pertenecen a la infra-

estructura («presupuesto excedido», «error»). Esa distinción es lo que permite un log de auditoría que un humano o un auditor regulatorio pueden leer sin escarbar en code. El registro del evento debe incluir: timestamp, agente_id, estado_anterior, estado_nuevo, razón (contrato verificable), y tokens consumidos hasta ese punto.

La contrapresión previene cascadas. Cuando el agente entra en la fase de Ejecución, no todas las herramientas pueden ser invocadas. El servidor MCP (capítulo 9) que le brinda acceso a las herramientas debe ser capaz de rechazarlas si violarían presupuesto, permisos o invariantes de dominio (capítulo 6). Ese rechazo es una transición de **Ejecución → Evaluación** con razón «herramienta rechazada por guardarraíles». El modelo, al recibirlo, debe saber que ese es un NO definitivo, no una opinión.

Primero, la observación cierra el bucle. Cuando una herramienta falla (la API externa está caída, el resultado es inválido), la transición natural es **Ejecución → Evaluación**. El evaluador, en Evaluación, debe decidir: ¿reintento? (la red se recupera), ¿cambiar de herramienta? (la especificación era ambigua), ¿terminal? (presupuesto agotado). Sin una observación clara de por qué falló, esa decisión es adivinanza. De ahí que el contrato de la herramienta (capítulo 4) incluya la estructura del error: `{ "is_error": true, "reason": "external_api_timeout", "recovery_hint": "retry in 30 seconds" }`.

Finalmente, el presupuesto no es una sugerencia. Cuando el agente ha consumido N iteraciones, o M tokens, o el tiempo T ha transcurrido, la máquina de estados salta a **Evaluación → Fallido** automáticamente, sin pedir permiso. La gobernanza (capítulo 1 (Vol. 4)) inyecta este límite en la máquina misma, no como una validación de aplicación: si el contador alcanza el tope, la transición no es posible.

► **PISTA TÉCNICA: Máquina de estados en TypeScript con presupuesto integrado**

```
// agent-state-machine.ts
// Implementación de la máquina de estados del agente con
auditabilidad incorporada.
// La política de presupuesto vive en el estado, no en el resto del
código.
```

```
enum AgentState {
  IDLE = "IDLE",
  PLANNING = "PLANNING",
  EXECUTING = "EXECUTING",
  EVALUATING = "EVALUATING",
  COMPLETED = "COMPLETED",
  FAILED = "FAILED",
}

interface StateTransition {
  timestamp: Date;
  fromState: AgentState;
  toState: AgentState;
  reason: string;
  tokensConsumedTotal: number;
  agentId: string;
}

interface AgentBudget {
  maxIterations: number;
  maxTokens: number;
  maxTimeMs: number;
  currentIterations: number;
  currentTokens: number;
  startTimeMs: number;
}

class AgentStateMachine {
  private state: AgentState = AgentState.IDLE;
  private transitions: StateTransition[] = [];
  private budget: AgentBudget;
  private llmPort: LlmPort; // del capítulo 6 (Vol. 4)

  constructor(
    agentId: string,
    maxIterations: number = 10,
    maxTokens: number = 100000,
    maxTimeMs: number = 300000 // 5 minutos
  ) {
    this.budget = {
      maxIterations,
      maxTokens,
      maxTimeMs,
      currentIterations: 0,
      currentTokens: 0,
      startTimeMs: Date.now(),
```

```
    };
  }

  // Transición: solo si es válida según presupuesto
  private canTransitionTo(nextState: AgentState): boolean {
    const elapsed = Date.now() - this.budget.startTimeMs;
    if (elapsed > this.budget.maxTimeMs) return false;
    if (this.budget.currentIterations >= this.budget.maxIterations)
      return false;
    if (this.budget.currentTokens >= this.budget.maxTokens) return
    false;
    return true;
  }

  async executeTask(input: string): Promise<string> {
    // Transición: IDLE → PLANNING
    this._transition(AgentState.PLANNING, "task.received");

    const plan = await this.llmPort.complete(
      "Planifica la tarea...",
      input,
      "BALANCED"
    );
    this.budget.currentTokens += plan.inputTokens + plan.outputTokens;

    while (true) {
      this.budget.currentIterations++;

      // Guardarraíles: ¿se puede seguir?
      if (!this.canTransitionTo(AgentState.EXECUTING)) {
        this._transition(
          AgentState.EVALUATING,
          "budget.exhausted"
        );
        break;
      }

      // Transición: PLANNING → EXECUTING
      this._transition(AgentState.EXECUTING, "plan.selected");

      const toolResult = await this.executeTool(plan);
      this.budget.currentTokens += toolResult.tokensUsed;

      // Transición: EXECUTING → EVALUATING
      this._transition(
        AgentState.EVALUATING,
```

```
      "observation.received"
    );

    // Evaluación: ¿reintentamos?
    if (toolResult.isError) {
      if (this.budget.currentIterations < this.budget.maxIterations)
      {
        // Reintento: EVALUATING → PLANNING
        this._transition(AgentState.PLANNING, "retry.required");
        continue;
      } else {
        // Presupuesto agotado: EVALUATING → FAILED
        this._transition(AgentState.FAILED,
        "max.iterations.reached");
        break;
      }
    } else {
      // Éxito: EVALUATING → COMPLETED
      this._transition(AgentState.COMPLETED, "success");
      return toolResult.content;
    }
  }

  // Si llegamos aquí, fue un fallo
  throw new Error(
    `Agent failed: ${this.state}. ` +
    `Iterations:
    ${this.budget.currentIterations}/${this.budget.maxIterations}, `
    +
    `Tokens: ${this.budget.currentTokens}/${this.budget.maxTokens}`
  );
}

private _transition(
  nextState: AgentState,
  reason: string
): void {
  const transition: StateTransition = {
    timestamp: new Date(),
    fromState: this.state,
    toState: nextState,
    reason,
    tokensConsumedTotal: this.budget.currentTokens,
    agentId: this.constructor.name,
  };
```

```
    // Log estructurado: auditoría inmediata
    console.log(
      JSON.stringify(transition, null, 2)
    );
    this.transitions.push(transition);

    this.state = nextState;
  }

  // Recurso para auditoría: el historial de transiciones
  getAuditTrail(): StateTransition[] {
    return [...this.transitions];
  }
}
```

La virtud de este diseño es que el presupuesto no es un comentario en el código: es una estructura que la máquina de estados consulta en cada iteración, automáticamente. Un agente que excede presupuesto no «continúa por error»; no puede transicionar, punto. Esa restricción vive cerca del estado, donde pertenece.

Arquitectura empresarial de agentes: la plataforma de producción

Cuando múltiples agentes especializados deben orquestar flujos complejos en una organización (decenas de equipos, latencias variables, gobernanza de IA, auditoría regulatoria) el diseño no puede ser ad-hoc. Emerge una necesidad arquitectónica clara: una **plataforma de agentes** que separe preocupaciones en capas lógicas, cada una con su responsabilidad única, su contrato explícito y su mecanismo de observabilidad.

Figura 16: Diagrama 8.3: Arquitectura de Plataformas Agénticas de Producción: Las cuatro capas de la orquestación empresarial (Abstracción de Modelos, Núcleo de Ejecución, Eventos y Evaluación, Observabilidad Transversal)

La arquitectura que el diagrama revela es una evolución natural de los cinco patrones que presentamos en la primera sección, y de

la máquina de estados que acaba de describirse. Primero, organiza explícitamente la división de responsabilidades:

Capa de Abstracción de Modelos (AI Gateway). Aquí reside toda la lógica que media entre los agentes y los proveedores de LLMs externos. El gateway cumple cuatro funciones simultáneamente: (1) **resiliencia**, encolando solicitudes fallidas para reintento automático; (2) **caché de prefijos**, reutilizando fragmentos de contexto estables a costo reducido; (3) **control de cuotas**, bloqueando nuevas llamadas si se alcanzó el presupuesto de tokens o dinero; (4) **polimorfismo de modelos**, enrutando solicitudes al modelo más económico o más capaz según el tier de tarea (capítulo 3). El axioma arquitectónico que gobierna esta capa es: **el agente nunca debe "saber" quién responde sus preguntas**.

A continuación, el Núcleo de Ejecución del Agente (Orquestador + MCP). Aquí vive la máquina de estados que ya describimos, el bucle ReAct (capítulo 4) y la capacidad de delegación a especialistas. El Orquestador es el corazón: recibe una tarea, descompone si es necesario (patrón jerárquico del capítulo 8), delega a especialistas y sintetiza. El Servidor MCP es el sistema nervioso: expone las herramientas que los agentes pueden invocar, validando permisos, esquemas y rechazando operaciones que violarían invariantes del dominio (capítulo 6). Juntos, son el lugar donde vive la **política de negocio pura**, sin detalles de infraestructura.

Además, el Ecosistema de Eventos y Evaluación Continua. Cuando un agente completa una tarea, no desaparece en silencio. Publica un evento en Kafka: quién ejecutó qué, con qué costo, con qué resultado. Ese evento fluye hacia dos destinos: (1) consumidores de negocio que actúan sobre el resultado (el siguiente paso en un pipeline); (2) el Eval Harness, que analiza si la respuesta cumple umbrales de calidad, detecta alucinaciones, y si encuentra problemas, publica a la DLQ para revisión humana. El patrón es: **la evaluación es continua, no posterior**; si el agente degrada, lo sabemos en minutos, no en la próxima auditoría trimestral.

Finalmente, la Capa de Telemetría Transversal (Observabilidad). Cada paso (cada transición de estado, cada llamada al modelo, cada invocación de herramienta) genera spans que OpenTelemetry (capítulo 8 (Vol. 4)) agrega en trazas distribuidas. La razón: cuando un

cliente reporta «mi solicitud tardó 5 minutos», necesitamos saber en segundos si fue el agente, el modelo, la base de datos remota, o una combinación. Esa traza es la única fuente de verdad para el debugging en producción.

► **PISTA TÉCNICA: Cómo las capas se comunican sin acoplamiento**

```
// PlatformAgentOrchestrator.java
// El orquestador de la plataforma: cómo coordina las capas sin conocer
// sus internales. Los puertos (interfaces) son el único contrato.

@Service
public class PlatformAgentOrchestrator {

    private final LlmPort llmGateway;              // Capa de
    Abstracción
    private final MCP_ServerPort toolExecutor;     // Núcleo
    private final EventBusPort eventPublisher;     // Eventos
    private final EvalHarnessPort evaluator;       // Evaluación
    private final ObservabilityPort tracer;        // Telemetría

    @Transactional
    public AgentResult executeTask(TaskRequest task) {
        // Crear traza raíz para esta tarea
        Span rootSpan = tracer.startSpan("agent.task",
            task.agentId, task.taskId);

        try {
            // PASO 1: Planificación (máquina de estados: PLANNING)
            rootSpan.addEvent("state.planning");
            Plan plan = generatePlan(task);

            // PASO 2: Ejecución (máquina de estados: EXECUTING)
            rootSpan.addEvent("state.executing");
            Result result = executeWithRetries(plan, rootSpan);

            // PASO 3: Publicar evento (desacoplamiento)
            eventPublisher.publish(new AgentExecutedEvent(
                taskId: task.taskId,
                agentId: task.agentId,
                result: result,
                tokensUsed: result.tokenMetrics,
                timestamp: Instant.now()
            ));
```

```
            // PASO 4: Evaluación asíncrona
            evaluator.asyncEval(result, task.agentId);

            rootSpan.addEvent("state.completed");
            return result;

        } catch (Exception e) {
            rootSpan.recordException(e);
            rootSpan.addEvent("state.failed");

            // Publicar a DLQ para revisión
            eventPublisher.publishToDLQ(new AgentFailedEvent(
                taskId: task.taskId,
                error: e.getMessage(),
                context: task
            ));
            throw e;
        } finally {
            rootSpan.end();
        }
    }

    private Result executeWithRetries(Plan plan, Span span) {
        int attempt = 0;
        while (attempt < MAX_ATTEMPTS) {
            try {
               // El gateway abstrae los reintentos internos y la caché
                LlmResponse response = llmGateway.complete(
                    plan.systemPrompt,
                    plan.userPrompt,
                    plan.tier
                );

                if (response.requiresToolUse()) {
                    // El MCP Server valida y ejecuta herramientas
                    ToolResult toolResult = toolExecutor.executeTool(
                        response.toolName,
                        response.toolInput,
                        plan.domainContext  // Invariantes del dominio
                    );

                    span.addEvent("tool.executed",
                        Map.of("tool", response.toolName,
                               "success", !toolResult.isError));

                  // El resultado vuelve al agente (máquina de estados)
```

```
                    plan = plan.withObservation(toolResult);
                } else {
                    // El agente alcanzó conclusión
                    return new Result(response.text, response.tokens);
                }

                attempt++;
            } catch (RateLimitException e) {
                // El gateway maneja el rate limiting
                span.addEvent("rate_limit",
                    Map.of("retry_after_ms", e.retryAfterMs));
                Thread.sleep(e.retryAfterMs);
                attempt++;
            }
        }

        throw new BudgetExhaustedException(
            "Max attempts reached for task " + plan.taskId);
    }
}
```

El poder de esta estructura es que cada capa puede evolucionarse sin tocar las otras. Si mañana decides cambiar de Kafka a Pulsar, el Orquestador no se entera: solo usa el EventBusPort. Si Claude lanza un modelo nuevo, el Gateway se configura sin recompilar el Orquestador. Si el Eval Harness necesita añadir una nueva métrica de calidad, la DLQ la captura y alimenta el aprendizaje. La **extensibilidad nace del desacoplamiento**.

Orquestadores limpios: los cuatro anti-patrones

El orquestador es a la Empresa Sintética lo que la raíz de composición es a un sistema con inyección de dependencias: el único lugar donde se decide quién habla con quién. El Código Limpio Sintético (capítulo 4) identifica sus cuatro degeneraciones recurrentes:

1. **El dios-orquestador.** El orquestador que, además de coordinar, opina, corrige y reescribe los resultados de sus especialistas. Es la god class otra vez: cuando el sintetizador del council empieza a «mejorar» el código del coder, hay dos roles mal mezclados, y un eval que ya no sabe a quién atribuir los fallos.

Regla: el orquestador decide y compone; no produce contenido de dominio.

2. **Los agentes charlatanes.** Eslabones que se pasan el contexto completo unos a otros «por si acaso». Cada salto duplica tokens y degrada la señal Liu et al., 2024. Regla: entre agentes viajan **contratos** (el JSON del prompt maestro, capítulo 5), no transcripciones.
3. **El bucle de cortesía.** Dos agentes que se validan mutuamente hasta el límite de presupuesto («buen punto», «de acuerdo contigo»). Es el modo de fallo conocido del debate sin estructura: el consenso prematuro. Regla: los debates tienen rondas fijas, roles adversariales explícitos y un sintetizador con autoridad para cerrar Du et al., 2023.
4. **La jerarquía infinita.** Orquestadores que delegan en orquestadores que delegan. Cada nivel añade latencia, costo y un punto de fallo de razonamiento; la evidencia de los marcos publicados es consistente en que un solo nivel de delegación captura casi todo el valor (Qian et al., 2024; Hong et al., 2024). Regla: un nivel de profundidad salvo justificación medida.

AOP para agentes: propagación de errores como aspecto transversal

Los cinco patrones y los cuatro anti-patrones anteriores comparten un punto ciego: ninguno dice qué hace el sistema cuando un eslabón del pipeline, una rama del fan-out o un consejero del council simplemente falla. Sin un diseño deliberado, cada agente termina reimplementando su propio *try/except*, su propio reintento, su propio formato de log, y cuando el orquestador crece hacia los enjambres de 20 a 96 agentes del capítulo 5 (Vol. 4), esa duplicación se convierte en el mayor foco de inconsistencia del sistema: un mismo tipo de fallo se registra distinto según quién lo capturó primero.

La Programación Orientada a Aspectos (AOP), formalizada por Kiczales et al. (1997) para separar *cross-cutting concerns* (logging, seguridad, transacciones) de la lógica de negocio en sistemas empresariales, encuentra en la orquestación de agentes un caso de uso casi de manual: interceptar cada llamada a herramienta o invocación de

agente con un comportamiento común, sin que el agente mismo necesite saber que ese comportamiento existe.

Los tres aspectos mínimos que todo orquestador de producción necesita:

1. **Aspecto de captura.** Envuelve cada llamada y normaliza cualquier excepción a un tipo común (`AgentError`) con metadata: qué agente, qué herramienta, qué intento, qué patrón de los cinco anteriores estaba en curso.
2. **Aspecto de propagación.** Decide si el error sube al llamador sin modificar (falla dura), se convierte en una entrada degradada para el sintetizador (falla suave), o dispara un reintento con backoff, la misma taxonomía del presupuesto de degradación del capítulo 5 del Vol. 4, caso 8.
3. **Aspecto de trazabilidad.** Adjunta el error a la traza OpenTelemetry (capítulo 1 (Vol. 4), Vol. 4) con el mismo esquema que una ejecución exitosa, para que la auditoría de reproducción cruzada (Vol. 3, capítulo 20) pueda comparar fallos entre proveedores con el mismo rigor con que compara éxitos.

```
def con_aspecto_de_errores(politica_degradacion):
    """Decorador que separa el manejo de errores de la lógica del
    agente."""
    def decorador(func_agente):
        @functools.wraps(func_agente)
        async def envoltura(*args, **kwargs):
            intento = 0
            while True:
                try:
                    resultado = await func_agente(*args, **kwargs)
                    registrar_traza(func_agente.__name__, "exito",
intento)
                    return resultado
                except Exception as e:
                    error = AgentError.desde(e,
agente=func_agente.__name__, intento=intento)
                    registrar_traza(func_agente.__name__, "error",
intento, error)
                    accion = politica_degradacion.decidir(error,
intento)
                    if accion == "reintentar":
                        intento += 1
                        await asyncio.sleep(backoff(intento))
```

```
                    continue
                if accion == "degradar":
                    return ResultadoDegradado(motivo=error)
                raise error  # falla dura: sube tal cual al
                orquestador
        return envoltura
    return decorador

@con_aspecto_de_errores(politica_degradacion=POLITICA_ESTANDAR)
async def agente_investigador(consulta):
    ...
```

El decorador es agnóstico del framework: el mismo aspecto envuelve una llamada al Task tool de Claude, un handoff de OpenAI Agents SDK, o un nodo de un `ParallelAgent` de Google ADK (capítulo 5, Vol. 4), la lógica de negocio del agente no cambia una línea al migrar de proveedor, que es, otra vez, el Backend Agnóstico sostenido a nivel de código, no solo de arquitectura.

La política de degradación no es una sola: cada patrón exige una por defecto distinta. El pipeline, por diseño, detiene todo ante una falla dura (esa rigidez es su ventaja documentada más arriba). El fan-out debe degradar en lugar de fallar: perder una fuente no debería cancelar la síntesis, siempre que el hueco quede documentado. El council debe hacer visible la degradación al sintetizador («2 de 3 consejeros respondieron»), nunca silenciarla como si el quórum completo hubiera opinado. El jerárquico debe propagar la falla del trabajador como un `AgentError` estructurado, no como una excepción cruda, para que el orquestador decida con la misma información que tendría un humano. El self-reflection loop debe fallar cerrado: si el evaluador mismo falla, la salida por defecto es "no aprobado", nunca "aprobado por ausencia de objeción", el mismo principio de la valla de permisos del capítulo 2 (Vol. 3).

Por qué esto es el camino hacia la fluidez, no un costo adicional. Un enjambre que maneja sus errores de forma transversal e inconsistente no es lento porque el modelo sea lento: es lento porque cada incidente exige que un ingeniero rastree manualmente en qué capa se perdió el error. La velocidad que hoy asociamos con GPT-6 Astra (corridas de decenas de agentes sostenidas durante horas) depende tanto del aspecto de errores como del modelo detrás: sin captura y propagación uniformes, un enjambre de esa escala simplemente no

sobrevive lo suficiente para completar la tarea. La excelencia técnica y la velocidad de negocio, en este caso, son la misma disciplina de ingeniería vista desde dos ángulos.

(*Caso compuesto ilustrativo; cifras de orden de magnitud, calculadas con los precios del apéndice B.*)

Una organización debe decidir si migra su monolito a microservicios, la pregunta del ejemplo 8.1. El arquitecto ejecuta el council tres veces para verificar estabilidad de la recomendación (una práctica barata que casi nadie usa: tres ejecuciones siguen costando centavos).

La traza, condensada. El consejero de escalabilidad cuantifica: el monolito sirve 50.000 usuarios con p95 estable; el cuello real es la base de datos, no el código. El de developer experience advierte: el equipo de doce personas no tiene experiencia operando sistemas distribuidos; la migración consumiría dos trimestres de su capacidad. El económico calcula el costo de oportunidad contra el backlog actual. El sintetizador produce: **DECISIÓN: no migrar ahora; extraer únicamente el módulo de reportes** (la fuente real de presión), con tres condiciones medibles que reabrirían la decisión y un primer paso para la semana.

La cuenta. Cuatro llamadas (tres consejeros en el modelo balanceado, síntesis en el premium), unos 18.000 tokens en total por ejecución: del orden de **15 a 20 centavos de dólar por corrida**, menos de un dólar por las tres. El contraste pedagógico: la alternativa humana (tres arquitectos senior, una reunión de dos horas) cuesta tres órdenes de magnitud más, y no deja traza estructurada. El council no reemplaza esa conversación humana: la **prepara**, entregando a los humanos un mapa de argumentos con sus tensiones ya explícitas. Ahí está su ROI real, y por eso el patrón se reserva para decisiones caras de revertir: en una decisión trivial, ni siquiera 20 centavos se justifican frente a una llamada simple.

Ejercicio resuelto 8.1: elija el patrón

Enunciado. Para cada escenario, elija el patrón de orquestación y justifique: (a) traducir 2.000 descripciones de producto a tres idiomas; (b) decidir el esquema de particionado de una base de datos

que costará semanas revertir; (c) generar una función con sus tests hasta que la suite pase.

Solución comentada.

- **(a) Ninguno: o fan-out trivial.** Son 6.000 tareas independientes y verificables por separado: una llamada bien diseñada por unidad, ejecutada por la API batch a mitad de precio (capítulo 1 (Vol. 4)). Quien proponga aquí un sistema multi-agente está comprando complejidad sin propiedad alguna a cambio: la primera rama del árbol de decisión existe precisamente para este caso.
- **(b) Council.** Decisión cara de revertir, con perspectivas legítimamente en tensión (rendimiento, operación, costo) y sin verificador automático posible. El debate estructurado mejora la factualidad de la recomendación Du et al., 2023 y el costo del patrón es despreciable frente al costo del error: el caso de estudio anterior es esta misma situación.
- **(c) Self-reflection loop.** Existe una señal de éxito objetiva y automática (la suite pasa), que es el requisito del patrón (Shinn et al., 2023; Madaan et al., 2023). Con presupuesto: máximo N iteraciones, y el evaluador es el test runner, determinista, no otro LLM opinando.

El meta-aprendizaje del ejercicio: la pregunta nunca es «¿qué patrón es mejor?» sino «¿qué propiedad necesito comprar y cuál es la señal de calidad disponible?». Verificabilidad automática → bucle; tensión de perspectivas → council; independencia → lote; pasos conocidos → pipeline.

Lo esencial del capítulo 8

- Más agentes no es mejor: sin control de costo, las comparaciones son marketing Kapoor et al., 2024. El patrón correcto es el mínimo que compra la propiedad necesaria.
- Cinco patrones canónicos: pipeline (defecto), fan-out (independencia + contrato), council (decisiones caras), jerárquico (descomposición dinámica), self-reflection (señal verificable + presupuesto).

- **Máquina de estados de producción:** el ciclo del agente (Inactivo → Planificación → Ejecución → Evaluación → Completado/Fallido) no es conceptual; es un autómata observable, auditable y con guardarraíles de presupuesto integrados en la estructura, no como validaciones posteriores.
- **Presupuesto como política codificada:** transiciones bloqueadas, no sugerencias ignoradas. Si maxIterations se alcanza, la máquina salta a Fallido sin pedir permiso.
- **Arquitectura empresarial de cuatro capas:** (1) Abstracción de Modelos (gateway con resiliencia y caché), (2) Núcleo de Ejecución (orquestador + MCP), (3) Eventos y Evaluación Continua (Kafka + Eval Harness), (4) Observabilidad Transversal (trazas distribuidas). El desacoplamiento entre capas es el axioma: cambiar un proveedor de LLM no debe recompilar el orquestador.
- Cuatro anti-patrones: dios-orquestador, agentes charlatanes, bucle de cortesía, jerarquía infinita.
- Entre agentes viajan contratos, no transcripciones.
- AOP separa el manejo de errores de la lógica de negocio en tres aspectos (captura, propagación, trazabilidad); cada uno de los cinco patrones exige una política de degradación por defecto distinta, y esa uniformidad (no un modelo más rápido) es lo que sostiene un enjambre a la escala de Astra.
- El council prepara la decisión humana con argumentos trazables por centavos; no la sustituye.
- **Auditoría estructurada:** cada transición de estado, cada evento, cada herramienta invocada registra quién, cuándo, costo y resultado. La DLQ captura los fallos; la traza los explica. Esa es la base de compliance, debugging y evaluación post-incidente.

Capítulo 9: MCP: el protocolo que une el mundo agéntico

El USB-C de la inteligencia artificial

El **Model Context Protocol** (MCP) es el estándar abierto, publicado por Anthropic a finales de 2024, que define cómo los agentes de IA se conectan con el mundo exterior: bases de datos, APIs, sistemas de archivos, servicios internos Anthropic, 2024b. Es, en esencia, el contrato de integración de la era agéntica, y su adopción desbordó rápidamente a su creador: durante 2025, los principales proveedores y herramientas del ecosistema anunciaron soporte para el protocolo, convirtiéndolo en el candidato más sólido a estándar de facto.

El problema que resuelve es el clásico M×N: antes de MCP, conectar M aplicaciones de IA con N sistemas externos exigía M×N integraciones a medida, con formatos propietarios incompatibles. MCP lo reduce a M+N: cada aplicación implementa el protocolo una vez (como *cliente*) y cada sistema lo implementa una vez (como *servidor*). Es la misma economía que estandarizó los puertos físicos (de ahí el apodo de «USB-C de la IA») y, antes, la que hizo de HTTP el idioma de la web.

Técnicamente, MCP es un protocolo basado en JSON-RPC 2.0 que expone tres primitivas:

- **Tools** (herramientas): funciones que el agente puede invocar: consultar una base de datos, crear un ticket, ejecutar una búsqueda.

- **Resources** (recursos): contenido que el agente puede leer: archivos, registros, documentación.
- **Prompts** (plantillas): instrucciones parametrizables que el servidor ofrece al cliente.

Justificación de diseño: diagrama de componentes para arquitectura del sistema

MCP es ante todo un *ecosistema modular*: múltiples clientes (aplicaciones de IA) que se conectan con múltiples servidores (sistemas externos) a través de un protocolo estándar. Un diagrama de componentes revela exactamente eso: las cajas (aplicación, servidor MCP), los puertos (Tools, Resources, Prompts), las conexiones desacopladas.

Un diagrama de clases mostraría la estructura interna de un cliente o servidor; un diagrama de secuencias mostraría un flujo específico de interacción. Aquí necesitamos ver *la arquitectura modular del ecosistema*, cómo se conectan los piezas sin conocer detalles de la otra.

El diagrama de componentes reaparece en este libro en el **Capítulo 8** (topologías multi-agente), donde nuevamente usamos componentes para mostrar cómo se conectan múltiples módulos especializados.

Figura 17: Diagrama 9.1: Ecosistema MCP (diagrama de componentes)

► **PISTA TÉCNICA: Construyendo un MCP server de producción**

```
# mcp_server_empresa.py
# MCP Server que expone herramientas del sistema interno al agente.
# Requiere: pip install mcp

import asyncio
import json
```

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

server = Server("sistema-empresa-v1")

# Definición de herramientas: la descripción ES el prompt del agente
@server.list_tools()
async def list_tools():
    return [
        Tool(
            name="consultar_estado_ticket",
            description="""Consulta el estado de un ticket en el
sistema de gestión de proyectos.
            USAR cuando el usuario pregunta sobre el estado de una
tarea, issue, o ticket.
            Retorna: estado actual, asignado, última actualización, y
bloqueos si existen.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "ticket_id": {
                        "type": "string",
                        "description": "ID del ticket (formato:
                        PROJ-123)",
                    }
                },
                "required": ["ticket_id"],
            },
        ),
        Tool(
            name="ejecutar_query_db",
           description="""Ejecuta una query de SOLO LECTURA en la base
de datos.
            SOLO permite SELECT. Rechaza automáticamente INSERT,
UPDATE, DELETE, DROP.
            USAR para: verificar datos, auditar estado, depurar
problemas de producción.
            NO USAR para: modificar datos (usar la API REST para
eso).""",
            inputSchema={
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
```

```
                        "description": "Query SQL de solo lectura
                        (SELECT)",
                    },
                    "database": {
                        "type": "string",
                       "enum": ["produccion", "staging", "analytics"],
                        "description": "Ambiente de base de datos",
                    },
                },
                "required": ["query", "database"],
            },
        ),
        Tool(
            name="obtener_metricas_sistema",
            description="""Obtiene métricas de performance del sistema
en tiempo real.
            Incluye: latencia de endpoints, tasa de errores, uso de
CPU/memoria,
            y alertas activas. USAR para diagnóstico de incidentes o
revisiones de salud.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "servicio": {
                        "type": "string",
                        "description": "Nombre del servicio (ej:
                        'api-gateway')",
                    },
                    "ventana_minutos": {
                        "type": "integer",
                        "description": "Ventana de tiempo en minutos
                        (default: 15)",
                        "default": 15,
                    },
                },
                "required": ["servicio"],
            },
        ),
    ]

@server.call_tool()
async def call_tool(name: str, arguments: dict):

    if name == "consultar_estado_ticket":
        ticket_id = arguments["ticket_id"]
        # Integración real con Jira/Linear/etc.
```

```
        ticket_data = await ticket_service.get(ticket_id)
        return [TextContent(
            type="text",
            text=json.dumps({
                "id": ticket_id,
                "estado": ticket_data.status,
                "asignado": ticket_data.assignee,
                "ultima_actualizacion":
                ticket_data.updated_at.isoformat(),
                "bloqueadores": ticket_data.blockers or [],
            }, indent=2, ensure_ascii=False),
        )]

    elif name == "ejecutar_query_db":
        query = arguments["query"].strip()

        # Guardarraíl de seguridad: solo SELECT, y validado en el
        SERVIDOR.
        # Nunca confíe en que el modelo respetará la descripción de la
        # herramienta: la descripción guía, el servidor impone.
        if not query.upper().startswith("SELECT") or ";" in
        query.rstrip(";"):
            return [TextContent(
                type="text",
                text="ERROR: Solo se permite una única query SELECT. "
                     "Para modificar datos, usa la API REST del
                     sistema.",
            )]

        database = arguments["database"]
        # Defensa adicional: usuario de BD de solo lectura + timeout
        results = await db_service.execute_readonly(query, database)
        return [TextContent(type="text",
                            text=json.dumps(results, indent=2,
default=str))]

    elif name == "obtener_metricas_sistema":
        servicio = arguments["servicio"]
        ventana = arguments.get("ventana_minutos", 15)
        metricas = await metrics_service.get(servicio, ventana)
        return [TextContent(type="text", text=json.dumps(metricas,
        indent=2))]

async def main():
    async with stdio_server() as (read_stream, write_stream):
        await server.run(read_stream, write_stream,
```

```
                        server.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())
```

Registro del MCP server en Claude Code:

```
// .mcp.json (configuración del proyecto)
{
  "mcpServers": {
    "sistema-empresa": {
      "command": "python",
      "args": ["mcp_server_empresa.py"],
      "env": {
        "DB_CONNECTION_STRING": "${DB_CONNECTION_STRING}",
        "TICKET_API_KEY": "${TICKET_API_KEY}"
      }
    }
  }
}
```

La seguridad de MCP es la seguridad de su organización

Un servidor MCP es, por definición, un puente entre un modelo probabilístico y sus sistemas reales. Cuatro reglas no negociables, que anticipan el capítulo 4 (Vol. 4):

1. **El servidor impone, la descripción sugiere.** Toda validación (solo lectura, límites, permisos) vive en el código del servidor. La descripción de la herramienta es un prompt, y los prompts se pueden manipular Greshake et al., 2023.
2. **Mínimo privilegio por construcción.** La herramienta de consulta usa un usuario de base de datos de solo lectura; la credencial con permisos de escritura ni siquiera existe en ese proceso.
3. **Los resultados de las herramientas son entrada no confiable.** Lo que vuelve de un sistema externo (una página web, un ticket escrito por un tercero) puede contener instrucciones inyectadas dirigidas al modelo. El capítulo 4 (Vol. 4) desarrolla el ataque y las defensas.
4. **Servidores de terceros se auditan como dependencias.** Instalar un MCP server es conceder a su agente las capacidades de

ese servidor; la cadena de suministro de servidores MCP merece el mismo escrutinio que la de paquetes de software.

■ **PISTA EJECUTIVA**

Para la dirección de tecnología, MCP convierte una pregunta recurrente («¿integramos la IA con el sistema X?») en una decisión de catálogo: ¿existe ya un servidor MCP para X, lo construimos, o esperamos? La recomendación estratégica es tratar los servidores MCP como **productos de plataforma** (capítulo 3): un catálogo interno, con propietarios, niveles de servicio y revisión de seguridad, del que todos los equipos consumen. La alternativa (cada equipo improvisando sus integraciones) reconstruye exactamente el problema M×N que el protocolo vino a eliminar.

Herramientas MCP limpias

Las cuatro reglas de seguridad anteriores tienen su contraparte de diseño en el Código Limpio Sintético (capítulo 4), porque un servidor MCP es, ante todo, una **interfaz pública**: y las interfaces públicas se diseñan para sus consumidores, que aquí son dos: el modelo (que lee las descripciones) y el ingeniero (que mantiene el servidor):

- **Nombres que se autoinvocan.** `consultar_estado_ticket` se elige correctamente entre veinte herramientas; `query_v2` exige que el modelo adivine. En catálogos grandes, la calidad de los nombres es la diferencia entre un agente preciso y uno errático.
- **Una herramienta, una acción.** La tentación de un `ejecutar_operacion(tipo, parametros)` genérico viola SRP y, peor, destruye la capacidad del arnés de aplicar permisos por acción: no se puede dar «solo lectura» a una herramienta que hace de todo.
- **Errores para el modelo, no para el log.** «ERROR: Solo se permite una única query SELECT. Para modificar datos, usa la API REST», el mensaje del ejemplo, enseña al modelo el camino correcto en la misma respuesta. Un `400 Bad Request` lo deja reintentando a ciegas, quemando presupuesto.
- **El esquema es el contrato.** Tipos estrictos, `enum` donde el dominio es cerrado, `required` mínimo: cada grado de libertad in-

necesario en el esquema es una forma nueva de invocación incorrecta que tendrá que depurar.

Ejercicio resuelto 9.1: añada la cuarta herramienta

Enunciado. Añada al servidor del ejemplo una herramienta `crear_ticket` que permita al agente abrir tickets de incidencia. Diseñe sus guardarraíles: ¿qué puede salir mal y cómo lo impide el servidor?

Solución comentada. La definición sigue la plantilla de las anteriores; lo interesante son los guardarraíles, porque esta es la primera herramienta **de escritura** del servidor:

```
Tool(
    name="crear_ticket",
    description="""Crea un ticket de incidencia en el sistema de
gestión.
    USAR cuando el usuario reporte un problema reproducible y pida
registrarlo.
    NO USAR para: consultas (usar consultar_estado_ticket), ni para
crear
    más de un ticket por incidencia.""",
    inputSchema={
        "type": "object",
        "properties": {
            "titulo": {"type": "string", "maxLength": 120},
            "descripcion": {"type": "string", "maxLength": 4000},
            "severidad": {"type": "string",
                          "enum": ["low", "medium", "high"]},
        },
        "required": ["titulo", "descripcion", "severidad"],
    },
)
```

Y en el manejador, las defensas que el esquema no puede expresar: (1) **idempotencia**: un hash de `titulo+descripcion` con ventana de 10 minutos rechaza duplicados, porque un agente en bucle de reintentos puede crear cincuenta tickets idénticos (el Token-Burn tiene primos); (2) **límite de tasa por sesión**: N tickets por hora por agente, presupuesto como control de seguridad (capítulo 4 (Vol. 4)); (3) **severidad `critical` excluida del enum**: escalar a crítico exige un humano: es la política de acciones de alto impacto del capítulo 1 (Vol. 3) aplicada en el lugar correcto, el servidor; (4) **autoría trazada**: el

ticket registra que lo creó un agente, con el id de sesión: auditoría total. Obsérvese que ninguna de las cuatro defensas vive en el prompt: la descripción *sugiere* el buen uso; el servidor lo *impone*.

El protocolo de admisión al catálogo

La pregunta de discusión con la que suele cerrarse esta conversación (¿con qué criterios entra un servidor MCP al catálogo?) no debería quedar abierta en una organización que ya opera en producción. Este libro propone un protocolo de puntuación, verificable en una revisión de una hora, que unifica las reglas de seguridad y de diseño ya vistas en este capítulo con el registro de dependencias y el presupuesto de degradación del Vol. 4 (capítulo 5, caso 8):

Criterio	0 puntos	1 punto	2 puntos
Mantenimiento	Sin dueño identificable	Mantenedor individual	Organización o proveedor con incentivo de largo plazo (capítulo 2)
Seguridad del esquema	Sin guardarraíles en el servidor	Guardarraíles parciales	Las cuatro defensas de este capítulo: idempotencia, límite de tasa, exclusión de acciones críticas del enum, autoría trazada
Versionado del contrato	Sin versión declarada	Versión sin política de deprecación	Semver explícito + ventana de deprecación documentada + contrato dirigido por el consumidor (Vol. 4, caso 8)

Criterio	0 puntos	1 punto	2 puntos
Trazabilidad	Sin registro	Log propio no estandarizado	Traza compatible con el esquema Open Telemetry (Vol. 4, capítulo 11)
Plan de degradación	Sin plan	Solo falla dura	Modo degradado documentado y probado en stage (Vol. 4, caso 8)

Umbrales de decisión: 7 a 10 puntos admite el servidor sin reservas; 4 a 6 lo admite únicamente en modo *shadow* de solo lectura en QA (la misma tabla de entornos del Vol. 4, caso 8) hasta elevar el puntaje; menos de 4 no se admite al catálogo, sin excepción de urgencia, la excepción de urgencia es, precisamente, cómo se acumula la deuda de gobernanza que el capítulo 1 (Vol. 3) mide en dólares.

Versionado del contrato, en concreto. Un esquema de versionado semántico específico para herramientas MCP evita la ambigüedad de «actualizamos el servidor» sin más contexto: **MAJOR** cuando se elimina una herramienta o se rompe su `inputSchema` (exige mínimo 90 días con ambas versiones activas en paralelo); **MINOR** cuando se añade una herramienta o un campo opcional (retrocompatible, sin ventana obligatoria); **PATCH** cuando solo cambia la redacción de una descripción para mejorar la invocación del modelo (capítulo 2, prompt de la herramienta). El equipo consumidor, no el equipo dueño del servidor, decide cuándo migrar de una versión MAJOR a la siguiente, el mismo principio de autonomía del consumidor que sostienen los contratos dirigidos por el consumidor.

Un catálogo que aplica este protocolo de forma consistente convierte «¿integramos la IA con el sistema X?» en un número reproducible, no en una negociación distinta cada vez que un equipo lo pregunta.

Lo esencial del capítulo 9

- MCP reduce la integración de M×N a M+N: cada aplicación y cada sistema implementan el protocolo una vez Anthropic, 2024b.
- Tres primitivas: tools (acciones), resources (lecturas), prompts (plantillas), sobre JSON-RPC 2.0.
- El servidor impone, la descripción sugiere; los resultados de herramientas son entrada no confiable Greshake et al., 2023.
- Herramientas limpias: nombres autoinvocables, una acción por herramienta, errores que enseñan, esquemas estrictos.
- Los servidores MCP se gestionan como productos de plataforma: catálogo, dueños, revisión de seguridad.
- El protocolo de admisión al catálogo puntúa mantenimiento, seguridad del esquema, versionado, trazabilidad y plan de degradación (0-10); por debajo de 4 no se admite, sin excepción de urgencia.

Preguntas para discusión

1. De los diez sistemas internos más consultados por sus equipos (tickets, métricas, documentación, CRM…), ¿cuántos son hoy accesibles para un agente de forma gobernada?
2. ¿Quién aprueba, en su organización, que un nuevo servidor MCP entre al catálogo? El protocolo de admisión de este capítulo da una respuesta reproducible: ¿su organización podría puntuar hoy sus cinco servidores MCP más usados contra esa tabla?

Capítulo 10: La Arquitectura del Contexto

El corpus de instrucciones también es código

Los capítulos anteriores de esta parte construyeron las piezas del sistema agéntico: el agente (capítulo 4), los roles (capítulo 5), las fronteras de dominio (capítulo 6), los skills (capítulo 7), la orquestación (capítulo 8) y las herramientas (capítulo 9). Falta la pieza que casi nadie diseña y que, sin embargo, gobierna a todas: el **corpus de instrucciones** que el equipo le da a la IA para que entienda *qué* hace el sistema y *cómo* debe proceder. Hoy ese corpus tiene mil nombres y ninguna arquitectura (`CLAUDE.md`, `AGENTS.md`, `ADR.md`, `spec.md`, `.cursorrules`, `copilot-instructions.md`, una docena de READMEs con reglas) y vive disperso en carpetas o suelto en la raíz, creciendo por acreción.

El capítulo 4 dejó sentada la tesis que ahora cobra su consecuencia más amplia: los artefactos de la era agéntica son *texto leído por un intérprete falible*, y eso es lo que siempre fue el código. Si el corpus de instrucciones es texto que un intérprete ejecuta, entonces **el corpus de instrucciones es código**, y merece la misma arquitectura que cualquier otro código. Hoy no la tiene. Lo que en un proyecto pequeño es un único archivo legible se convierte, cuando el equipo crece, en un monolito de miles de líneas donde nadie sabe ya si *ya se le dijo* a la IA que vigile cierto error, si esa regla sigue vigente o si contradice otra escrita tres meses atrás por alguien que ya no está. El que hace el onboarding del proyecto, humano o agente, hereda la peor de las deudas: la que no se ve hasta que se la pisa.

Y aquí está el giro que eleva el problema de incomodidad a de-

fecto medible: **el monolito de contexto no solo cansa al humano; degrada al modelo**. La evidencia es la misma que el capítulo 4 usó para la ventana de contexto: el rendimiento de los modelos cae cuando la información relevante está enterrada en el medio de un contexto largo, con la conocida curva en U Liu et al., 2024. Un archivo de reglas de cinco mil líneas es, literalmente, enterrar. La instrucción crítica que en un archivo de cincuenta líneas el modelo respeta, en uno de cinco mil la pierde, no por desobediencia, sino por física del contexto. El corpus inflado es, además, la **causa número uno del Token-Burn** que el capítulo 1 (Vol. 4) cuantifica: si cada invocación arrastra el monolito entero «por contexto», se paga el prefijo completo en cada iteración. Organizar el corpus para cargar solo el dominio relevante no es higiene estética: es la cura simultánea de un problema de calidad y de un problema de costo.

La regla de dependencia aplicada a los archivos `.md`

La solución no hay que inventarla: lleva décadas escrita. Parnas (1972) enseñó que un sistema se descompone en módulos ocultando dentro de cada uno las decisiones que pueden cambiar; Evans (2003) dio a esos módulos una frontera con sentido de negocio, el *bounded context*; Martin (2017) los ordenó con una regla única, las dependencias apuntan hacia adentro; y Conway (1968) advirtió que la estructura del artefacto terminará reflejando la del equipo, lo queramos o no. **La Arquitectura del Contexto** es, sencillamente, aplicar esas cuatro lecciones al corpus de instrucciones en lugar de al código fuente.

En la práctica, tres capas:

- **El núcleo (reglas transversales).** Lo que vale para todo el sistema y rara vez cambia: el lenguaje ubicuo, los invariantes de seguridad (capítulo 4 (Vol. 4)), las convenciones de estilo, la política de acciones irreversibles. Es pequeño, estable y se lee entero. Es el anillo interior: ningún dominio lo modifica, todos lo respetan.

- **Los dominios (bounded contexts).** Una carpeta por dominio de negocio (`facturacion/`, `inventario/`, `identidad/`), y dentro, solo las reglas, especificaciones y decisiones (ADRs) de *ese* dominio. La información se oculta donde pertenece Parnas, 1972: quien trabaja en facturación no carga ni lee las reglas de inventario. La frontera del archivo es la frontera del dominio.
- **El manifiesto (el gateway).** Un único índice en la raíz que no contiene reglas, sino el mapa: qué dominios existen, qué archivo gobierna cada uno y a quién pertenece. Es el equivalente, en la capa de conocimiento, del AI Gateway del capítulo 3: el punto de entrada que enruta. Un agente que llega a implementar un *feature* lee primero el manifiesto, descubre qué dominio toca y carga *solo* ese contexto.

```
contexto/
├── MANIFIESTO.md            # el gateway: mapa de dominios y dueños.
NO contiene reglas.
├── nucleo/                  # anillo interior: transversal y estable
│   ├── lenguaje-ubicuo.md
│   ├── seguridad.md         # invariantes no negociables (cap. 14)
│   └── estilo.md
├── facturacion/             # bounded context (cap. 6)
│   ├── reglas.md            # invariantes del dominio
│   ├── spec.md              # qué hace, no cómo
│   └── decisiones/          # ADRs locales al dominio
│       └── 2026-03-tarifas.md
├── inventario/
│   └── reglas.md
└── prompts-maestros/        # casos de uso de negocio, activables
(cap. 5 y 7)
    ├── alta-de-cliente.md
    └── conciliacion-mensual.md
```

La regla de dependencia se respeta tal cual: los dominios dependen del núcleo, nunca al revés; ningún dominio importa a otro (si dos dominios necesitan compartir una regla, esa regla pertenece al núcleo, no a un dominio que el otro espía). Y el manifiesto, como todo buen *gateway*, no sabe nada de las reglas que enruta: solo dónde viven. Quien haya leído el capítulo 6 reconocerá que esto no es más que DDD (el mismo que actúa de sistema inmunológico para el código generado) aplicado un nivel más arriba: a las instrucciones que generan ese código.

Agentes guardianes y prompts maestros

La estructura de archivos es la mitad del patrón; la otra mitad es quién la cuida y cómo se activa. Aquí la Parte II se cierra sobre sí misma.

Un agente guardián por dominio. Cada *bounded context* tiene un skill especialista (capítulo 7) cuyo único trabajo es velar por la coherencia de *su* carpeta: detectar reglas duplicadas, contradictorias u obsoletas, y rechazar en revisión cualquier cambio que viole los invariantes del núcleo. Es el patrón council del capítulo 8 puesto al servicio del conocimiento: el guardián de facturación no opina sobre inventario, y por eso opina bien sobre facturación. Cuando una regla nueva llega por *pull request* (porque el corpus se versiona y se revisa como código, no se edita a mano en producción), el guardián del dominio afectado es el primer revisor.

Un orquestador en el gateway. Sobre los guardianes vive un orquestador (capítulo 8, patrón jerárquico) que habita el dominio del manifiesto. No conoce las reglas de cada dominio, ni debe; conoce el mapa. Ante una petición de negocio («implementa el descuento por volumen»), lee el manifiesto, identifica que toca `facturacion/`, delega en su guardián y carga solo ese contexto. El orquestador es a los dominios de conocimiento lo que el AI Gateway (capítulo 3) es a los proveedores de modelo: sabe a quién llamar sin saber cómo hace su trabajo el llamado.

La carpeta de prompts maestros. Por último, los casos de uso que el negocio pide repetidamente no se reescriben cada vez: viven versionados en `prompts-maestros/`, cada uno como una plantilla que el capítulo 5 ya describió (una estructura, muchos parámetros) y que activa un flujo completo con todo su contexto acotado. «Alta de cliente» o «conciliación mensual» dejan de ser instrucciones improvisadas en un chat y se vuelven artefactos de ingeniería: con dueño, con historial y con la suite de evals (capítulo 3 (Vol. 4)) que certifica que siguen funcionando cuando el modelo o las reglas cambian.

Caso de estudio: el onboarding que dejó de leer miles de líneas

(*Escenario ilustrativo compuesto, fiel a un patrón recurrente.*)

Un equipo de plataforma acumuló, en dieciocho meses, un `CLAUDE.md` de casi cuatro mil líneas: reglas de seis dominios, decisiones fosilizadas, tres convenciones de nombres que se contradecían y un par de instrucciones que apuntaban a un servicio ya apagado. Cada ingeniero nuevo (y cada agente) lo cargaba entero. Los síntomas eran los del capítulo 1 (Vol. 4) y los del capítulo 4 a la vez: el costo por iteración subía sin que la calidad lo hiciera, y el agente «olvidaba» reglas que estaban escritas, pero enterradas en la línea 2.700.

La refactorización no tocó una sola línea de código de producto: reorganizó el corpus según la Arquitectura del Contexto. Lo transversal y vigente subió al núcleo; cada dominio recibió su carpeta; las decisiones muertas se archivaron con fecha en lugar de borrarse (la regla del Boy Scout del capítulo 4, aplicada al conocimiento); y un manifiesto de treinta líneas reemplazó al portón de cuatro mil. El resultado, medido con la propia instrumentación del equipo: el onboarding dejó de ser «lee todo y reza» y pasó a «el agente carga el dominio del feature»; el contexto medio por invocación cayó de miles a cientos de líneas, con su ahorro directo de tokens (capítulo 1 (Vol. 4)); y la pregunta que abría este capítulo (*¿ya se le dijo a la IA que vigile este error?*) pasó de ser una arqueología de media hora a un `grep` en una carpeta.

Límites y buenas prácticas

La honestidad que el libro exige obliga al reverso. **Esta arquitectura no es gratis ni siempre conviene.** Para un proyecto pequeño o en sus primeras semanas, un único archivo legible es superior: imponerle bounded contexts a un sistema que cabe en una cabeza es la sobre-ingeniería que el capítulo 4 advierte: la *god class* tiene un gemelo opuesto, igual de dañino, en la fragmentación prematura. El patrón se justifica cuando el corpus deja de caber en una lectura, cuando varios equipos lo editan en paralelo, o cuando el costo del

contexto inflado empieza a doler en la factura. Tres buenas prácticas gobiernan su adopción:

- **El manifiesto nunca contiene reglas, solo el mapa.** En cuanto el gateway empieza a acumular lógica, deja de ser un gateway y se vuelve un segundo monolito. Si crece, es señal de que falta un dominio.
- **Una regla, un dueño, un lugar.** Lo único peor que una regla enterrada es la misma regla escrita en dos dominios con dos redacciones distintas. La fuente de verdad es única (DRY, capítulo 4); lo compartido sube al núcleo.
- **El corpus se versiona y se revisa como código.** Un cambio de regla sin revisión es un *commit* directo a la rama principal (capítulo 4); el guardián del dominio es su primer revisor y la suite de evals (capítulo 3 (Vol. 4)), su prueba de regresión.

Como toda la Parte II, el principio es lo estable y la herramienta lo efímero: los nombres de archivo de hoy (`CLAUDE.md`, `AGENTS.md`) cambiarán; la idea de que el corpus de instrucciones es arquitectura, y que se gobierna con fronteras, dueños y una regla de dependencia, sobrevivirá a todos ellos.

Memoria del Agente: el Ciclo de Vida Observable

La Arquitectura del Contexto se completa cuando el agente mismo se vuelve observable. A través de toda su ejecución, un agente recorre estados (inactivo, procesando, aguardando, completado) cada transición es un evento que el equipo debe poder monitorear, testear y corregir. Comprender estos estados no es un lujo de observabilidad; es el fundamento de la ingeniería: no puedes reparar o optimizar lo que no ves.

> **Justificación de diseño: diagrama de estados para comportamiento dinámico**
>
> El ciclo de vida del agente es un proceso que cambia en el tiempo: nace inactivo, se activa, herramientas la bloquean, se completa o falla. Un diagrama de estados captura exactamente ese *cambio temporal*: transiciones claras,

eventos que las disparan, los tiempos en los que el agente puede ser monitoreado o interrumpida.

Un diagrama de clases mostraría qué objetos existen; una secuencia mostraría quién llama a quién. Aquí necesitamos el *qué pasa en el tiempo*, la dimensión que importa para observabilidad, optimización y debugging.

El diagrama de estados reaparece más adelante en el **Capítulo 7 (Vol. 4)** (cuando el agente aguarda un evento en el log), donde de nuevo el tiempo y la transición son lo que interesa.

Figura 18: Diagrama 10.1: Ciclo de vida del Agente: máquina de estados con transiciones observables

Cada estado tiene una instrumentación natural: - **Inactivo → Activo:** el agente recibió un prompt. Registra marca de tiempo, costo inicial. - **Activo → LlamadaHerramienta:** LLM propuso una herramienta. Registra cuál y parámetros. - **LlamadaHerramienta → Aguardando:** la herramienta se envió. Comienza medición de latencia. - **Aguardando → Activo:** resultado disponible. Latencia registrada, siguiente iteración. - **Activo → Completado:** respuesta final sin herramientas. Ciclo cerrado, costo registrado. - **Activo/Aguardando → Error:** invariante violada, timeout agotado. Clave para patrones de fallo.

La observabilidad en tiempo de ejecución (capítulo 8 (Vol. 4)) recolecta estas transiciones. Los evals (capítulo 3 (Vol. 4)) verifican que el agente no queda atrapado en un estado, p.ej. Aguardando > N segundos sugiere herramienta lenta o agente bloqueada. El *gateway* (capítulo 3) usa estos estados para tomar decisiones: si el agente entra en Error, escala humana; si Completado dentro del presupuesto, registra éxito.

Lo esencial del capítulo 10

- El corpus de instrucciones (los `*.md` que la IA lee para saber qué y cómo) es código: texto leído por un intérprete falible, y

merece arquitectura.

- El monolito de contexto degrada al modelo (la información enterrada se usa peor; Liu et al., 2024) y es la causa #1 de TokenBurn (capítulo 1 (Vol. 4)). Es un defecto medible, no una incomodidad.
- La Arquitectura del Contexto aplica gigantes conocidos a los archivos: ocultación de información Parnas, 1972, bounded contexts (Evans, 2003), regla de dependencia Martin, 2017, ley de Conway (1968). Tres capas: núcleo, dominios, manifiesto-gateway.
- Un guardián especialista por dominio (capítulo 7), un orquestador en el gateway que enruta sin conocer las reglas (capítulos 3 y 8), y `prompts-maestros/` versionados como casos de uso de negocio (capítulo 5).
- No siempre conviene: en proyectos pequeños un solo archivo gana. El patrón se justifica cuando el corpus no cabe en una lectura, varios equipos lo editan o el contexto inflado duele en la factura.

Estado del Arte: Sistemas Multi-Agente basados en LLM y el Rol del AI Engineer

Revisión crítica de literatura indexada (2020–2026), elaborada como respaldo académico del Volumen 1: Fundamentos

1. Alcance y método de esta revisión

Este estado del arte no sustituye una revisión sistemática formal con protocolo PRISMA registrado; es una síntesis crítica dirigida a sostener académicamente la tesis del libro, construida sobre revisiones sistemáticas ya publicadas y verificables en bases indexadas (ACM Digital Library, IEEE Xplore, arXiv como fuente de preprints en un campo de publicación acelerada) más un muestreo de estudios primarios citados por esas revisiones. Cuando una afirmación proviene de una SLR publicada, se cita la SLR; cuando proviene de un estudio primario, se cita el estudio primario. No se fabricó ninguna cifra ni hallazgo: cada dato de esta sección es verificable en la fuente citada.

2. Líneas de investigación principales

Línea 1: LLMs aplicados a tareas de ingeniería de software (LLM4SE). Es la línea más consolidada y la de mayor volumen de literatura. La revisión sistemática de referencia (Hou et al., 2024,

ACM TOSEM) analizó 395 artículos publicados entre enero de 2017 y enero de 2024, extraídos de siete bases indexadas, y documenta un crecimiento sostenido de publicaciones desde 2023 impulsado por la adopción industrial de LLMs junto con sistemas agénticos autónomos, el mismo punto de inflexión que este libro fecha en el capítulo 1.

Línea 2: Sistemas multi-agente basados en LLM para ingeniería de software (LMA4SE). Es la línea que sostiene directamente los capítulos 5 y 8 de este volumen. He, Treude y Lo (2025, *ACM TOSEM* 34(5)) publicaron la revisión sistemática de referencia sobre LMA aplicados a las distintas fases del ciclo de vida del software, mapeando el panorama actual de aplicaciones. Es consistente con Ramírez-Rueda et al. (CONISOFT, 2024), que estudian específicamente la integración de sistemas multi-agente y LLMs para generación automática de código, y con la línea de trabajo de Rasheed, Waseem y colegas (Universidad de Tampere), autores de CodePori y de un artículo de posición temprano sobre agentes autónomos en desarrollo de software (2023).

Línea 3: Patrones de prompting e ingeniería de contexto. Sasaki et al. (COMPSAC, 2024) realizaron una revisión sistemática específica de patrones de *prompt engineering* en ingeniería de software, un antecedente directo del capítulo 10 de este volumen (Arquitectura del Contexto).

Línea 4: Productividad, adopción y percepción del desarrollador. Dos SLR de 2025 abordan el lado humano: Ferino, Hoda, Grundy y Treude estudian la perspectiva de desarrolladores novatos ante la adopción de LLMs; Mohamed, Assi y Guizani revisan sistemáticamente el impacto de los asistentes LLM en la productividad del desarrollador. Esta línea sostiene el capítulo 2 (El AI Engineer) y se conecta con hallazgos organizacionales más recientes de literatura gris de alta credibilidad (Bell, 2026, *Scaling AI Adoption in Engineering*, O'Reilly: no es literatura académica indexada, pero sí una síntesis de cientos de entrevistas con CTOs, citada aquí como antecedente práctico, no como evidencia científica).

Línea 5 (Costo, eficiencia energética y sostenibilidad de LLMs en producción. Shi, Yang y Lo (2025, *ACM TOSEM* 34(5)) revisan la literatura sobre LLMs eficientes y "verdes" en ingeniería de soft-

ware) antecedente directo del capítulo 12 del Vol. 4 de esta serie, mencionado aquí porque ya empieza a emerger como preocupación en la literatura de 2024–2025, no solo como intuición de practicante.

3. Marcos teóricos utilizados en la literatura

La literatura revisada no converge en una única teoría, sino en una combinación de tres marcos que rara vez se citan juntos explícitamente, aunque coexisten en la práctica:

- **Marcos de arquitectura de software clásica** (Fowler, Martin, Parnas), aplicados por analogía a agentes, el propio Vol. 1 hace esto explícitamente en el capítulo 4, pero la literatura académica revisada (Hou et al., 2024; He et al., 2025) rara vez cita estas fuentes fundacionales de ingeniería de software; tiende a tratar los sistemas LMA como un objeto de estudio nuevo, sin anclarlo en la disciplina de arquitectura de software de las últimas cinco décadas. **Esta es una discrepancia real que vale la pena señalar**: la literatura de LMA4SE es rica en descripción empírica de qué patrones de coordinación existen (secuencial, jerárquico, debate/consenso, la misma taxonomía del capítulo 8), pero pobre en anclarlos a principios de diseño ya establecidos (acoplamiento, cohesión, separación de responsabilidades). Este libro sí hace esa conexión explícita; es una contribución diferenciadora, no solo una síntesis de lo ya publicado.
- **Marcos de razonamiento y planificación** (ReAct de Yao et al., 2023; Tree of Thoughts de Yao et al., 2023; Reflexion de Shinn et al., 2023), que la literatura sí trata como fundacionales y que este volumen también adopta en el capítulo 4 original del libro completo (patrones fundamentales, ahora distribuido entre los volúmenes).
- **Marcos de agentes generativos con memoria y comportamiento social** (Park et al., 2023, *Generative Agents*), influyentes en la línea de investigación pero con una aplicación práctica en ingeniería de software todavía limitada según Hou et al. (2024): la mayoría de los estudios primarios de la SLR se concentran en generación y reparación de código, no en simulación social de agentes.

4. Metodologías aplicadas en los estudios primarios

Predominan tres familias metodológicas en la literatura revisada:

1. **Revisión sistemática de literatura (SLR) con protocolo Kitchenham/Keele.** Es el estándar de facto en LLM4SE: Hou et al. (2024) sigue explícitamente las guías de Kitchenham y Keele (2007) y de Kitchenham et al. (2009) para SLR en ingeniería de software. La mayoría de las revisiones citadas en esta sección adoptan el mismo protocolo.
2. **Estudios de *benchmark*** (SWE-bench de Jimenez et al., 2024, citado también en nuestro propio Vol. 4), miden capacidad de resolución de tareas reales de GitHub, no percepción ni adopción.
3. **Estudios cualitativos de percepción** (entrevistas, encuestas), la línea de Ferino et al. (2025) y Mohamed et al. (2025), metodológicamente más débil en poder de generalización pero más cercana a las preguntas organizacionales que aborda el Vol. 3 de esta serie.

Limitación metodológica recurrente, señalada por los propios autores: Hou et al. (2024) reconocen explícitamente que incluyeron artículos de arXiv sin revisión por pares para 2023–2024, dado que es un campo de publicación acelerada, aplicando un proceso de evaluación de calidad para filtrarlos. Es una decisión razonable dada la velocidad del campo, pero introduce un sesgo hacia resultados aún no validados por pares, algo que este libro también asume al citar código y herramientas de 2026 que no han pasado por revisión académica formal (Claude Agent SDK, MCP, Archify). La honestidad exige señalarlo en ambos casos.

5. Hallazgos recurrentes

- El crecimiento de la literatura sobre LMA4SE es reciente y abrupto: prácticamente toda la producción académica sustancial sobre sistemas *multi*-agente (a diferencia de agentes individuales) se concentra en 2023–2026, lo cual es consistente

con la periodización que este libro hace en el capítulo 1 (el "despertar agéntico").
- Los patrones de coordinación identificados empíricamente en la literatura (He et al., 2025) coinciden, en líneas generales, con los cinco patrones canónicos del capítulo 8 de este volumen, lo cual sugiere que la taxonomía de este libro no es idiosincrática, sino consistente con lo que la comunidad académica está documentando de forma independiente.
- La literatura de productividad (Mohamed et al., 2025) reporta resultados mixtos y dependientes del contexto, no una mejora universal, un hallazgo que refuerza, en vez de contradecir, la tesis de este libro de que la arquitectura y el criterio humano determinan el resultado, no el modelo en sí.

6. Discrepancias entre autores

La discrepancia más señalable no es de datos, sino de encuadre: una parte de la literatura (particularmente los estudios de *benchmark* puro) trata la capacidad del modelo como la variable explicativa central del éxito de un sistema agéntico. Otra parte, más cercana a la línea de adopción organizacional (Ferino et al., 2025; Mohamed et al., 2025) y a la literatura de ingeniería de software clásica que este libro invoca, sitúa el proceso, la arquitectura y la organización como la variable que realmente explica la varianza en los resultados. Este libro toma partido explícito por la segunda posición, y lo hace con el respaldo de que la evidencia de productividad "mixta y dependiente del contexto" es más compatible con una explicación organizacional/arquitectónica que con una puramente relacionada con la capacidad del modelo.

7. Limitaciones metodológicas de la literatura revisada

- **Sesgo hacia código abierto y tareas verificables automáticamente.** La mayoría de los estudios primarios de las SLR citadas evalúan generación o reparación de código en repositorios pú-

blicos con pruebas automatizadas (el mismo sesgo que afecta a SWE-bench). Los sistemas agénticos en dominios de misión crítica o con datos privados, el foco del "caso sectorial" del capítulo 5 (Vol. 4) de esta serie, están sistemáticamente subrepresentados en la literatura académica indexada.

- **Ventana temporal corta.** Toda la literatura sobre LMA4SE es posterior a 2023; no existe todavía perspectiva longitudinal sobre mantenibilidad o deuda técnica de sistemas agénticos a varios años de operación, una brecha que este libro solo puede señalar, no resolver, dado que la industria misma no tiene ese historial todavía.
- **Escasa reproducibilidad.** Varias de las revisiones citadas señalan que un porcentaje significativo de los estudios primarios no publica código ni datos de forma reproducible, una limitación estructural del campo, no de esta síntesis en particular.

8. Brechas de conocimiento

1. **Ausencia de anclaje explícito en arquitectura de software clásica** (sección 3): una brecha que este libro busca cerrar activamente, no solo señalar.
2. **Falta de estudios longitudinales sobre gobernanza y auditoría de enjambres en producción**: el Índice de Autonomía Operativa y la auditoría de reproducción cruzada propuestos en el Vol. 3 de esta serie no tienen, hasta donde esta revisión pudo verificar, un antecedente directo publicado en literatura indexada, son una contribución original de este libro, no una síntesis de lo existente, y se presentan como tal.
3. **Escasa investigación sobre el rol emergente del "AI Engineer"** como categoría profesional distinta del ML Engineer: la literatura académica revisada trata la productividad del desarrollador de forma agregada, sin la diferenciación de roles que el capítulo 2 de este volumen propone.

9. Relación con la tesis central del libro

La tesis de este libro (que la arquitectura y los patrones de ingeniería perduran más allá del modelo de turno, y que la responsabilidad de lo irreversible se mantiene humana) encuentra respaldo indirecto pero consistente en la literatura revisada: el hallazgo recurrente de resultados de productividad "mixtos y dependientes del contexto" (sección 5) es exactamente lo que predeciría una tesis centrada en arquitectura y no en capacidad del modelo, y es difícil de explicar bajo una tesis centrada solo en el modelo. La literatura no *prueba* la tesis de este libro (ninguna de las SLR citadas fue diseñada para probarla), pero tampoco la contradice, y en varios puntos (patrones de coordinación, crecimiento abrupto post-2023, sesgo hacia tareas verificables) la literatura documenta empíricamente fenómenos que este libro interpreta arquitectónicamente.

Referencias

Anthropic (2024). *Model Context Protocol and related technical documentation*. *As cited in the manuscript; multiple 2024 documents (a/b/c)*.

Bender, Emily M.; Gebru, Timnit; McMillan-Major, Angelina; Shmitchell, Shmargaret (2021). *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?*. Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT).

Bommasani, Rishi; Hudson, Drew A.; Adeli, Ehsan; others (2021). *On the Opportunities and Risks of Foundation Models*. Stanford Center for Research on Foundation Models (CRFM).

Brown, Tom B.; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; others (2020). *Language Models Are Few-Shot Learners*. Advances in Neural Information Processing Systems (NeurIPS).

Chase, Harrison (2025). *As cited in the manuscript (LangChain)*.

Chen, Mark; Tworek, Jerry; Jun, Heewoo; Yuan, Qiming; others (2021). *Evaluating Large Language Models Trained on Code*. *arXiv preprint (Codex)*.

Conway, Melvin E. (1968). *How Do Committees Invent?*. Datamation.

Du, Yilun; Li, Shuang; Torralba, Antonio; Tenenbaum, Joshua B.; Mordatch, Igor (2023). *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. *arXiv preprint*.

Evans, Eric (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Ferber, Jacques (1999). *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley.

Fowler, Martin (2018). *Refactoring: Improving the Design of Existing Code*. (2nd ed.) Addison-Wesley.

Greshake, Kai; Abdelnabi, Sahar; Mishra, Shailesh; Endres, Christoph; Holz, Thorsten; Fritz, Mario (2023). *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. *arXiv preprint*.

Hong, Sirui; Zheng, Xiawu; Chen, Jonathan; Cheng, Yuheng; others (2024). *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. *ICLR*.

Ji, Ziwei; Lee, Nayeon; Frieske, Rita; others (2023). *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys.

Jimenez, Carlos E.; Yang, John; Wettig, Alexander; others (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. *ICLR*.

Kapoor, Sayash; Stroebl, Benedikt; Siegel, Zachary S.; Nadgir, Arvind; Narayanan, Arvind (2024). *AI Agents That Matter*. *arXiv preprint*.

Khattab, Omar; Singhvi, Arnav; Maheshwari, Paridhi; others (2024). *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*. International Conference on Learning Representations (ICLR).

Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; others (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).

Liu, Nelson F.; Lin, Kevin; Hewitt, John; Paranjape, Ashwin; Bevilacqua, Michele; Petroni, Fabio; Liang, Percy (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics (TACL).

Madaan, Aman; Tandon, Niket; Gupta, Prakhar; Hallinan, Skyler; others (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. Advances in Neural Information Processing Systems (NeurIPS).

Martin, Robert C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

Martin, Robert C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Mitchell, Melanie; others (2025). *As cited in the manuscript*. *Bibliographic detail not fully verified; see manuscript context*.

NIST (2023). *AI Risk Management Framework 1.0*. National Institute of Standards and Technology.

OpenAI (2024). *Technical documentation, as cited in the manuscript*.

Ouyang, Long; Wu, Jeffrey; Jiang, Xu; Almeida, Diogo; others (2022). *Training Language Models to Follow Instructions with Human Feedback*. Advances in Neural Information Processing Systems (NeurIPS).

Packer, Charles; Fang, Vivian; Patil, Shishir G.; Lin, Kevin; Wooders, Sarah; Gonzalez, Joseph E. (2024). *MemGPT: Towards LLMs as Operating Systems*. *arXiv preprint*.

Park, Joon Sung; O'Brien, Joseph C.; Cai, Carrie J.; Morris, Meredith Ringel; Liang, Percy; Bernstein, Michael S. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. *arXiv preprint*.

Parnas, David L. (1972). *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM.

Peng, Baolin; others (2023). *As cited in the manuscript*. *Bibliographic detail not fully verified; see manuscript context*.

Qian, Chen; Cong, Xin; Yang, Cheng; others (2024). *ChatDev: Communicative Agents for Software Development*.

Russell, Stuart; Norvig, Peter (2020). *Artificial Intelligence: A Modern Approach*. (4th ed.) Pearson.

Schick, Timo; Dwivedi-Yu, Jane; Dess'i, Roberto; Raileanu, Roberta; others (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. Advances in Neural Information Processing Systems (NeurIPS).

Shinn, Noah; Cassano, Federico; Berman, Edward; Gopinath, Ashwin; Narasimhan, Karthik; Yao, Shunyu (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. Advances in Neural Information Processing Systems (NeurIPS).

Skelton, Matthew; Pais, Manuel (2019). *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press.

Snell, Charlie; Lee, Jaehoon; Xu, Kelvin; Kumar, Aviral (2024). *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*. *arXiv preprint*.

Uni'on Europea (2024). *Reglamento (UE) de Inteligencia Artificial (EU AI Act)*.

Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Lukasz; Polosukhin, Illia (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

Wang, Shawn (2023). *The Rise of the AI Engineer*. *Latent Space* (*blog*), *as cited in the manuscript.*

Wei, Jason; Wang, Xuezhi; Schuurmans, Dale; Bosma, Maarten; Ichter, Brian; Xia, Fei; Chi, Ed; Le, Quoc; Zhou, Denny (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems (NeurIPS).

Wooldridge, Michael (2009). *An Introduction to MultiAgent Systems*. (2nd ed.) Wiley.

Wu, Qingyun; others (2024). *As cited in the manuscript*. *Bibliographic detail not fully verified; see manuscript context.*

Yao, Shunyu; Yu, Dian; Zhao, Jeffrey; Shafran, Izhak; Griffiths, Thomas L.; Cao, Yuan; Narasimhan, Karthik (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. Advances in Neural Information Processing Systems (NeurIPS).

Yao, Shunyu; Zhao, Jeffrey; Yu, Dian; Du, Nan; Shafran, Izhak; Narasimhan, Karthik; Cao, Yuan (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR).

Zheng, Lianmin; Chiang, Wei-Lin; Sheng, Ying; Zhuang, Siyuan; others (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems (NeurIPS).

Ziegler, Daniel M.; others (2024). *As cited in the manuscript*. *Bibliographic detail not fully verified; see manuscript context.*

Ferino, S., Hoda, R., Grundy, J., & Treude, C. (2025). Novice developers' perspectives on adopting LLMs for software development: A systematic literature review. *arXiv preprint*.

He, J., Treude, C., & Lo, D. (2025). LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5).

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo,

D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 1–79.

Kitchenham, B., Brereton, O. P., Budgen, D., Turner, M., Bailey, J., & Linkman, S. (2009). Systematic literature reviews in software engineering, A systematic literature review. *Information and Software Technology*, 51(1), 7–15.

Mohamed, A., Assi, M., & Guizani, M. (2025). The impact of LLM-assistants on software developer productivity: A systematic literature review. *arXiv preprint*.

Ramírez-Rueda, R., Benítez-Guerrero, E., Mezura-Godoy, C., & Bárcenas, E. (2024). Transforming software development: A study on the integration of multi-agent systems and large language models for automatic code generation. *12th International Conference in Software Engineering Research and Innovation (CONISOFT)*, 11–20.

Rasheed, Z., Waseem, M., Kemell, K.-K., Xiaofeng, W., Duc, A. N., Systä, K., & Abrahamsson, P. (2023). Autonomous agents in software development: A vision paper. *arXiv preprint arXiv:2311.18440*.

Sasaki, Y., Washizaki, H., Li, J., Sander, D., Yoshioka, N., & Fukazawa, Y. (2024). Systematic literature review of prompt engineering patterns in software engineering. *IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, 670–675.

Shi, J., Yang, Z., & Lo, D. (2025). Efficient and green large language models for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5), 1–22.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.